ACADEMIC PROJECT REPORT ON

Smart Campus & Academic Suite

Submitted to partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

By

V. Sai Rishi	**25911A67B7**
Mummadisetty Advyth	**25911A67A3**
P.V.S Sebareesh	**25911A67C4**
K. Bharath Reddy	**25911A6784**

Under the Esteemed Guidance of

Ms. M. PRASANNA KUMARI

Assistant Professor



Department of Computer Science and Engineering (Data Science)

VIDYA JYOTHI INSTITUTE OF TECHNOLOGY

(Approved by AICTE, New Delhi & Affiliated to JNTUH, Hyderabad)

An Autonomous Institution

AZIZ NAGAR, C B POST, HYDERABAD-500075

2026-2027

VIDYA JYOTHI
INSTITUTE OF TECHNOLOGY
AN AUTONOMOUS INSTITUTION

Accredited by NAAC & NBA, Approved by AICTE New Delhi & Permanently Affiliated to JNTUH

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (DATA SCIENCE)

CERTIFICATE

This is to certify that the project titled "**Smart Campus & Academic Suite"** is being submitted by **V. Sai Rishi (25911A67B7), Mummadisetty Advyth (25911A67A3), P.V.S Sebareesh (25911A67C4), K. Bharath Reddy (25911A6784).** In partial fulfillment for the award of the Degree of Bachelor of Technology in **Computer Science & Engineering (Data Science)**, is a record of Bonafide work carried out by them under my guidance and supervision. These results embodied in this project report have not been submitted to any other University or Institute for the award of any degree.

Head of the Department
Dr.K.S.R.K.SARMA

Internal Guide
Ms. M. Prasanna Kumari
Assistant Professor

DECLARATION

We, **V. Sai Rishi, Mummadisetty Advyth, P.V.S Sebareesh and K. Bharath Reddy**, bearing Roll Numbers **25911A67B7, 25911A67A3, 25911A67C4 and 25911A6784** hereby declare that the project entitled, **"Smart Campus & Academic Suite"** submitted for the degree of Bachelor of Technology in **Computer Science and Engineering (Data Science)** is original and has been done by us and this work is not copied and submitted anywhere for the award of any degree.

Date:

Place: Hyderabad

V. Sai Rishi (25911A67B7)

Mummadisetty Advyth (25911A67A3)

P.V.S Sebareesh (25911A67C4)

K. Bharath Reddy (25911A6784)

ACKNOWLEDGEMENT

We express our sincere gratitude to our internal guide **Ms. M. Prasanna Kumari**, Assistant Professor, Department of Computer Science and Engineering (Data Science), Vidya Jyothi Institute of Technology, for the valuable guidance, encouragement and constant support given throughout this project.

We are grateful to **Dr.K.S.R.K Sarma**, Associate Professor and HOD department of CSE (DS), Vidya Jyothi Institute of Technology Hyderabad, for his timely cooperation and valuable suggestions while carrying out this work. It is his kindness that made us learn more from him.

We whole-heartedly convey our gratitude to Dean of Accreditations **Dr. A. Padmaja** for her constructive encouragement.

We would like to take this opportunity to express our gratitude to our principal **Dr.A.Srujana** for providing necessary infrastructure to complete this project.

We would like to thank our Parents and all the Faculty Members who have contributed to our progress through the course to come to this stage.

V. Sai Rishi	**(25911A67B7)**
Mummadisetty Advyth	**(25911A67A3)**
P.V.S Sebareesh	**(25911A67C4)**
K. Bharath Reddy	**(25911A6784)**

ABSTRACT

Academic records in many college departments are still kept in attendance registers and separate spreadsheets. The same data is typed several times, grades and attendance percentages are calculated by hand, students learn that they are short of attendance too late to recover, and nothing prevents duplicate enrollments or a teacher being booked into two classes at once.

Smart Campus & Academic Suite is a department-level academic management system built to solve these problems. All data is kept in one normalised (3NF) MySQL 8.0 database of eleven tables. The academic rules are enforced inside the database itself: CHECK, UNIQUE and foreign-key constraints reject invalid data, triggers compute totals and grades and record every change of marks in an audit table, views compute attendance percentages, the below-75% defaulter list and GPA, and a stored procedure declares the defaulters. Two applications share this database. A Django 5.2 web portal gives Admin, Faculty and Student users role-based pages in which each user sees and changes only what their role allows. A Java 17 engine, written with core object-oriented programming and raw JDBC, provides the same reports from the command line, including a multithreaded dashboard and CSV export, and is called safely from the portal through a validated subprocess bridge that exchanges JSON.

The system was developed following the software development life cycle: requirements were specified in an IEEE 830 style SRS, the design was documented with UML, data flow and entity relationship diagrams, and the implementation was verified with 29 JUnit tests, 71 Django tests and 78 documented unit, integration, white-box and black-box test cases, all of which pass. Ordinary pages respond in about 50-70 ms and engine reports in under one second on a laptop. The project shows that placing business rules in a shared database keeps several applications consistent, and it serves as one worked example for the DBMS, Java, Django and Software Engineering laboratories.

Keywords: academic management system, MySQL, triggers, Django, Java, JDBC, role-based access control, software engineering.

TABLE OF CONTENTS

S.No	Chapter Name	Page No
1	**Chapter 1: Introduction**	**1**
	1.1 Introduction to the project	1
	1.2 Background of the project	1
	1.3 Problem statement	2
	1.4 Motivation for the project	2
	1.5 Objectives of the project	2
	1.6 Scope of the project	3
	1.7 Existing system	3
	1.8 Limitations of the existing system	4
	1.9 Proposed system	4
	1.10 Advantages of the proposed system	4
	1.11 Organization of the report	5
2	**Chapter 2: Literature Review**	**6**
	2.1 Introduction	6
	2.2 Review of existing research papers	6
	2.3 Review of existing applications or systems	7
	2.4 Comparison of related systems	8
	2.5 Research gaps and limitations	8
	2.6 Proposed approach based on the literature review	8
	2.7 Summary	9
3	**Chapter 3: Software Requirements Specification (SRS)**	**10**
	3.1 Introduction	10
	3.2 Purpose of the software	11
	3.3 Functional requirements	12
	3.4 Non-functional requirements	12
	3.5 Hardware requirements	13
	3.6 Software requirements	13
	3.7 User requirements	14
	3.8 System constraints	14
	3.9 Use case specifications	14
	3.10 Feasibility study	17
4	**Chapter 4: System Analysis and Design**	**19**
	4.1 Introduction	19
	4.2 System Architecture	19
	4.3 High-level design	20
	4.4 Low-level design	22
	4.5 UML diagrams	22
	4.6 Data Flow Diagrams	38
	4.7 Entity Relationship Diagram (ERD)	40

S.No	Chapter Name	Page No
	4.8 Database design	41
	4.9 Database tables and attributes	43
	4.10 Input design	46
	4.11 Output design	46
	4.12 User interface design	47
	4.13 Module design	47
	4.14 Security and access control design	48
5	**Chapter 5: System Development and Implementation**	**49**
	5.1 Introduction	49
	5.2 Development environment	49
	5.3 Technologies used	49
	5.4 Programming languages	50
	5.5 Frameworks and libraries	50
	5.6 Database implementation	50
	5.7 Module-wise implementation	51
	5.8 Algorithm or pseudocode	52
	5.9 Important code snippets	53
	5.10 Implementation screenshots	56
	5.11 Integration of modules	63
	5.12 Challenges faced during development	64
	5.13 Summary	64
6	**Chapter 6: Software Testing**	**65**
	6.1 Introduction	65
	6.2 Testing objectives	65
	6.3 Testing methodology	65
	6.4 Unit testing	66
	6.5 Integration testing	66
	6.6 System testing	68
	6.7 Functional testing	68
	6.8 Performance testing	69
	6.9 Security testing	70
	6.10 User Acceptance Testing (UAT)	70
	6.11 Test cases and test results	71
	6.12 Defect identification and correction	78
	6.13 Test summary	78
7	**Chapter 7: Deployment and Installation**	**79**
	7.1 Introduction	79
	7.2 Deployment architecture	79
	7.3 Deployment environment	79
	7.4 System installation procedure	79
	7.5 Configuration details	80

S.No	Chapter Name	Page No
	7.6 Database setup	80
	7.7 Application deployment steps	81
	7.8 User access and credentials management	81
	7.9 User manual	81
	7.10 Deployment screenshots	82
	7.11 Deployment challenges	83
8	**Chapter 8: Maintenance and Future Enhancements**	**84**
	8.1 Introduction	84
	8.2 Types of maintenance	84
	8.3 Bug fixing and updates	84
	8.4 Performance improvement	85
	8.5 Security updates	85
	8.6 User feedback and enhancements	85
	8.7 Limitations of the current system	85
	8.8 Future enhancements	86
	8.9 Maintenance plan	86
9	**Chapter 9: Results and Discussion**	**87**
	9.1 Introduction	87
	9.2 Results obtained	87
	9.3 Output screenshots	87
	9.4 Performance evaluation	93
	9.5 Comparison with the existing system	94
	9.6 Discussion of results	94
	9.7 Achievement of project objectives	95
	9.8 Summary	95
10	**Chapter 10: Conclusion**	**96**
	10.1 Conclusion	96
	10.2 Summary of the project	96
	10.3 Contribution of the project	96
	10.4 Learning outcomes	96
	10.5 Scope for future work	97
11	**References**	**98**
12	**Appendix A: Software Configuration Management**	**100**
	A.1 Configuration identification	100
	A.2 Version control	100
	A.3 Baselines	101
	A.4 Change control	101
	A.5 Build and release management	102
	A.6 SCM tools	102
13	**Appendix B: Risk Management**	**103**
14	**Appendix C: Automated Test Log**	**104**

LIST OF FIGURES

S.No	Figure Number & Name	Page No
1	Figure 4.1: System architecture of the Smart Campus & Academic Suite	20
2	Figure 4.2: High-level design: three tiers and the modules in each (rotated to fit the page)	21
3	Figure 4.3: Use case diagram	23
4	Figure 4.4: Class diagram: Java domain model (package com.smartcampus.models)	24
5	Figure 4.5: Class diagram: checked exception hierarchy (package com.smartcampus.exceptions)	25
6	Figure 4.6: Class diagram: configuration and DAO layer of the Java engine (generic CrudDao, JDBC implementations) (rotated to fit the page)	26
7	Figure 4.7: Class diagram: service and CLI layers of the Java engine (rotated to fit the page)	27
8	Figure 4.8: Object diagram: one defaulter's CS301 record (values read from smart_campus_db)	28
9	Figure 4.9: Sequence diagram: Admin runs a Java report from the portal (rotated to fit the page)	29
10	Figure 4.10: Sequence diagram: faculty saves marks; the database grades and audits them (rotated to fit the page)	30
11	Figure 4.11: Collaboration (communication) diagram: admin enrolls a student	31
12	Figure 4.12: Activity diagram: faculty marks entry and grading	32
13	Figure 4.13: Activity diagram: from attendance to a declared defaulter	33
14	Figure 4.14: Statechart diagram: life cycle of a marks record	34
15	Figure 4.15: Statechart diagram: attendance status of a student	35
16	Figure 4.16: Component diagram	36
17	Figure 4.17: Deployment diagram (as run for this project)	37
18	Figure 4.18: Data flow diagram, level 0 (context diagram)	38
19	Figure 4.19: Data flow diagram, level 1	39
20	Figure 4.20: Data flow diagram, level 2: process 4.0 "Record marks and grade"	40
21	Figure 4.21: Entity relationship diagram (11 tables of smart_campus_db)	41
22	Figure 4.22: Login page	47
23	Figure 5.1: Student dashboard of a defaulter (25cse004) with the 75% warning	57
24	Figure 5.2: Student dashboard of a student in good standing (25cse001)	58
25	Figure 5.3: Faculty dashboard: own courses and weekly timetable (ananya.rao)	59

S.No	Figure Number & Name	Page No
26	Figure 5.4: Faculty course roster for CS301	59
27	Figure 5.5: Faculty attendance sheet for a date	60
28	Figure 5.6: Faculty marks sheet; totals and grades come from the database	61
29	Figure 5.7: Admin dashboard	62
30	Figure 5.8: Admin student list with search and pagination	62
31	Figure 5.9: Admin student form	63
32	Figure 5.10: Timetable clash refused by the trigger, shown as a readable message	63
33	Figure 7.1: Database build with init_db.py	80
34	Figure 7.2: Maven build of the engine JAR	81
35	Figure 7.3: Engine help output after deployment	82
36	Figure 7.4: Portal running after deployment (admin dashboard)	83
37	Figure 9.1: Admin defaulter list with the declare button	87
38	Figure 9.2: GPA report: weighted GPA next to the official CGPA	88
39	Figure 9.3: Enrollments list and enroll form	89
40	Figure 9.4: Marks audit history written by the AFTER UPDATE trigger	90
41	Figure 9.5: Delete confirmation page	90
42	Figure 9.6: Java engine dashboard run from the portal, with thread names and timings	91
43	Figure 9.7: Typed error card from the engine page	91
44	Figure 9.8: Access denied (403) page for a wrong role	92
45	Figure 9.9: Django admin over the domain tables and the four views	92
46	Figure 9.10: Engine dashboard in the terminal	93
47	Figure 9.11: Engine defaulter report in the terminal	93
48	Figure 9.12: Engine student lookup in the terminal	93

LIST OF TABLES

S.No	Table Number & Name	Page No
1	Table 1.1: User types and their functions in the proposed system	4
2	Table 2.1: Comparison of related systems with the proposed system	8
3	Table 3.1: User characteristics	10
4	Table 3.2: Functional requirements	12
5	Table 3.3: Non-functional requirements	12
6	Table 3.4: Hardware requirements	13
7	Table 3.5: Software requirements	13
8	Table 3.6: Use case specification UC1: Log in	14
9	Table 3.7: Use case specification UC2: View own academic record	15
10	Table 3.8: Use case specification UC3: Record attendance	15
11	Table 3.9: Use case specification UC4: Enter marks	15
12	Table 3.10: Use case specification UC5: Manage master data	16
13	Table 3.11: Use case specification UC6: Enroll student	16
14	Table 3.12: Use case specification UC7: Add timetable entry	16
15	Table 3.13: Use case specification UC8: Declare defaulters	17
16	Table 3.14: Use case specification UC9: Run engine report	17
17	Table 3.15: Cost estimate	18
18	Table 3.16: Project schedule (from the Git history)	18
19	Table 4.1: Java engine packages	22
20	Table 4.2: Django apps	22
21	Table 4.3: CASE and support tools	37
22	Table 4.4: Main functional dependencies	42
23	Table 4.5: Triggers in smart_campus_db	42
24	Table 4.6: Views, functions and procedure	43
25	Table 4.7: Table departments	43
26	Table 4.8: Table faculty	43
27	Table 4.9: Table students	43
28	Table 4.10: Table courses	44
29	Table 4.11: Table enrollments	44
30	Table 4.12: Table attendance_records	44
31	Table 4.13: Table marks	44
32	Table 4.14: Table marks_audit	44
33	Table 4.15: Table rooms	45
34	Table 4.16: Table time_slots	45
35	Table 4.17: Table timetable_entries	45
36	Table 4.18: CHECK constraints	45
37	Table 4.19: Input forms	46
38	Table 4.20: Outputs of the system	46
39	Table 4.21: Module design	47
40	Table 4.22: Role and access matrix (HTTP status returned)	48

S.No	Table Number & Name	Page No
41	Table 4.23: Security threats and measures	48
42	Table 5.1: Development environment	49
43	Table 5.2: Technologies	49
44	Table 5.3: Source size by language (counted from the repository)	50
45	Table 5.4: Frameworks and libraries	50
46	Table 5.5: Project team	52
47	Table 5.6: Development challenges and solutions	64
48	Table 6.1: Test levels and tools	65
49	Table 6.2: Unit test cases	66
50	Table 6.3: JUnit results (mvnw clean package, build SUCCESS)	66
51	Table 6.4: Django test results (71 tests, result: OK)	66
52	Table 6.5: Integration test cases	67
53	Table 6.6: System test: live pages	68
54	Table 6.7: Requirement traceability	68
55	Table 6.8: Page response times (5 requests each)	69
56	Table 6.9: Java engine command times (5 runs each, including JVM start-up)	69
57	Table 6.10: Security test cases (error guessing)	70
58	Table 6.11: User acceptance walk-through	70
59	Table 6.12: Statement and branch coverage of fn_grade_points	71
60	Table 6.13: Branch coverage of role_required	71
61	Table 6.14: Path coverage of enroll_student	72
62	Table 6.15: Condition coverage of MarksSheetForm.clean and trg_marks_after_update	72
63	Table 6.16: Loop testing	73
64	Table 6.17: JaCoCo coverage of the Java engine by the JUnit suite	73
65	Table 6.18: coverage.py results for the Django suite	74
66	Table 6.19: Equivalence partitioning and boundary value test cases	74
67	Table 6.20: Decision table: enrollment	76
68	Table 6.21: Decision table: marks entry	76
69	Table 6.22: Decision table test cases	76
70	Table 6.23: State transition test cases	77
71	Table 6.24: Defects found and corrected	78
72	Table 6.25: Test summary	78
73	Table 7.1: Deployment environment	79
74	Table 7.2: Configuration variables	80
75	Table 8.1: Maintenance plan	86
76	Table 9.1: Results obtained	87
77	Table 9.2: Performance evaluation	93
78	Table 9.3: Existing system vs proposed system	94
79	Table 9.4: Achievement of objectives	95
80	Table A.1: Configuration items	100
81	Table A.2: Git history of the project	100

S.No	Table Number & Name	Page No
82	Table A.3: Baselines	101
83	Table A.4: Build and release	102
84	Table A.5: SCM tools	102
85	Table B.1: Risk register	103
86	Table C.1: JUnit tests	104
87	Table C.2: Django tests	104

LIST OF ABBREVIATIONS

S.No	Abbreviations	Description
1	AICTE	All India Council for Technical Education
2	API	Application Programming Interface
3	CASE	Computer-Aided Software Engineering
4	CGPA	Cumulative Grade Point Average
5	CLI	Command-Line Interface
6	CRUD	Create, Read, Update, Delete
7	CSE (DS)	Computer Science and Engineering (Data Science)
8	CSRF	Cross-Site Request Forgery
9	CSS	Cascading Style Sheets
10	CSV	Comma-Separated Values
11	DAO	Data Access Object
12	DBMS	Database Management System
13	DFD	Data Flow Diagram
14	ERD	Entity Relationship Diagram
15	ERP	Enterprise Resource Planning
16	FD	Functional Dependency
17	FK	Foreign Key
18	FR	Functional Requirement
19	GPA	Grade Point Average
20	HOD	Head of the Department
21	HSTS	HTTP Strict Transport Security
22	HTML	HyperText Markup Language
23	HTTP(S)	HyperText Transfer Protocol (Secure)
24	IEEE	Institute of Electrical and Electronics Engineers
25	JAR	Java Archive
26	JDBC	Java Database Connectivity
27	JDK	Java Development Kit
28	JNTUH	Jawaharlal Nehru Technological University Hyderabad
29	JSON	JavaScript Object Notation
30	JVM	Java Virtual Machine
31	LTS	Long-Term Support
32	MVT	Model-View-Template
33	NFR	Non-Functional Requirement
34	OOP	Object-Oriented Programming
35	ORM	Object-Relational Mapping
36	PK	Primary Key
37	RBAC	Role-Based Access Control
38	SCM	Software Configuration Management
39	SDLC	Software Development Life Cycle
40	SQL	Structured Query Language
41	SRS	Software Requirements Specification
42	TLS	Transport Layer Security

S.No	Abbreviations	Description
43	UAT	User Acceptance Testing
44	UML	Unified Modeling Language
45	URL	Uniform Resource Locator
46	UTC	Coordinated Universal Time
47	VJIT	Vidya Jyothi Institute of Technology
48	XSS	Cross-Site Scripting

CHAPTER 1: INTRODUCTION

1.1 Introduction to the project

Smart Campus & Academic Suite is a college academic management system for one department. It keeps students, faculty, courses, enrollments, class timetables, attendance and marks in a single MySQL database, `smart_campus_db`, and gives each kind of user the part of that data they need. Students see their own attendance, marks, grades and CGPA. Faculty record attendance and marks for the courses they teach. The administrator manages master data, enrolls students, builds the timetable, publishes the defaulter list and reads academic reports.

The system has three parts that share the same database. A **MySQL 8.0** schema holds the data and the rules that must always hold (constraints, triggers, views and a stored procedure). A **Java 17** engine reads and writes the database through raw JDBC and runs reports from the command line. A **Django 5.2** web portal offers role-based pages for Admin, Faculty and Student, and can call the Java engine. The project was built as one codebase that serves four third-semester labs: DBMS, Java, Django and Software Engineering. This report is the Software Engineering deliverable and follows the SDLC from planning to maintenance.

1.2 Background of the project

In most colleges, attendance is marked in registers or spreadsheets kept by each teacher, internal and external marks are typed into separate sheets, and the examination section computes grades and CGPA later. A student usually learns that their attendance has fallen below the 75% requirement only when the defaulter list is put up near the end of the semester. Timetables are prepared by hand, so a teacher can be booked into two classes at once without anyone noticing until the clash happens.

Relational databases were designed to remove exactly this kind of duplicated, inconsistent data by keeping each fact once and deriving everything else from it [1]. Role-based access control gives each user only the operations their role needs [2], [3]. The project applies both ideas to the daily academic work of a department.

1.3 Problem statement

Academic records in the department are spread across registers, spreadsheets and separate examination files. The same student details are typed in several places, grades and attendance percentages are calculated by hand, and nothing stops two inconsistent copies from existing. Students cannot see their own standing in time to act on it, faculty cannot record attendance and marks in one place, and the administrator has no reliable, current view of defaulters, GPA or timetable clashes. **The problem is to build one consistent, secure system in which each user records or views academic data according to their role, and in which the academic rules (grading, the 75% attendance rule, timetable clashes, unique enrollments) are enforced automatically.**

1.4 Motivation for the project

- Students should know their attendance and marks while there is still time to improve them.
- Teachers should spend less time on registers and grade calculations, and more on teaching.
- Rules such as "75% attendance" and the grading scale should be applied identically for everyone.
- One shared database lets the DBMS, Java, Django and Software Engineering labs be demonstrated on one realistic system instead of four unrelated toy programs.

1.5 Objectives of the project

- **O1.** Design a normalised (3NF) relational database for departments, faculty, students, courses, enrollments, attendance, marks, rooms and timetables, with all integrity rules declared in MySQL.
- **O2.** Compute grades, attendance percentages, defaulters and GPA in the database (triggers, functions and views) so that every application gets the same answer.
- **O3.** Build a role-based web portal in which Admin, Faculty and Student each see and change only what their role allows.
- **O4.** Build an object-oriented Java engine that reads and writes the same

database through JDBC and produces reports from the command line, including a concurrent dashboard and CSV export.

- **O5.** Connect the portal to the engine safely, so an administrator can run engine reports from the browser.
- **O6.** Verify the system with automated tests and documented test cases, and record the design with UML, ER and data flow diagrams.

1.6 Scope of the project

In scope: one department's academic year (demo data: 3rd semester CSE, academic year 2026-27, 10 students, 5 faculty members, 6 courses, 60 enrollments, 720 attendance records and 60 marks rows). The system covers master data, enrollments, timetables, attendance, marks, grading, defaulters, GPA reports, a marks audit trail, a command-line engine and a web portal with three roles.

Out of scope: fee payment, admissions, library and hostel modules, e-mail or SMS notifications, a mobile app, and hosting on a public server. The demo logins share one password and are meant only for local evaluation.

1.7 Existing system

The existing (manual) process followed in the department works as follows:

1. The office keeps student and faculty details in spreadsheets; each new semester's list is typed again.
2. Each teacher marks attendance in a paper register or a personal spreadsheet for every class.
3. Near the cut-off date, teachers total the attendance by hand and send it to the office, which prepares the defaulter list (students below 75%).
4. Internal marks are entered by teachers in one sheet; external marks arrive later from the examination branch in another.
5. Totals, grade points, letter grades and CGPA are computed manually or with spreadsheet formulas that differ from sheet to sheet.
6. The timetable is drawn by hand; clashes are noticed only when a teacher is due in two rooms at once.
7. Students learn their standing only from notice boards or by asking the teacher.

1.8 Limitations of the existing system

- **Redundancy and inconsistency:** the same student appears in many sheets, and the copies drift apart.
- **Manual calculation errors:** percentages, totals and grades are recomputed by hand.
- **Late information:** students find out they are defaulters too late to recover.
- **No enforcement of rules:** nothing prevents a duplicate enrollment, marks above the maximum, a Saturday attendance entry or a teacher double-booking.
- **No access control or audit trail:** anyone with the file can change any value, and nobody can tell who changed a mark or what it was before.
- **No consolidated reports:** department-wide GPA or attendance summaries take days to prepare.

1.9 Proposed system

The proposed system replaces the registers and spreadsheets with one database and three role-based views of it. What each user type can do:

Table 1.1: User types and their functions in the proposed system

User type	What the proposed system provides
Student	Own profile, per-course attendance percentage, internal/external marks, letter grade, weighted GPA and official CGPA, class timetable, and a warning banner when any course is below 75%. A student can never open another student's record.
Faculty	Own courses and weekly timetable, course roster, attendance sheet for any weekday, marks sheet with totals and grades computed by the database, and recent marks changes. A teacher cannot open another teacher's course.
Admin	Dashboard with counts and averages; add, edit, search and delete students, faculty and courses; enroll students; add timetable entries with clash checks; live defaulter list and "declare defaulters"; GPA report; marks audit history; the Java engine page; Django admin.
Java engine (CLI)	Student lookup, attendance, defaulters, GPA report, enroll, declare defaulters, a concurrent dashboard, CSV export and an interactive menu; `--json` output for the portal.

1.10 Advantages of the proposed system

- Each fact is stored once (3NF), so there is no duplicate data to drift apart.
- Grades, attendance percentages, defaulters and GPA are computed by the database the same way for every user and every application.
- Invalid data is rejected at the source: CHECK, UNIQUE and foreign-key

constraints and triggers stop bad marks, weekend attendance, duplicate enrollments and timetable clashes.

- Every page checks the user's role; students and faculty see only their own records.
- Every marks change is recorded automatically in an audit table.
- Reports that took days are available instantly (measured page times are about 50-70 ms, Chapter 9).
- The Java engine gives the same reports from a terminal or a CSV file, without the web server.

1.11 Organization of the report

Chapter 2 reviews related research and existing systems. Chapter 3 is the Software Requirements Specification. Chapter 4 presents the architecture, UML diagrams, data flow diagrams, ER diagram and database, interface and security design. Chapter 5 describes the implementation. Chapter 6 covers testing, with unit, integration, white-box and black-box test cases and their real results. Chapter 7 explains deployment and installation, Chapter 8 maintenance, Chapter 9 the results and measured performance, and Chapter 10 concludes. Appendix A describes software configuration management, Appendix B risk management and Appendix C lists every automated test run.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Before designing the system we studied published work on relational data modelling, role-based access control, and web-based academic and student information systems, and we examined three existing open-source systems used by schools and colleges. The sources below were located by web search and their bibliographic details were checked on the publisher or repository page; the full references are listed in the References section.

2.2 Review of existing research papers

[1] E. F. Codd, "A relational model of data for large shared data banks" (1970).
Codd introduced the relational model: data stored as relations (tables), a normal form for them, and data independence between applications and storage. **Relevance:** the project's schema follows this model and is normalised to 3NF so that each fact is stored once.

[2] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, "Role-based access control models" (1996).
The paper defines the RBAC family of models, in which permissions are attached to roles and users acquire permissions through their roles, which simplifies security administration. **Relevance:** the portal uses three roles (Admin, Faculty, Student) and checks the role on every page.

[3] D. F. Ferraiolo, J. F. Barkley and D. R. Kuhn, "A role-based access control model and reference implementation within a corporate intranet" (1999).
The authors describe an RBAC model and a reference implementation for web servers on an intranet. **Relevance:** it shows RBAC enforced by the web application itself, which is the approach taken in `core/roles.py`.

[4] M. N. F. Hidayah and H. Sofyan, "Design and implementation of an integrated web-based academic management and student admission monitoring system using Django framework" (2026).
A web system that brings academic data, payment administration and admissions into one platform, replacing manual and spreadsheet work; built with Python, Django and MySQL. **Relevance:** the closest match in technology; it confirms Django + MySQL

as a practical stack for academic systems.

[5] J. Thilagavathi, G. P. Narasimha, S. Yasaswini and E. Aiswarya, "College management system using Python & Django" (2024).

A Django platform that automates admissions, course and exam scheduling, attendance tracking, resource booking and reporting; it uses SQLite for storage. **Relevance:** similar functions, but the rules live in application code and the database is a single-user SQLite file.

[6] M. J. Davis and Z. Huang, "ERP in higher education: A case study of SAP and Campus Management" (2007).

A case study of a university that replaced legacy student software with an ERP system, discussing critical success factors and staff acceptance. **Relevance:** large ERP suites are costly and need much adaptation; a department-level system can be small and fit its own rules exactly.

2.3 Review of existing applications or systems

[7] openSIS Classic (OS4ED).

A web-based, GPL-licensed student information system for K-12, trade schools and higher education. Its repository lists PHP 8 with MySQL 5.7/8.0 or MariaDB, and features such as student and staff data, course manager and scheduling, attendance, a teacher gradebook, report cards and transcripts. The Community Edition is listed as version 9.3.

[8] Fedena.

A school management system available in a free community (open-source) edition and paid editions. Attendance management and timetable management are among its core modules; the vendor's feature pages describe automated attendance reports and substitute-teacher allocation.

[9] Moodle Attendance activity (mod_attendance).

A plugin for the Moodle learning management system, maintained by Dan Marsden, that lets teachers create attendance sessions and record statuses for students in a Moodle course. It handles attendance only; marks live in Moodle's own gradebook.

2.4 Comparison of related systems

Table 2.1: Comparison of related systems with the proposed system

Feature	openSIS [7]	Fedena [8]	Moodle Att. [9]	Django CMS [5]	Django AMS [4]	Proposed
Stack	PHP, MySQL	Web app	PHP plugin	Django, SQLite	Django, MySQL	MySQL, Java 17, Django 5.2
Attendance	Yes	Yes	Yes	Yes	Not stated	Yes
Marks / grading	Gradebook	Not reviewed	Moodle gradebook	Exams	Academic data	Marks + DB-computed grades
Rules enforced in the DB	Not stated	Not stated	Not stated	No (SQLite file)	Not stated	Yes: CHECK, UNIQUE, FK, 9 triggers
Role-based portals	Yes	Yes	Teacher/ student	Yes	Yes	Admin, Faculty, Student
Marks audit trail	Not stated	Not stated	Not stated	Not stated	Not stated	Yes (AFTER trigger)
Timetable clash check	Scheduling	Timetable module	No	Scheduling	Not stated	Yes (trigger + UNIQUE)
Command-line / batch reports	Not stated	Not stated	No	No	No	Yes (Java engine, CSV)
Cost	Free (GPL)	Free / paid	Free	Academic	Academic	Free (open tools)

"Not stated" means the feature was not mentioned in the source we reviewed; it does not mean the system lacks it.

2.5 Research gaps and limitations

- The Django-based academic systems [4], [5] place their rules in application code; nothing stops another program writing invalid data straight into the database.
- None of the reviewed sources describes an audit trail for marks changes.
- Large ERP suites [6] fit higher education only after heavy customisation and cost far more than a department can spend.
- The open-source suites [7], [8] are complete products; they are not designed to be studied lab by lab, with the database, OOP engine and web layer each visible on its own.
- Attendance-only tools [9] do not connect attendance to marks, GPA or the timetable.

2.6 Proposed approach based on the literature review

From the relational model [1] the project takes a 3NF schema in which derived values (attendance %, grades, GPA) are computed by views and triggers instead of being

typed in. From RBAC [2], [3] it takes three fixed roles, derived from the login and enforced on every request. Unlike [4] and [5], the academic rules are placed in the database itself, so the Django portal and the Java engine (and any future client) obey the same rules. A marks audit trail and a timetable clash trigger fill gaps not addressed in the reviewed sources. The system is kept small and department-sized, in contrast to the ERP approach studied in [6].

2.7 Summary

Six research papers and three existing systems were reviewed. They confirm the value of web-based academic systems and of role-based access, and show two gaps that this project addresses: enforcing academic rules in the shared database rather than in one application, and keeping an automatic audit trail of marks.

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

3.1 Introduction

This chapter specifies the requirements of Smart Campus & Academic Suite in the style of IEEE Std 830 [18]. It is written for the project team, the internal guide and the lab evaluators. Definitions of abbreviations are given in the List of Abbreviations; references are listed at the end of the report.

3.1.1 Product perspective

The product is a new, self-contained system that replaces registers and spreadsheets. It consists of three cooperating components that share one MySQL database: the database layer (schema, views, triggers, functions and a procedure), the Java engine (command line, JDBC) and the Django portal (web, role-based). The portal calls the engine as a separate process. There is no dependency on any external college system.

3.1.2 Product functions

- **Admin:** manage students, faculty and courses; enroll students; build the timetable; publish defaulters; view GPA and marks-audit reports; run engine commands; use Django admin.
- **Faculty:** view own courses, roster and timetable; record attendance; enter marks.
- **Student:** view own attendance, marks, grades, GPA/CGPA, timetable and defaulter warning.
- **Engine user (terminal):** run reports, enroll, declare defaulters, export CSV, interactive menu.

3.1.3 User characteristics

Table 3.1: User characteristics

User	Characteristics	Technical skill assumed
Admin	Office or department staff responsible for academic records	Comfortable with web forms
Faculty	Teachers of one or more courses	Basic web use
Student	Undergraduate students (II year)	Basic web use, often on a phone
Engine user	Team member or lab evaluator	Command line

3.1.4 Operating environment

Server side: Windows 10/11, macOS or Linux with MySQL Server 8.0.16 or newer, JDK 17 or newer and Python 3.10 or newer. Client side: any current web browser (tested with Chromium); pages are responsive and work at phone width.

3.1.5 Design and implementation constraints

- One MySQL 8.0 database shared by both applications; credentials only from the repository's `.env` file.
- SQL scripts are the source of truth for the schema; Django models are unmanaged (`managed = False`).
- Java: no frameworks (no Spring, Hibernate or connection pool); raw JDBC with `PreparedStatement` only.
- Django 5.2 LTS (Django 6.1 needs MySQL 8.4); vanilla HTML5 and hand-written CSS3, no CSS frameworks.
- The portal may start the engine only through `core/java_bridge.py`, with an allow-list and a timeout.

3.1.6 Assumptions and dependencies

- One department and one academic year are active at a time; the academic year starts in July.
- Each login username is unique across students and faculty (enforced by triggers).
- MySQL, Java and Python are installed on the server machine; the engine JAR has been built.
- Attendance is recorded only for Monday to Friday sessions.

3.2 Purpose of the software

The purpose of the software is to keep all academic records of a department in one consistent database, to compute grades, attendance percentages, defaulters and GPA automatically and identically for everyone, and to give students, faculty and the administrator secure, role-appropriate access to that data through a web portal and a command-line engine.

3.3 Functional requirements

Table 3.2: Functional requirements

ID	Actor	Requirement
FR1	All	The system shall authenticate users by username and password and redirect each user to the dashboard of their role.
FR2	All	The system shall derive the role from the login: superuser = Admin, a username found in `students` = Student, in `faculty` = Faculty; any other login gets an access-denied page.
FR3	All	A logged-in user shall be able to change their own password and log out (POST only).
FR4	Student	A student shall view their own profile, enrolled courses and weekly timetable.
FR5	Student	A student shall view per-course attendance (classes held, attended, percentage).
FR6	Student	A student shall view internal, external and total marks, grade points, letter grade, weighted GPA and official CGPA.
FR7	Student	The student dashboard shall show a warning when any course is below 75% attendance.
FR8	Faculty	A faculty member shall view only the courses they teach and their weekly timetable.
FR9	Faculty	A faculty member shall record attendance (Present/Absent) for a course on a weekday; re-saving a date shall update it.
FR10	Faculty	A faculty member shall enter internal (0-40) and external (0-60) marks; total, grade points and letter grade shall be computed by the database.
FR11	Admin	The admin shall add, edit, search, paginate and delete students, faculty and courses.
FR12	Admin	The admin shall enroll a student in a course of the student's own semester; duplicate enrollments shall be refused.
FR13	Admin	The admin shall add timetable entries; a teacher, room or section double-booking shall be refused with a readable message.
FR14	Admin	The admin shall view the live defaulter list and declare defaulters (stored procedure).
FR15	Admin	The admin shall view the GPA report and the marks audit history.
FR16	Admin	The admin shall run allowed Java engine commands from the portal and see the result or a typed error.
FR17	System	Every change of marks shall be written to an audit table with old and new values and the time of change.
FR18	Engine	The Java engine shall provide `student`, `whois`, `attendance`, `defaulters`, `gpa-report`, `courses`, `enroll`, `refresh-defaulters`, `dashboard`, `export`, `menu` and `help` commands, with text or `--json` output.
FR19	Engine	The engine shall export the GPA, defaulter and attendance reports to CSV files.
FR20	Engine	The `dashboard` command shall load the three reporting views concurrently and report per-task thread names and timings.

3.4 Non-functional requirements

Table 3.3: Non-functional requirements

ID	Category	Requirement	How it is met / measured
NFR1	Performance	Portal pages shall respond in under 1 s on the development machine.	Median 47-69 ms for 15 pages; engine pages about 0.7 s (Section 6.8).
NFR2	Performance	A page shall make at most one Java engine call.	Enforced in `core/views.py`.

ID	Category	Requirement	How it is met / measured
NFR3	Security	Every page shall check the user's role; students and faculty see only their own records.	`role_required` / `RoleRequiredMixin`; URL-wide test walks every route.
NFR4	Security	All POST forms shall be protected against CSRF; output shall be HTML-escaped; SQL shall be parameterised.	Django CSRF and auto-escaping; ORM and `PreparedStatement` (tests BB29-BB33).
NFR5	Security	Passwords shall be stored hashed; credentials never committed.	Django PBKDF2 hashing; `.env` is git-ignored.
NFR6	Integrity	Academic rules shall be enforced by the database, not only by the application.	CHECK, UNIQUE, FK constraints and 9 triggers.
NFR7	Reliability	A missing JAR, Java or database error shall show a message, never a crash.	Typed `EngineError`; friendly DB errors; custom 403/404/500 pages.
NFR8	Usability	Pages shall be responsive down to phone width and use plain language messages.	CSS Grid/Flexbox with media queries; `core/db_errors.py`.
NFR9	Maintainability	Code shall be layered (models, DAO, services, CLI; MVT) and covered by automated tests.	29 JUnit + 71 Django tests; 96% Django line coverage.
NFR10	Portability	The system shall run on Windows, macOS and Linux.	Maven Wrapper, Python venv, `.env`; LF line endings for `mvnw`.

3.5 Hardware requirements

Table 3.4: Hardware requirements

Component	Minimum	Used for development and testing
Processor	Dual-core 64-bit, 2 GHz	AMD Ryzen 5 3550H, 4 cores / 8 threads
Memory	4 GB RAM	13.9 GB usable RAM
Storage	2 GB free (MySQL, JDK, Python, Maven cache)	SSD, 22 GB free
Display / network	Any; localhost only	Laptop display, localhost

3.6 Software requirements

Table 3.5: Software requirements

Software	Version required	Version used
Operating system	Windows 10/11, macOS or Linux	Windows 11 Home Single Language (build 26200)
Database server	MySQL 8.0.16+ (CHECK enforcement)	MySQL 8.0.42
Java	JDK 17+	OpenJDK 17.0.20.1
Build tool	Maven (via wrapper)	Maven 3.9.16 via mvnw
Python	3.10+	Python 3.13.3
Web framework	Django 5.2 LTS	Django 5.2.18
DB drivers	PyMySQL; MySQL Connector/J	PyMySQL 1.2.3; mysql-connector-j 9.7.0
Browser	Any current browser	Chromium (Playwright), Microsoft Edge

3.7 User requirements

- **Student:** "I want to see my attendance and marks for every course, and to be warned early if I am below 75%."
- **Faculty:** "I want to mark attendance and enter marks for my own courses quickly, and not have to calculate totals or grades."
- **Admin:** "I want one place to manage students, faculty, courses and the timetable, to see who the defaulters are, and to know who changed a mark."
- **Evaluator:** "I want to run the reports from the command line and see the same numbers as the web portal."

3.8 System constraints

- Marks: internal 0-40, external 0-60, stored as DECIMAL(5,2); external marks require internal marks.
- Attendance status is PRESENT or ABSENT; one record per enrollment per date; weekdays only.
- Credits 1-6, hours per week 1-10, semester 1-8, section a single capital letter, department code 2-10 capital letters, academic year in the form 2026-27, designation from a fixed list (CHECK constraints).
- A student can enroll in a course only once, and only in a course of their own semester.
- A faculty member, a room and a section can each have only one class per time slot.
- The JAR is called with a 30-second timeout; only six commands are allowed from the portal.

3.9 Use case specifications

The use case diagram is shown in Figure 4.3. The main use cases are specified below.

Table 3.6: Use case specification UC1: Log in

Field	Description
Use case ID	UC1
Name	Log in
Actors	Student, Faculty, Admin
Description	User signs in and is sent to the dashboard of their role.
Pre-condition	User has an account.

Field	Description
Main flow	1. User opens /accounts/login/. 2. Enters username and password. 3. System authenticates and derives the role. 4. System redirects to /student/, /faculty/ or /admin-portal/.
Alternate flows	3a. Wrong credentials: the form shows an error. 3b. Login matches no role: 403 page.
Post-condition	User is logged in with a session.

Table 3.7: Use case specification UC2: View own academic record

Field	Description
Use case ID	UC2
Name	View own academic record
Actors	Student
Description	Student views attendance, marks, grades, GPA/CGPA and timetable.
Pre-condition	Logged in as a student.
Main flow	1. Student opens /student/. 2. System finds the student by username. 3. System reads v_attendance_summary, marks and the timetable for that student only. 4. Page shows the data and a warning for courses below 75%.
Alternate flows	None: the URL takes no student id, so another record cannot be requested.
Post-condition	No data changed.

Table 3.8: Use case specification UC3: Record attendance

Field	Description
Use case ID	UC3
Name	Record attendance
Actors	Faculty
Description	Faculty marks Present/Absent for every student of a course on a date.
Pre-condition	Logged in as the course's teacher.
Main flow	1. Faculty opens the course's attendance page. 2. Chooses a weekday date. 3. Marks each student. 4. Saves. 5. System writes the records in one transaction.
Alternate flows	2a. Weekend date: refused by the CHECK constraint with a message. 1a. Another teacher's course: 403.
Post-condition	Attendance stored; v_attendance_summary updated.

Table 3.9: Use case specification UC4: Enter marks

Field	Description
Use case ID	UC4
Name	Enter marks
Actors	Faculty
Description	Faculty enters internal and external marks; the database grades them.

Field	Description
Pre-condition	Logged in as the course's teacher.
Main flow	1. Faculty opens the marks sheet. 2. Enters internal (0-40) and external (0-60) marks. 3. Saves. 4. Triggers compute total, grade points and letter grade. 5. Changes are written to marks_audit.
Alternate flows	2a. Out-of-range or external without internal: form error. 1a. Another teacher's course: 403.
Post-condition	Marks graded and audited.

Table 3.10: Use case specification UC5: Manage master data

Field	Description
Use case ID	UC5
Name	Manage master data
Actors	Admin
Description	Admin adds, edits, searches and deletes students, faculty and courses.
Pre-condition	Logged in as admin.
Main flow	1. Admin opens a list. 2. Searches or pages. 3. Adds or edits through a form, or deletes after a confirmation page. 4. System saves through core/services.py.
Alternate flows	3a. Rule broken (duplicate, bad section, username used by faculty): friendly message on the form. 3b. Course with enrollments: delete refused (RESTRICT).
Post-condition	Data changed consistently.

Table 3.11: Use case specification UC6: Enroll student

Field	Description
Use case ID	UC6
Name	Enroll student
Actors	Admin
Description	Admin enrolls a student in a course for an academic year.
Pre-condition	Student and course exist.
Main flow	1. Admin opens Enrollments. 2. Chooses student, course and year. 3. System checks the semester match. 4. Inserts the enrollment.
Alternate flows	3a. Semester mismatch: refused. 4a. Already enrolled: UNIQUE violation shown as a sentence.
Post-condition	Enrollment created.

Table 3.12: Use case specification UC7: Add timetable entry

Field	Description
Use case ID	UC7
Name	Add timetable entry
Actors	Admin
Description	Admin schedules a course in a slot, room and section.
Pre-condition	Course, slot and room exist.

Field	Description
Main flow	1. Admin opens Timetable. 2. Fills the form. 3. Trigger and UNIQUE keys check clashes. 4. Entry saved.
Alternate flows	3a. Teacher, room or section already booked: refused with names in the message.
Post-condition	Timetable updated.

Table 3.13: Use case specification UC8: Declare defaulters

Field	Description
Use case ID	UC8
Name	Declare defaulters
Actors	Admin
Description	Admin publishes the defaulter list at the attendance cut-off.
Pre-condition	Attendance recorded.
Main flow	1. Admin opens Defaulters (live list from v_defaulter_list). 2. Clicks "Declare defaulter list now". 3. System runs CALL sp_refresh_defaulter_flags(). 4. Shows the number flagged.
Alternate flows	None.
Post-condition	students.is_defaulter updated for every student.

Table 3.14: Use case specification UC9: Run engine report

Field	Description
Use case ID	UC9
Name	Run engine report
Actors	Admin
Description	Admin runs an allowed Java engine command from the portal.
Pre-condition	JAR built; logged in as admin.
Main flow	1. Admin opens /engine/. 2. Chooses a command and arguments. 3. Portal validates them and runs the JAR with --json. 4. Result is shown as tables.
Alternate flows	3a. Invalid argument: form error. 3b. JAR or Java missing, timeout, engine error: typed error card.
Post-condition	Report shown (enroll and refresh-defaulters also write).

3.10 Feasibility study

Technical feasibility

All technologies are mature, free and well documented: MySQL 8.0, Java 17 LTS, Django 5.2 LTS and Python 3. They run on the existing laptop (Section 3.5). The only integration risk, calling Java from Django, was solved with a subprocess bridge and a JSON contract, and was verified by integration test IT08 (Chapter 6). The project is technically feasible.

Economic feasibility

Table 3.15: Cost estimate

Item	Cost
MySQL Community Server, OpenJDK, Python, Django, PyMySQL, Maven, JUnit	Free (open-source licences)
Development tools: VS Code, Git, PlantUML, LibreOffice, Playwright	Free
Hardware	Existing personal laptop (no purchase)
Hosting	Not needed: runs on localhost for the lab
Team effort	Four students, during the lab hours of the semester

No licence or hardware cost is involved, so the project is economically feasible.

Operational feasibility

Each role sees a small dashboard with plain-language messages, and the tasks mirror what users already do (marking attendance, entering marks). Errors are explained in sentences (for example "Attendance can only be recorded for Monday to Friday") rather than database codes. The portal works in any browser and at phone width, so users need no training beyond a short demo. The system is operationally feasible.

Schedule feasibility

The work was planned in five phases and completed between 2026-10-15 (first commit) and 2026-11-04 (28 commits, listed in Appendix A).

Table 3.16: Project schedule (from the Git history)

Phase	Work	Completed (commit date)
1	Scaffolding and build configuration	2026-10-15
2	MySQL schema, views, triggers, seed data, init_db.py	2026-10-16
3	Java engine: config, models, DAOs, services, CLI, tests	2026-10-31
4	Django portal: three role portals, Java bridge, tests	2026-10-31
4.5	Review and polish: audit trail, CSV export, menu, CRUD, security settings	2026-11-01
5	Software Engineering documentation and this report	2026-11-01
5.1	Report converted to a group report	2026-11-04

All phases fitted within the semester's lab schedule, so the project is feasible in time.

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

4.1 Introduction

This chapter turns the requirements of Chapter 3 into a design. It describes the architecture, the high- and low-level design, the UML diagrams, data flow diagrams, the ER diagram and database design, the input, output and user-interface design, the modules and the security design. Every diagram was drawn from the actual code and schema, and the database facts were read from the running MySQL server.

4.2 System Architecture

The system follows a three-tier architecture with a shared database. The presentation tier is the browser (HTML5 and CSS3 pages) and the terminal (Java CLI). The application tier has two independent programs: the Django portal and the Java engine. The data tier is one MySQL database, which also holds the business rules (constraints, triggers, views, procedure). The portal talks to the engine only through a guarded subprocess call that exchanges JSON.

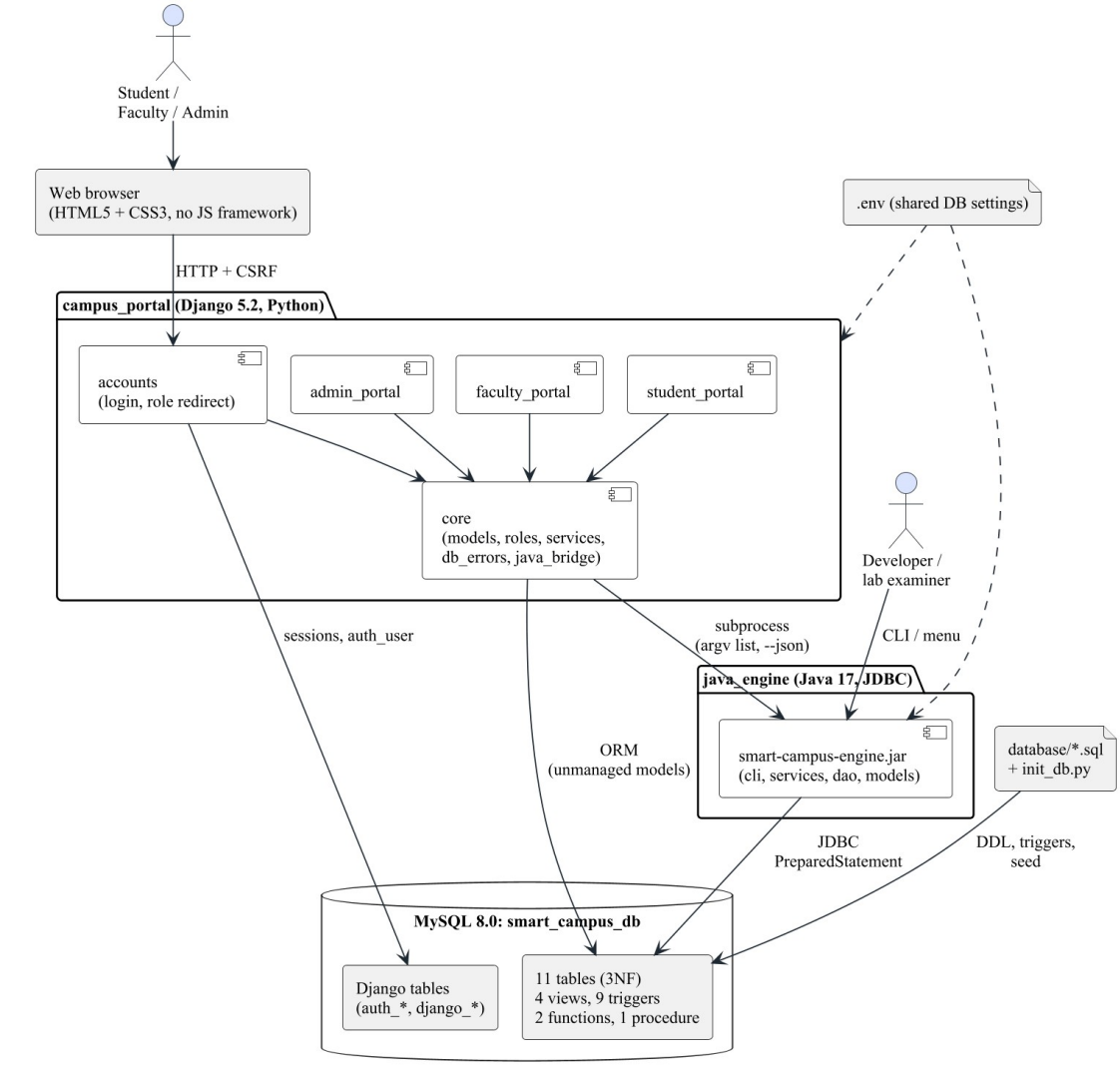


Figure 4.1: System architecture of the Smart Campus & Academic Suite

4.3 High-level design

At high level the system is divided into the modules shown in Figure 4.2. The database tier is built by three SQL scripts run in order (`schema.sql`, `views_triggers.sql`, `seed_data.sql`) by `init_db.py`. The Java engine is layered as config, models, DAO, services and CLI. The Django project has a `core` app (models, roles, services, bridge) and four feature apps (`accounts`, `student_portal`, `faculty_portal`, `admin_portal`).

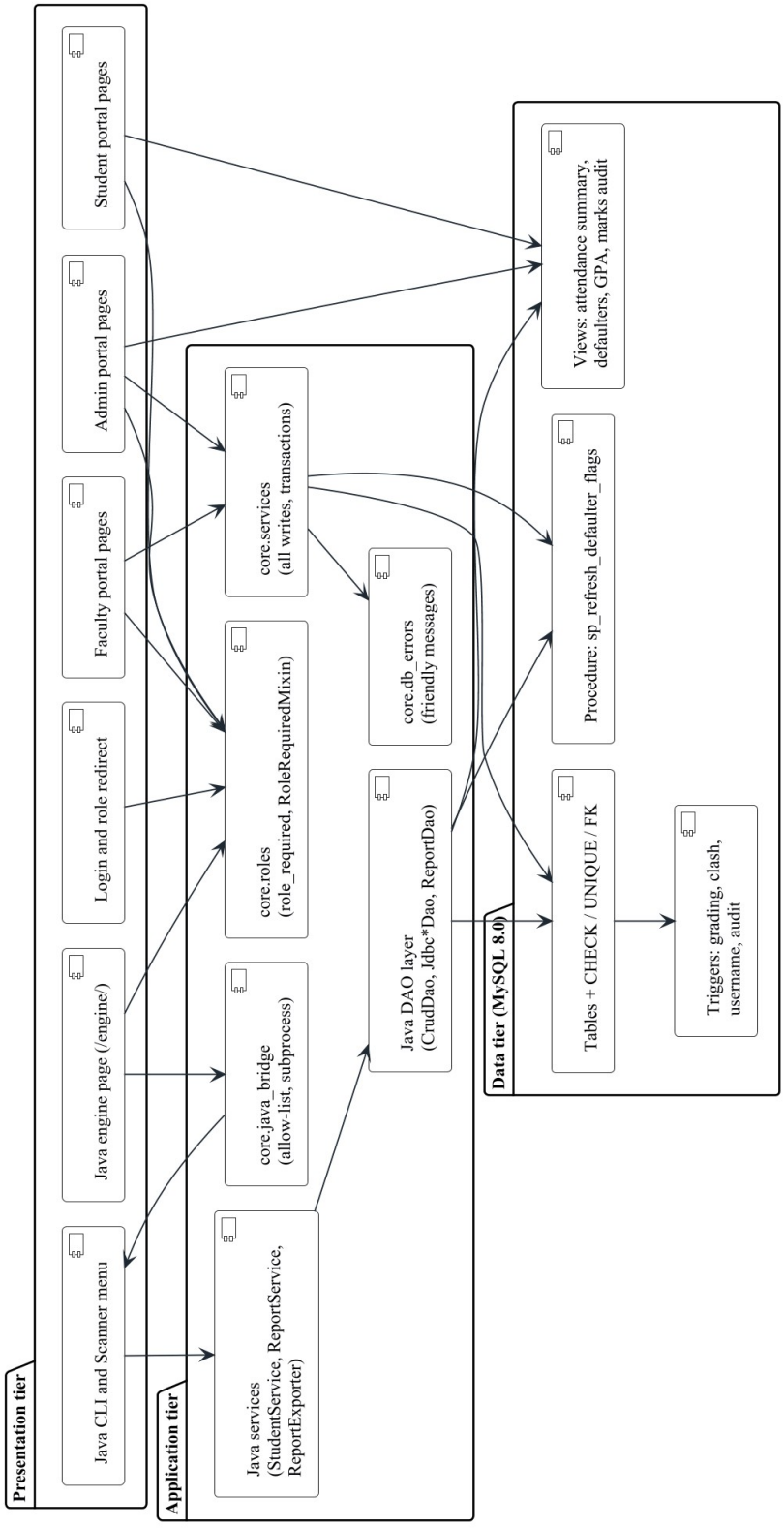


Figure 4.2: High-level design: three tiers and the modules in each (rotated to fit the page)

4.4 Low-level design

The low-level design fixes the responsibility of each package or app, the interfaces between them and the data contract of the bridge.

Table 4.1: Java engine packages

Java package	Files	Responsibility
config	2	`DatabaseConfig` reads DB_* from environment variables, then `.env`; `ConnectionFactory` opens one JDBC connection per call.
models	15	Abstract `Person` with `Student` and `Faculty`; `Course`, `Enrollment`, `AttendanceRecord`, `Marks`; records for view rows.
dao	18	Generic `CrudDao<T, ID>` and entity DAO interfaces; `Jdbc*Dao` implementations; `JdbcDaoSupport` with `queryList` / `queryOne`; `ReportDao` for views and procedure.
services	5	`StudentService` (profile, whois, enroll with semester rule), `ReportService` (defaulters, GPA, concurrent dashboard), `ReportExporter` (CSV).
exceptions	8	Checked hierarchy rooted at `SmartCampusException`.
cli	11	Argument parsing, command registry, text tables, `JsonWriter`, interactive menu.

Table 4.2: Django apps

Django app	Responsibility
core	Unmanaged models for 10 tables and 4 views (marks_audit is read through v_marks_audit); `roles.py` (role derivation and checks); `services.py` (all writes, transactions); `db_errors.py` (friendly messages); `java_bridge.py` and the `/engine/` view; `listing.py` (search and pagination); Django admin.
accounts	Login, logout, password change, `/` role redirect, `seed_demo_users` command.
student_portal	`/student/`: own profile, attendance, marks, CGPA, warning, timetable.
faculty_portal	`/faculty/`: own courses, timetable, roster, attendance sheet, marks sheet.
admin_portal	`/admin-portal/`: dashboard, CRUD, enrollments, timetable, reports, marks audit.

Bridge contract. The portal runs `java -Dfile.encoding=UTF-8 -jar smart-campus-engine.jar <command> [args] --json`. On success the engine prints one line `{"ok":true,"command":...,"data":...}` on stdout with exit code 0; on failure it prints `{"ok":false,...,"error":{"type":...,"message":...}}` on stderr with exit code 1 (runtime/database error) or 2 (bad usage). The portal branches on `error.type`, never on the message.

4.5 UML diagrams

All UML diagrams were written as PlantUML text in `docs/src/diagrams/` and rendered to PNG (see 4.5.10). Class names, methods and values match the source code and the database.

4.5.1 Use Case Diagram

Three human actors use the portal. The MySQL database appears as a system actor, because its triggers and procedure compute grades, record the audit trail, check timetable clashes and declare defaulters.

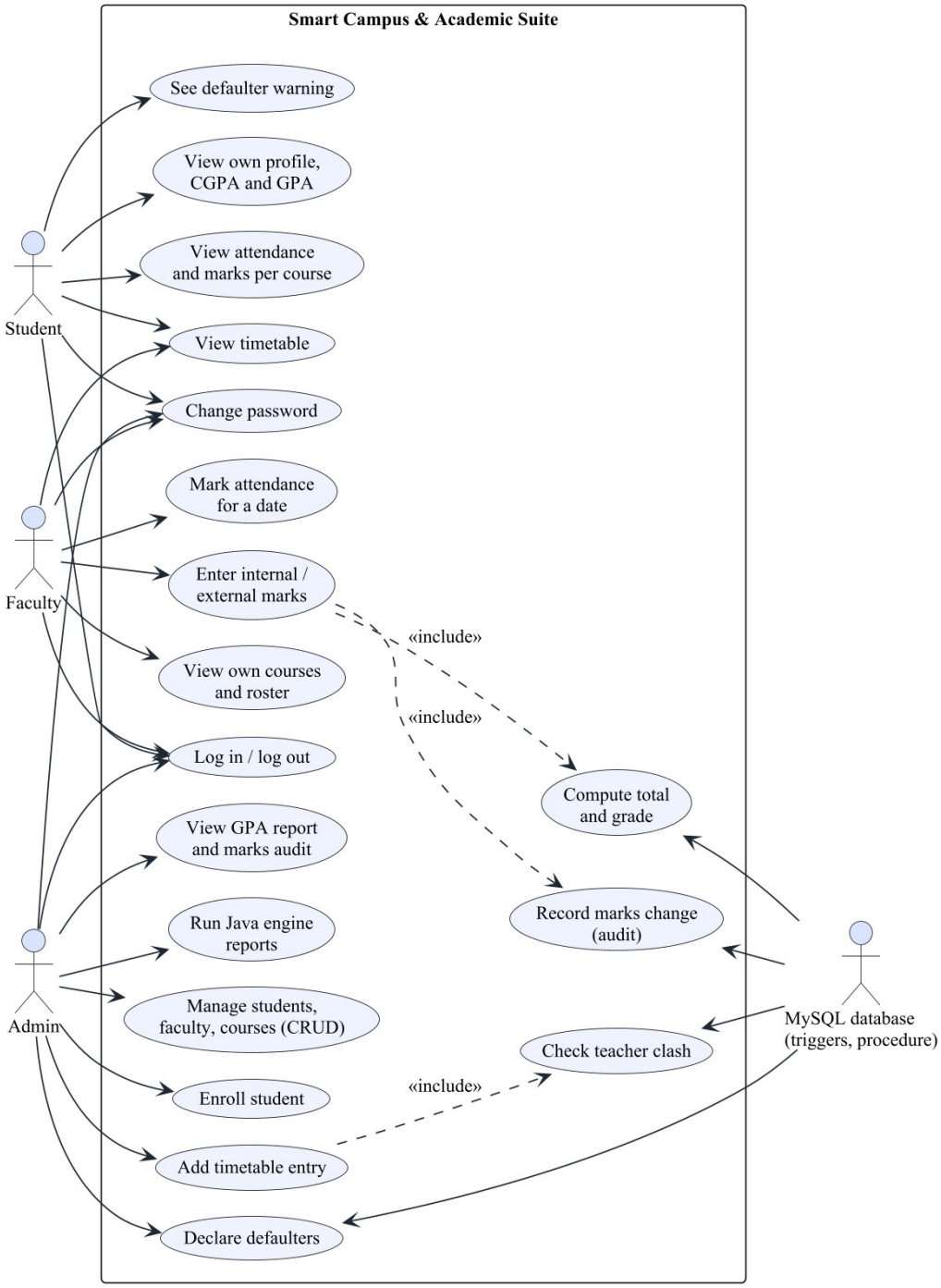


Figure 4.3: Use case diagram

4.5.2 Class Diagram

The domain model shows abstraction and inheritance (`Person` to `Student`/`Faculty`, with `describe()` overridden), encapsulation (private fields validated in constructors) and Java records for read-only view rows.

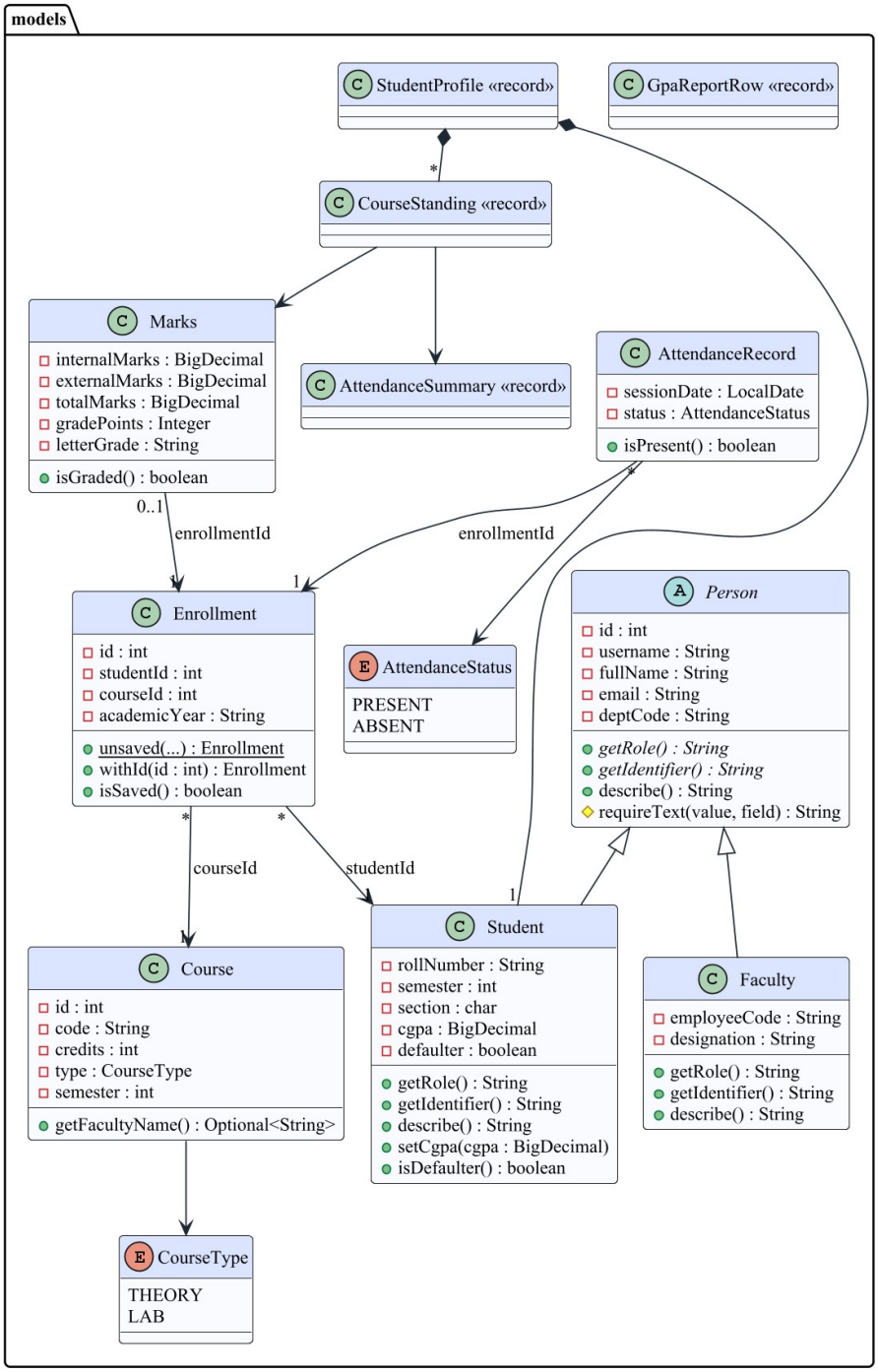


Figure 4.4: Class diagram: Java domain model (package com.smartcampus.models)

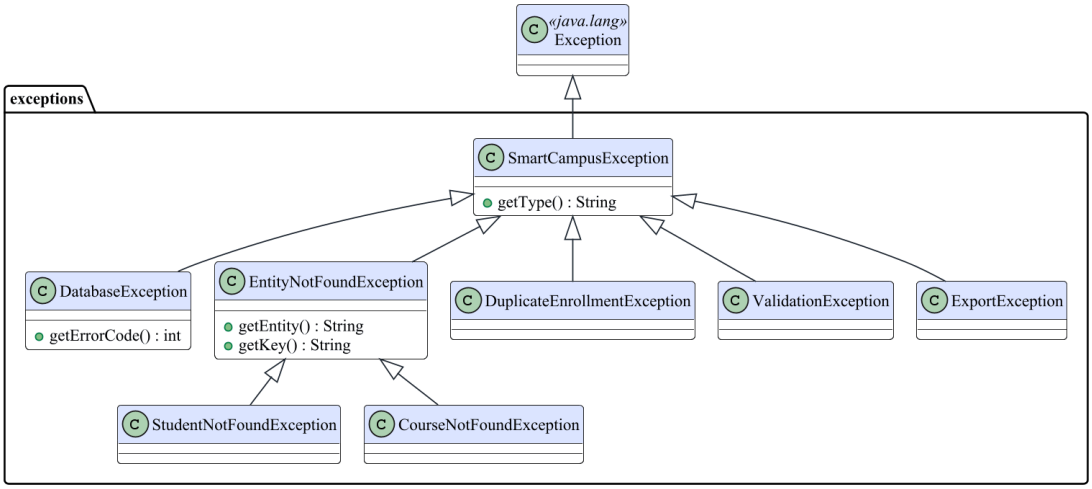


Figure 4.5: Class diagram: checked exception hierarchy (package com.smartcampus.exceptions)

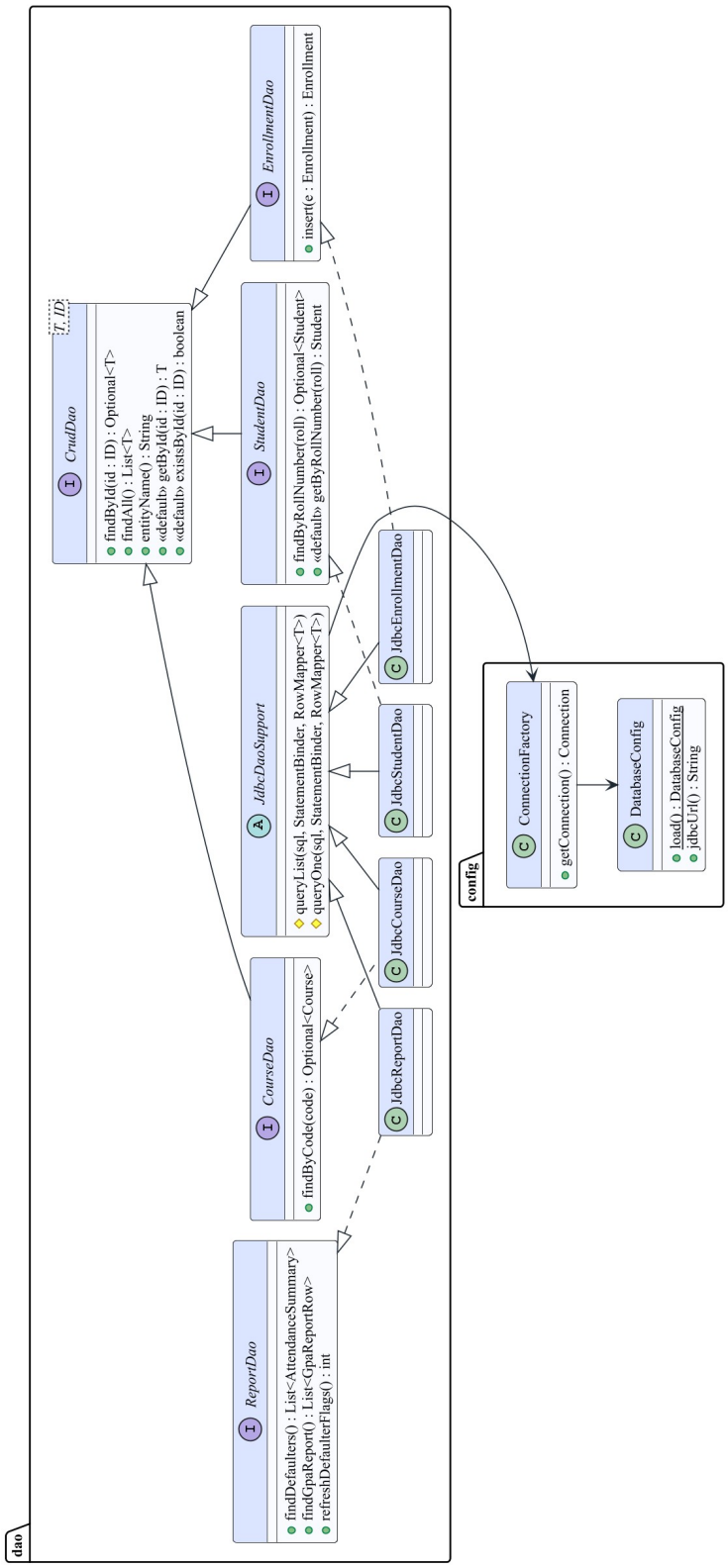


Figure 4.6: Class diagram: configuration and DAO layer of the Java engine (generic CrudDao, JDBC implementations) (rotated to fit the page)

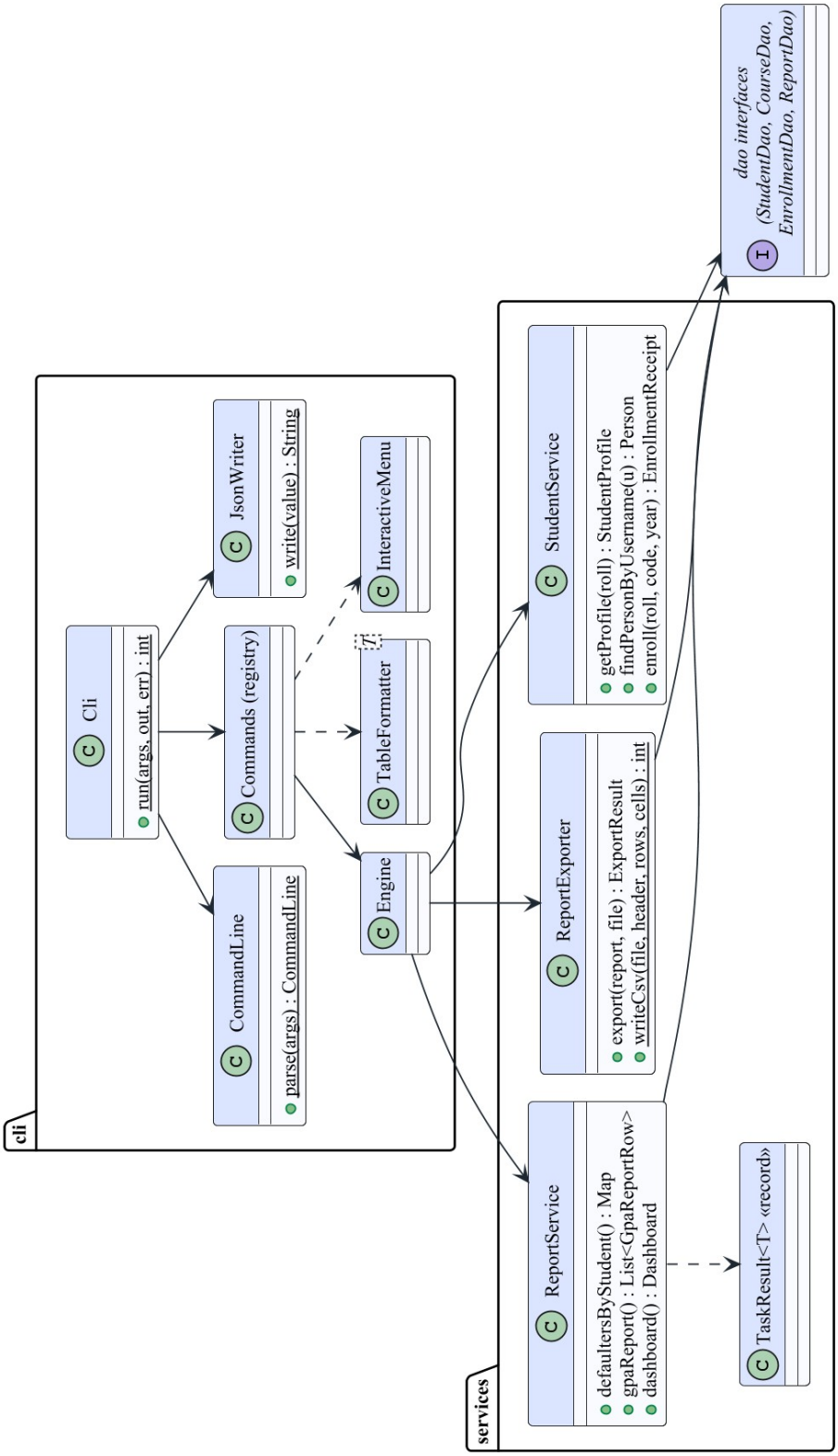


Figure 4.7: Class diagram: service and CLI layers of the Java engine (rotated to fit the page)

4.5.3 Object Diagram

The object diagram is a snapshot of real rows: defaulter 25CSE004's CS301 enrollment, its attendance summary and marks, as read from `smart_campus_db`.

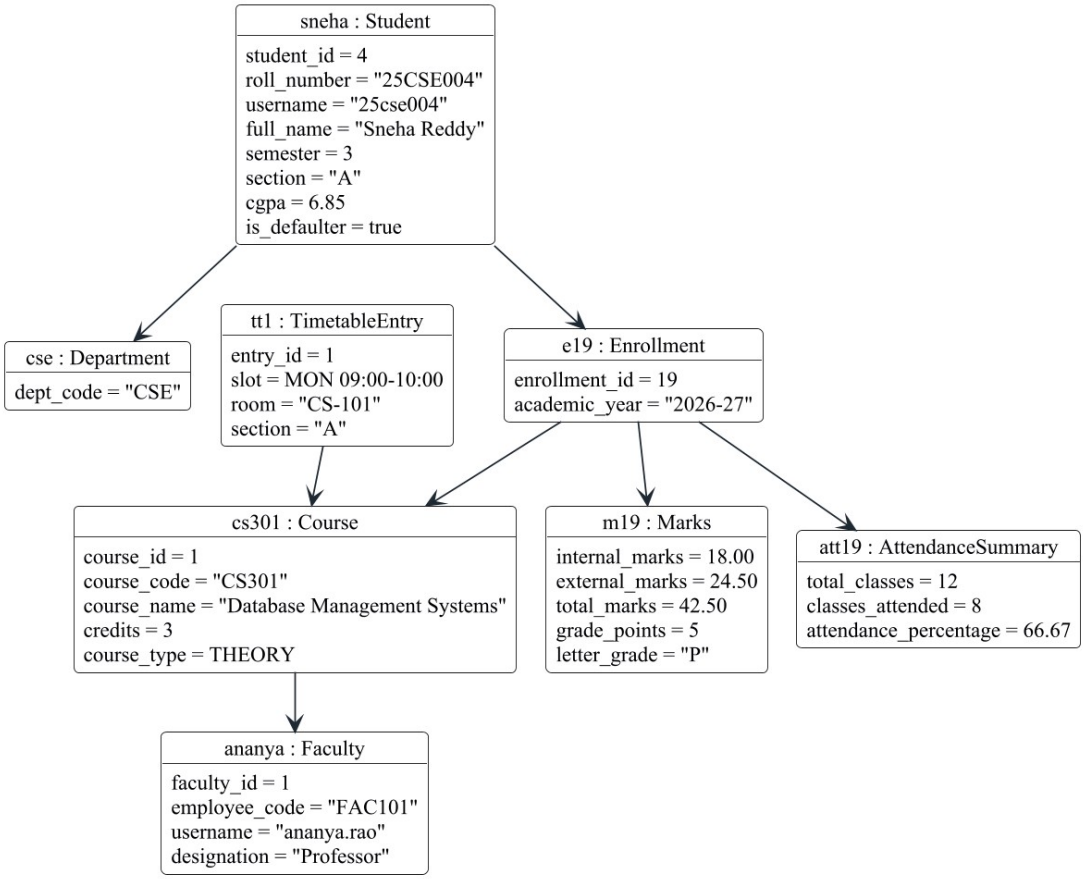


Figure 4.8: Object diagram: one defaulter's CS301 record (values read from smart_campus_db)

4.5.4 Sequence Diagram

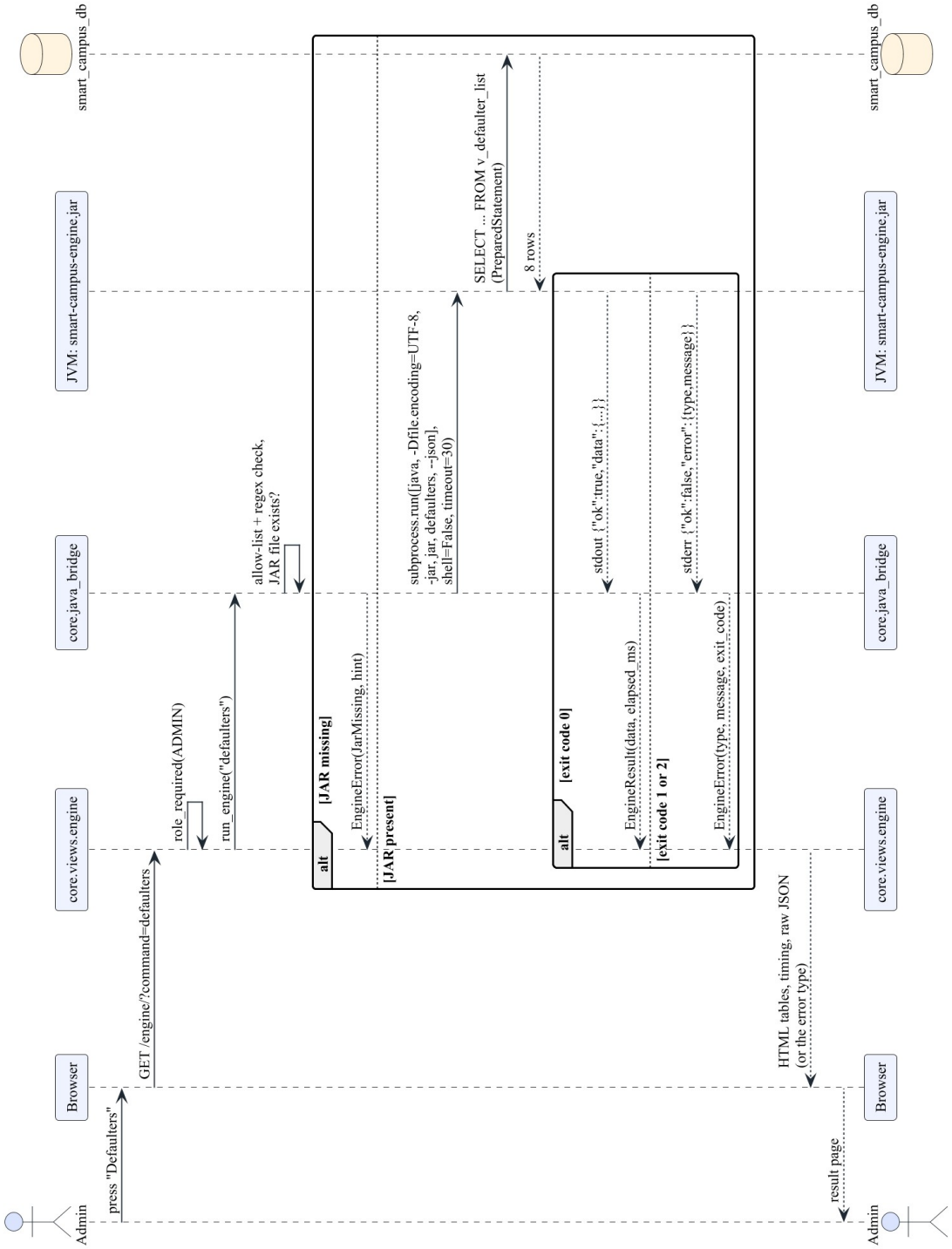


Figure 4.9: Sequence diagram: Admin runs a Java report from the portal (rotated to fit the page)

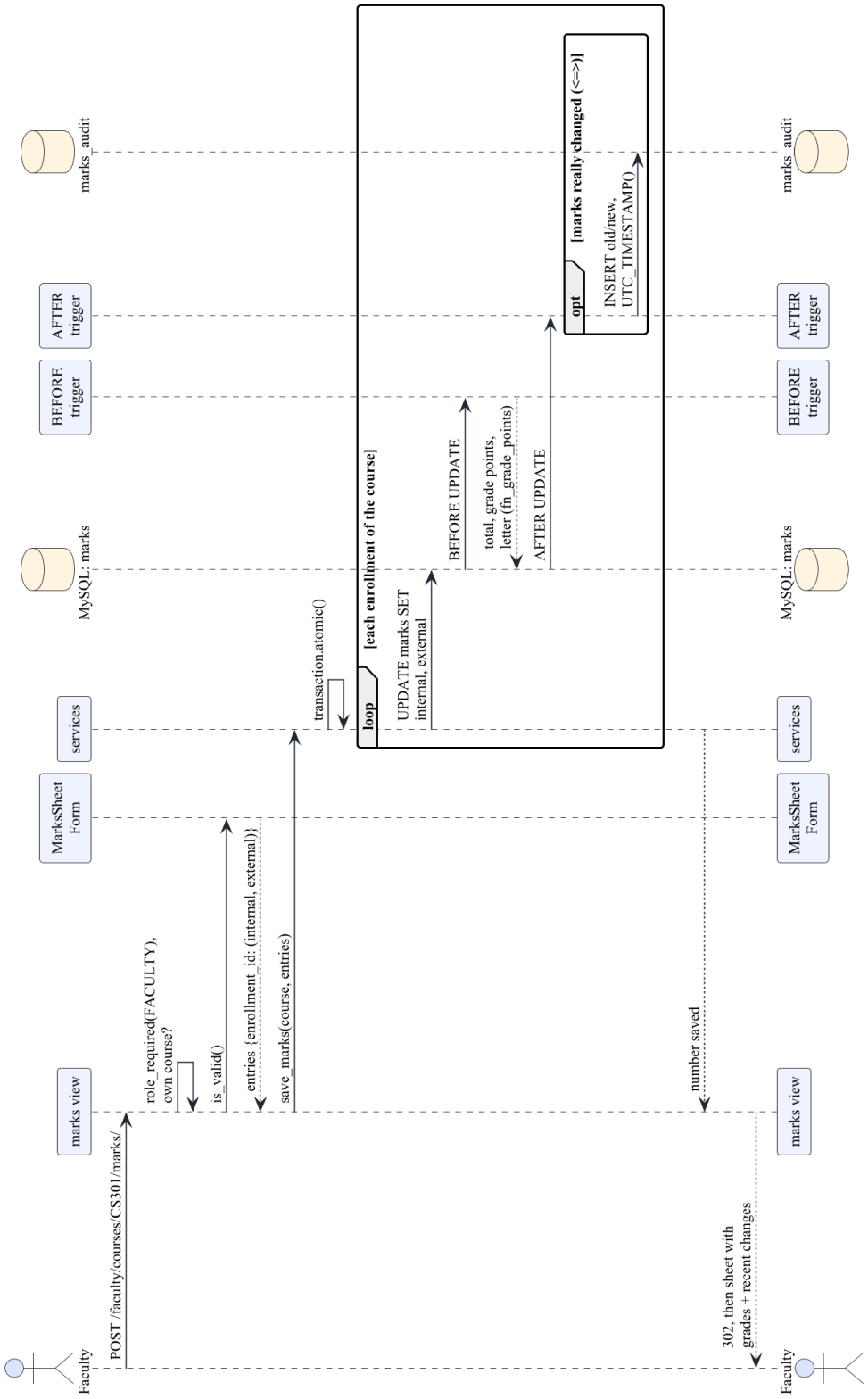


Figure 4.10: Sequence diagram: faculty saves marks; the database grades and audits them (rotated to fit the page)

4.5.5 Collaboration Diagram

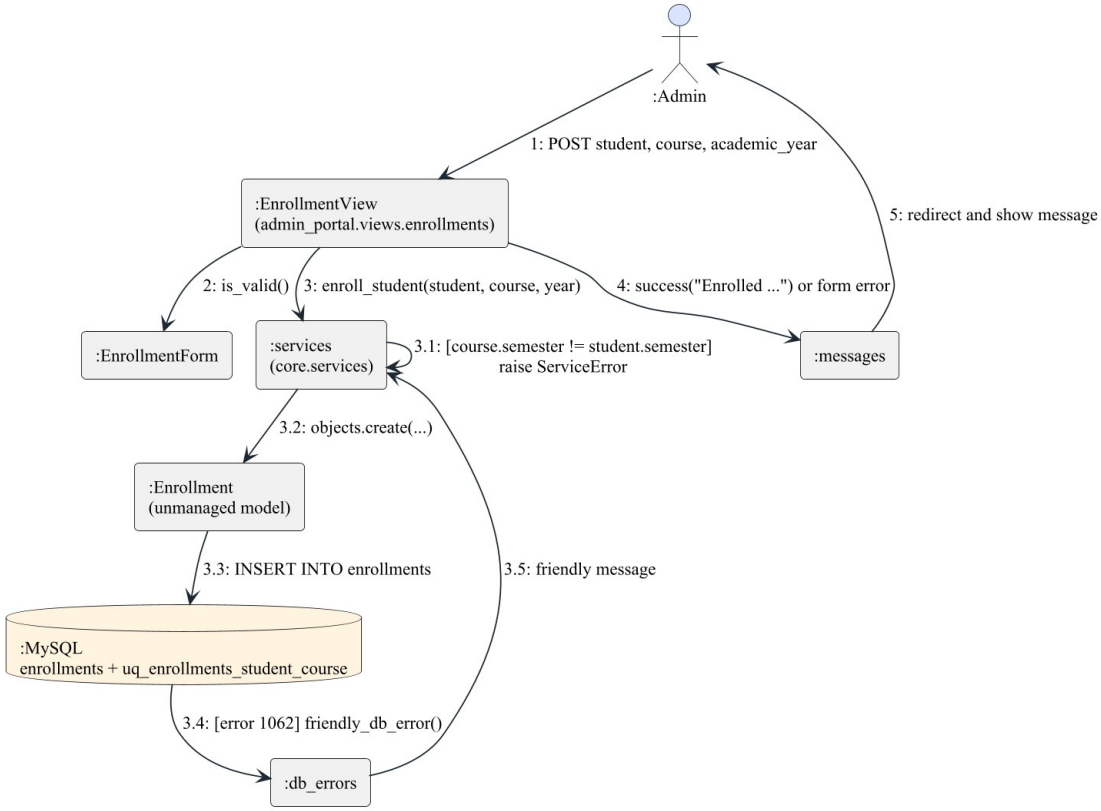


Figure 4.11: Collaboration (communication) diagram: admin enrolls a student

4.5.6 Activity Diagram

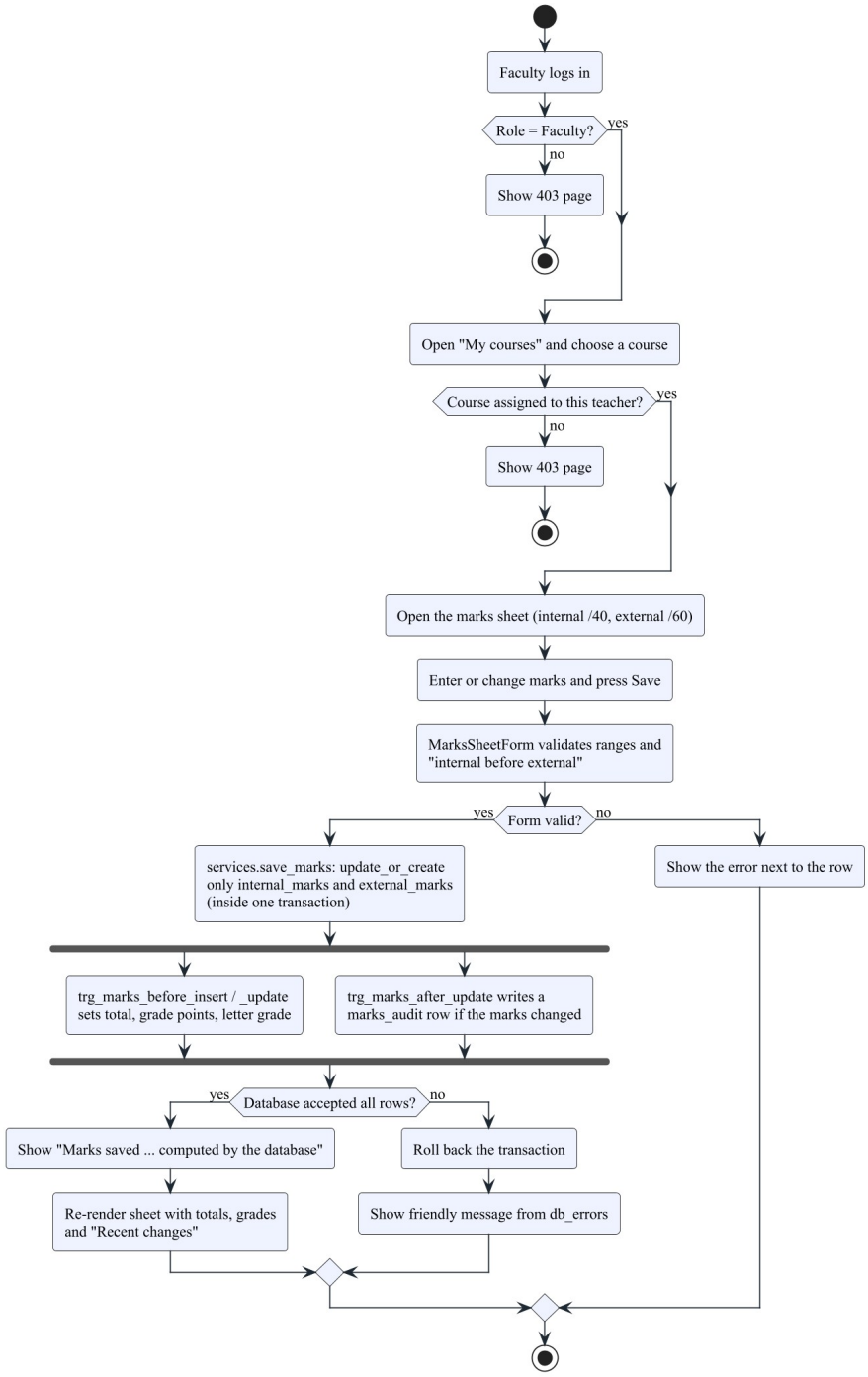


Figure 4.12: Activity diagram: faculty marks entry and grading

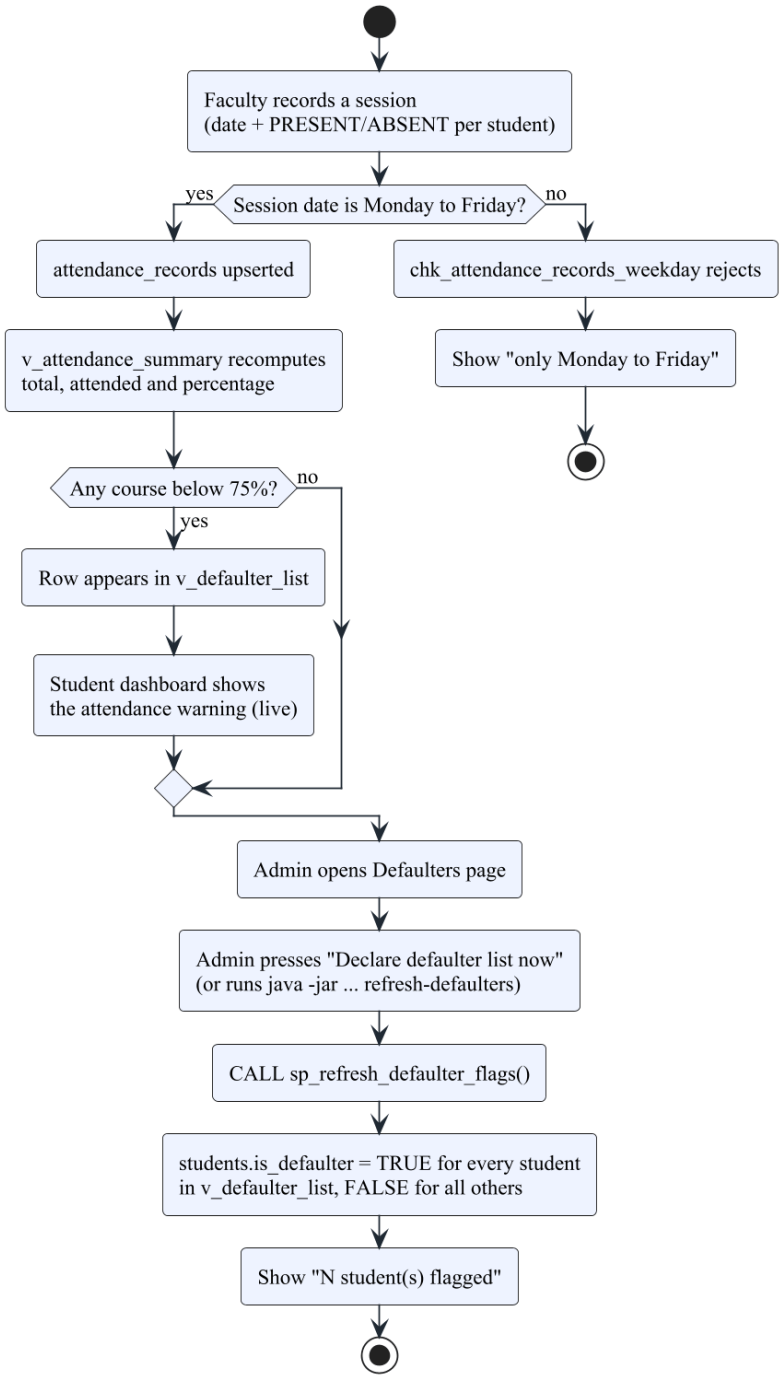


Figure 4.13: Activity diagram: from attendance to a declared defaulter

4.5.7 State Diagram

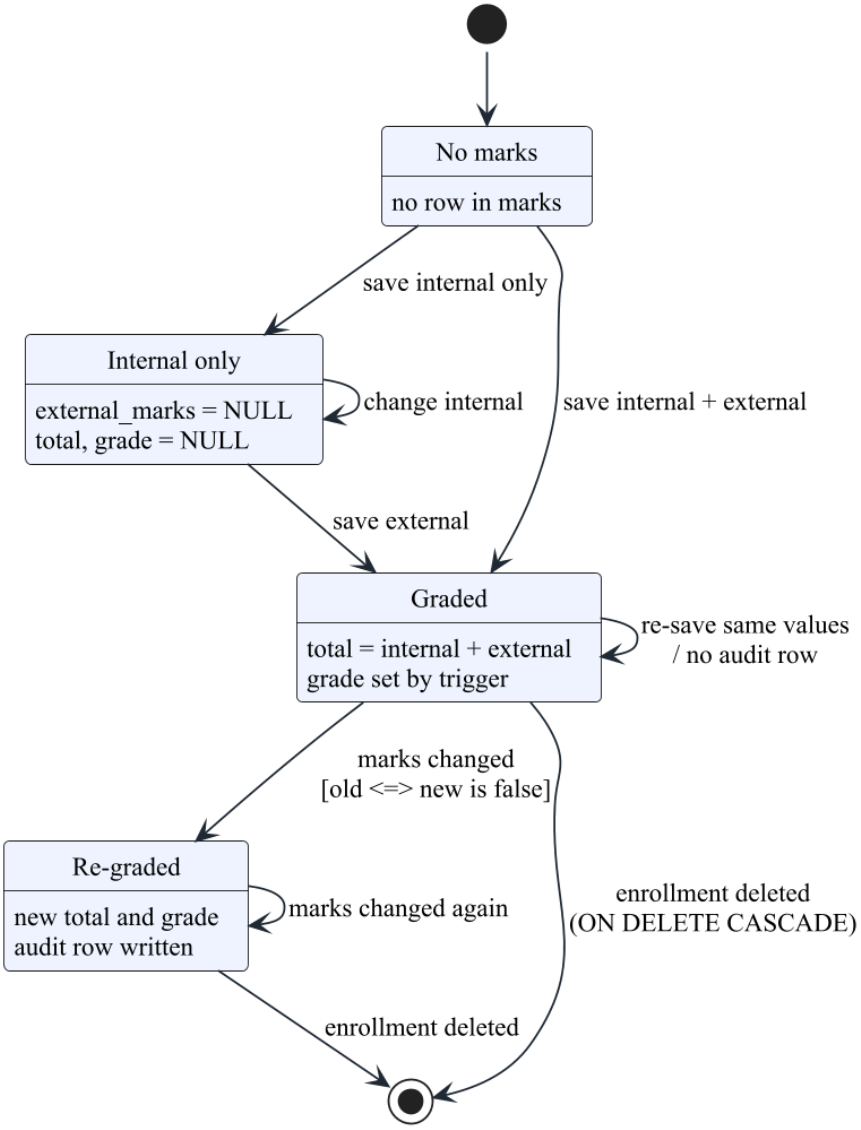


Figure 4.14: Statechart diagram: life cycle of a marks record

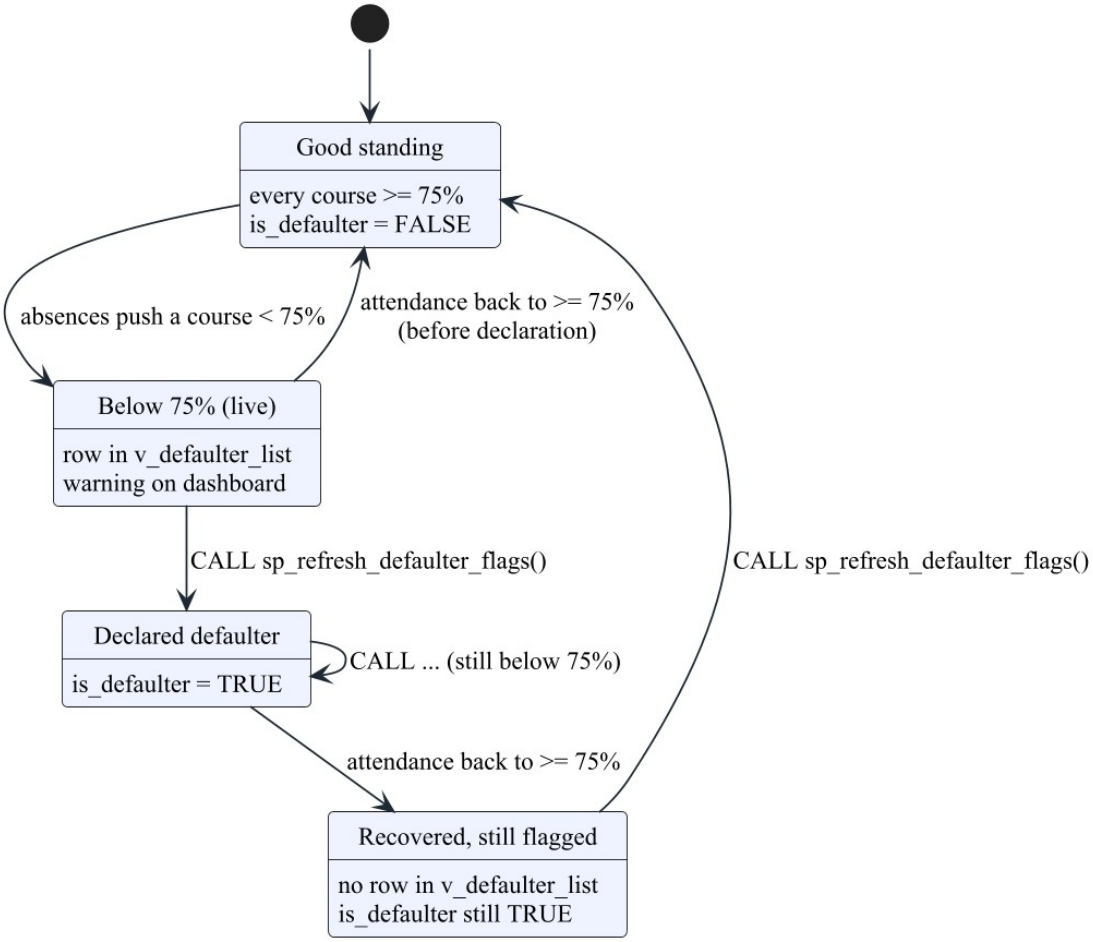


Figure 4.15: Statechart diagram: attendance status of a student

4.5.8 Component Diagram

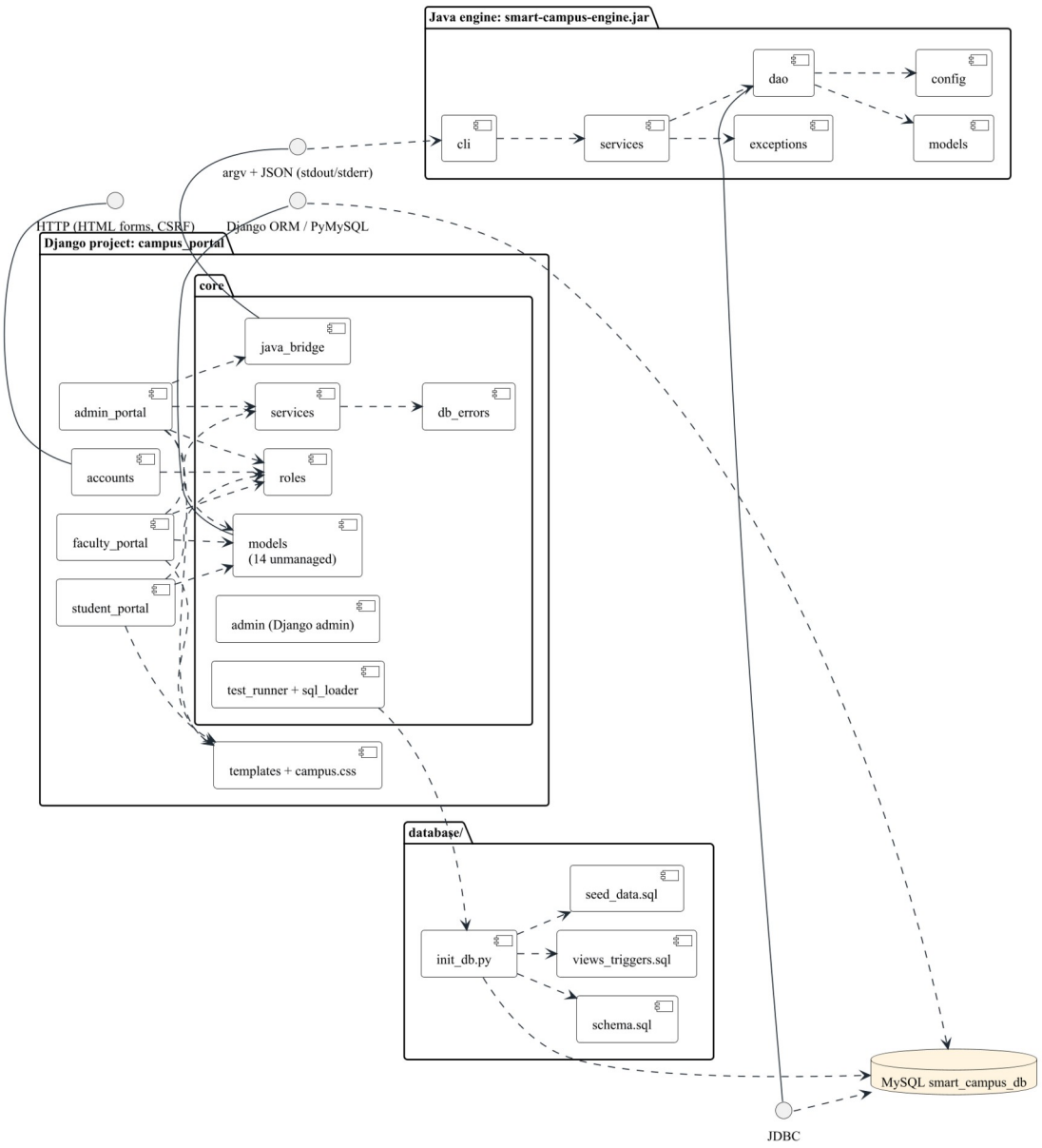


Figure 4.16: Component diagram

4.5.9 Deployment Diagram

For the lab the whole system runs on one machine: the browser, the Django development server, the JVM started per request and the MySQL server all communicate over localhost.

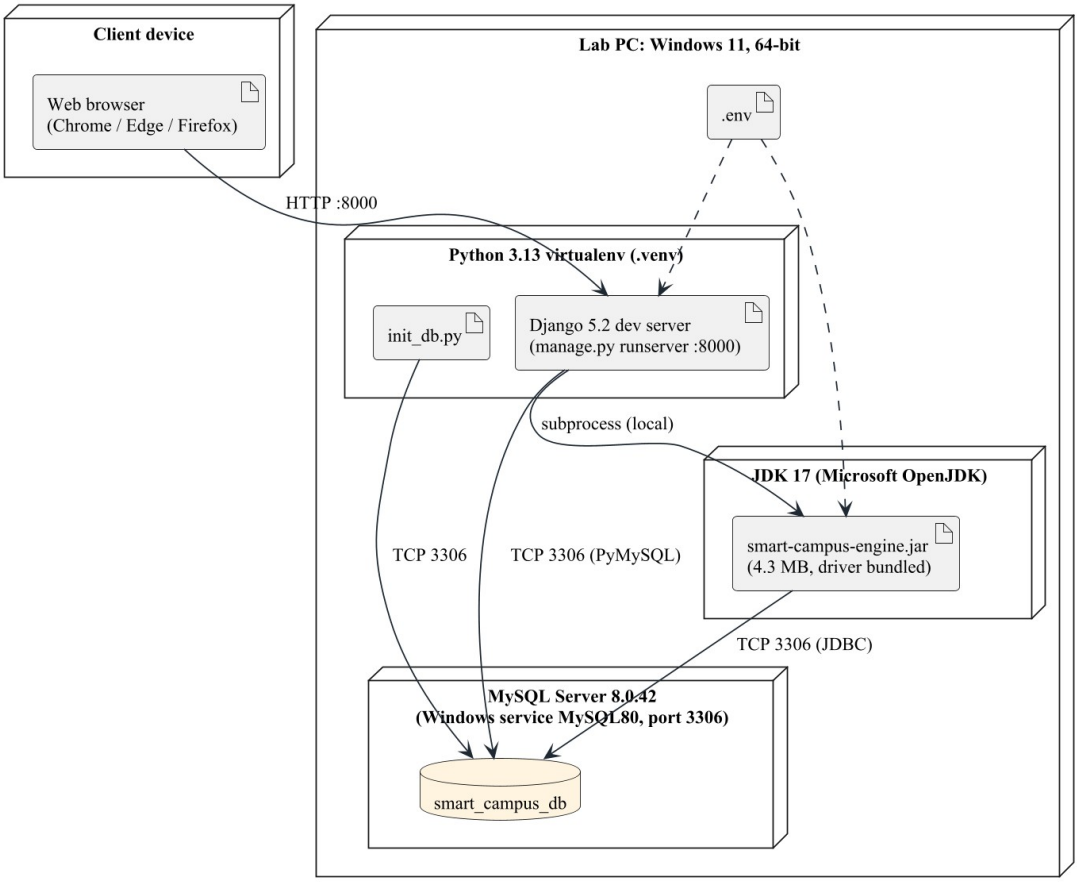


Figure 4.17: Deployment diagram (as run for this project)

4.5.10 CASE tools used

The diagrams were **not** drawn with StarUML or Rational Rose. They were written as plain-text **PlantUML 1.2026.9** sources (`docs/src/diagrams/*.puml`, with a shared style file `_style.iuml`) and rendered to PNG at 170 dpi by `docs/src/render_diagrams.py`, using PlantUML's built-in Smetana layout engine so that Graphviz is not needed. The project's own design notes in `docs/database_design.md` and `docs/django_portal.md` use **Mermaid**, which GitHub and VS Code render directly. Text-based diagrams were chosen because they are kept in Git next to the code, show clear differences when changed, and can be regenerated with one command.

Table 4.3: CASE and support tools

Tool	Used for
PlantUML 1.2026.9 (Smetana layout)	All 21 report diagrams: UML, ERD, DFD
Mermaid (GitHub / VS Code preview)	ER, flow and sequence diagrams in docs/*.md
Visual Studio Code 1.143.1	Editing code, SQL and diagram sources
Git 2.53.0 and GitHub	Version control (Appendix A)

Tool	Used for
Maven Wrapper, JUnit 6, JaCoCo 0.8.16	Java build, unit tests and coverage
Django test runner, coverage.py 7.17.0	Django tests and coverage
Playwright 1.64.1 (Chromium)	Screenshots and page timings
python-docx 1.2.0, LibreOffice 26.8.2	Building and checking this report

4.6 Data Flow Diagrams

The data flow diagrams use the Gane-Sarson notation: external entities as rectangles, processes as numbered rounded boxes and data stores as open-ended boxes named after the real tables.

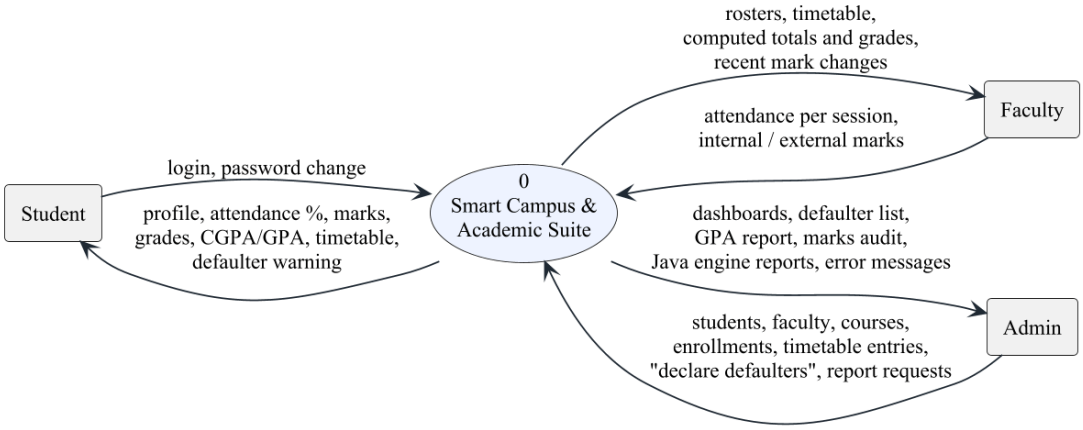


Figure 4.18: Data flow diagram, level 0 (context diagram)

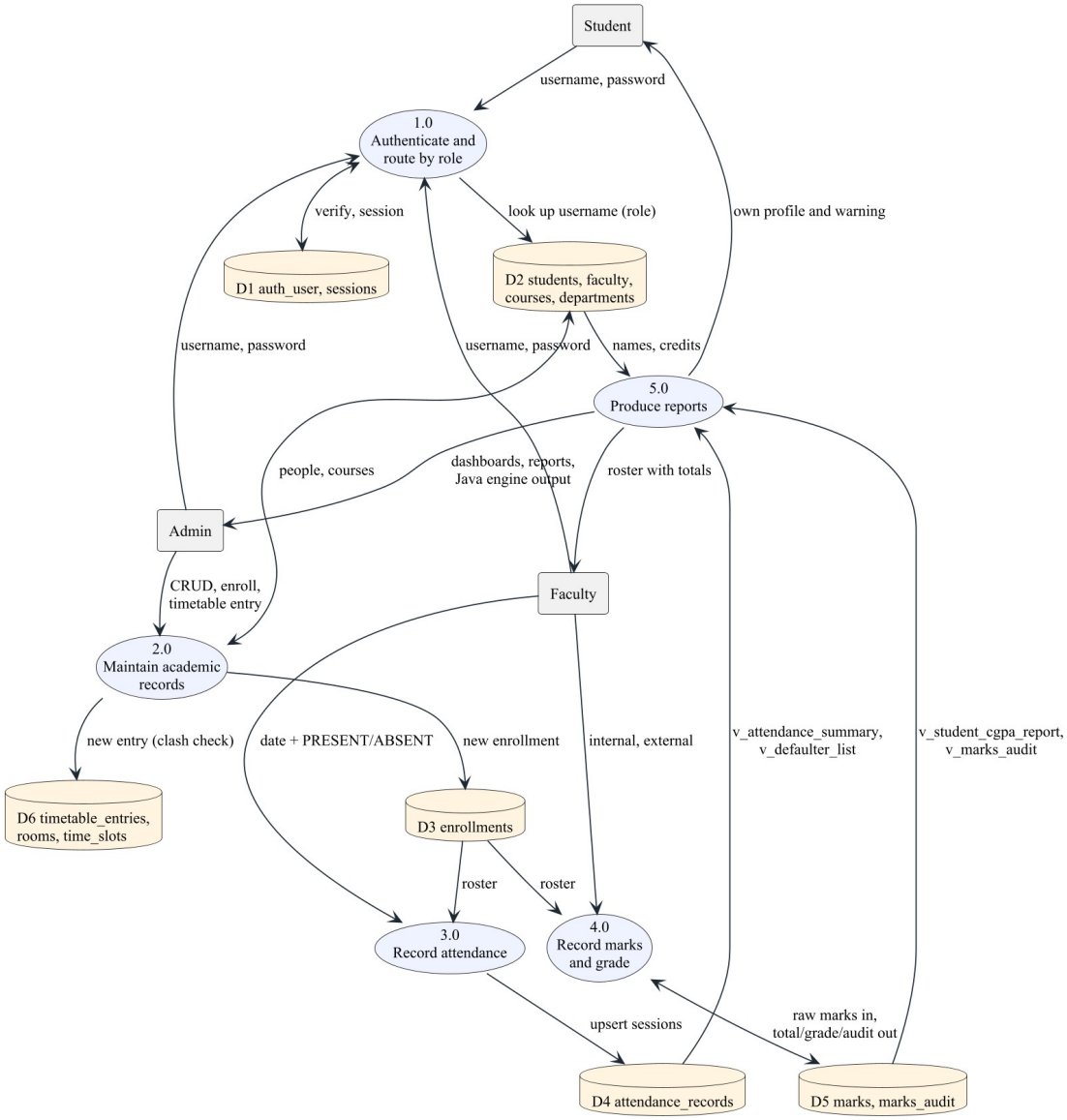


Figure 4.19: Data flow diagram, level 1

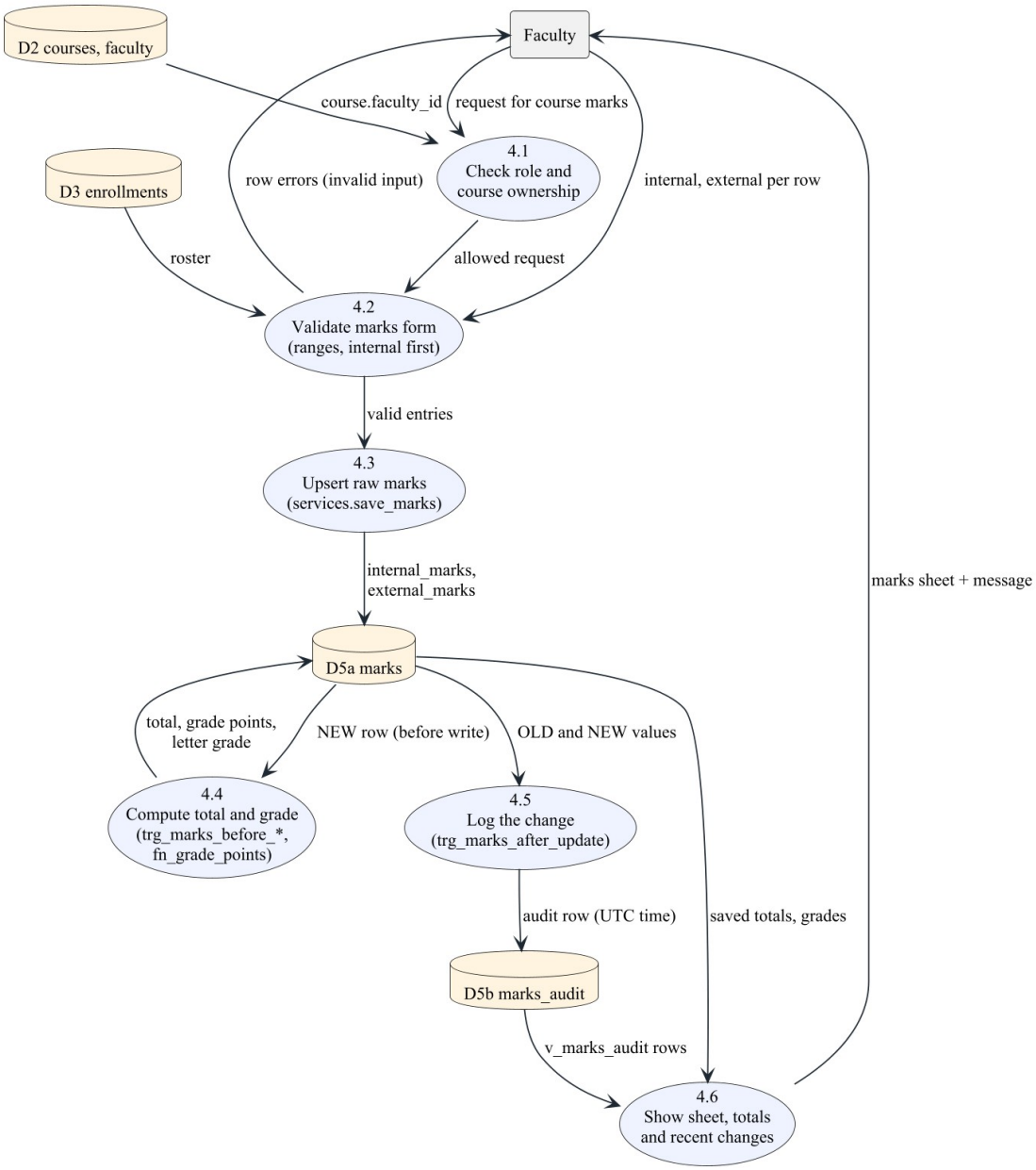


Figure 4.20: Data flow diagram, level 2: process 4.0 "Record marks and grade"

4.7 Entity Relationship Diagram (ERD)

The ERD shows the 11 domain tables with their primary keys, foreign keys and cardinalities. `marks_audit` deliberately has no foreign key, so the history survives when a marks row is deleted.

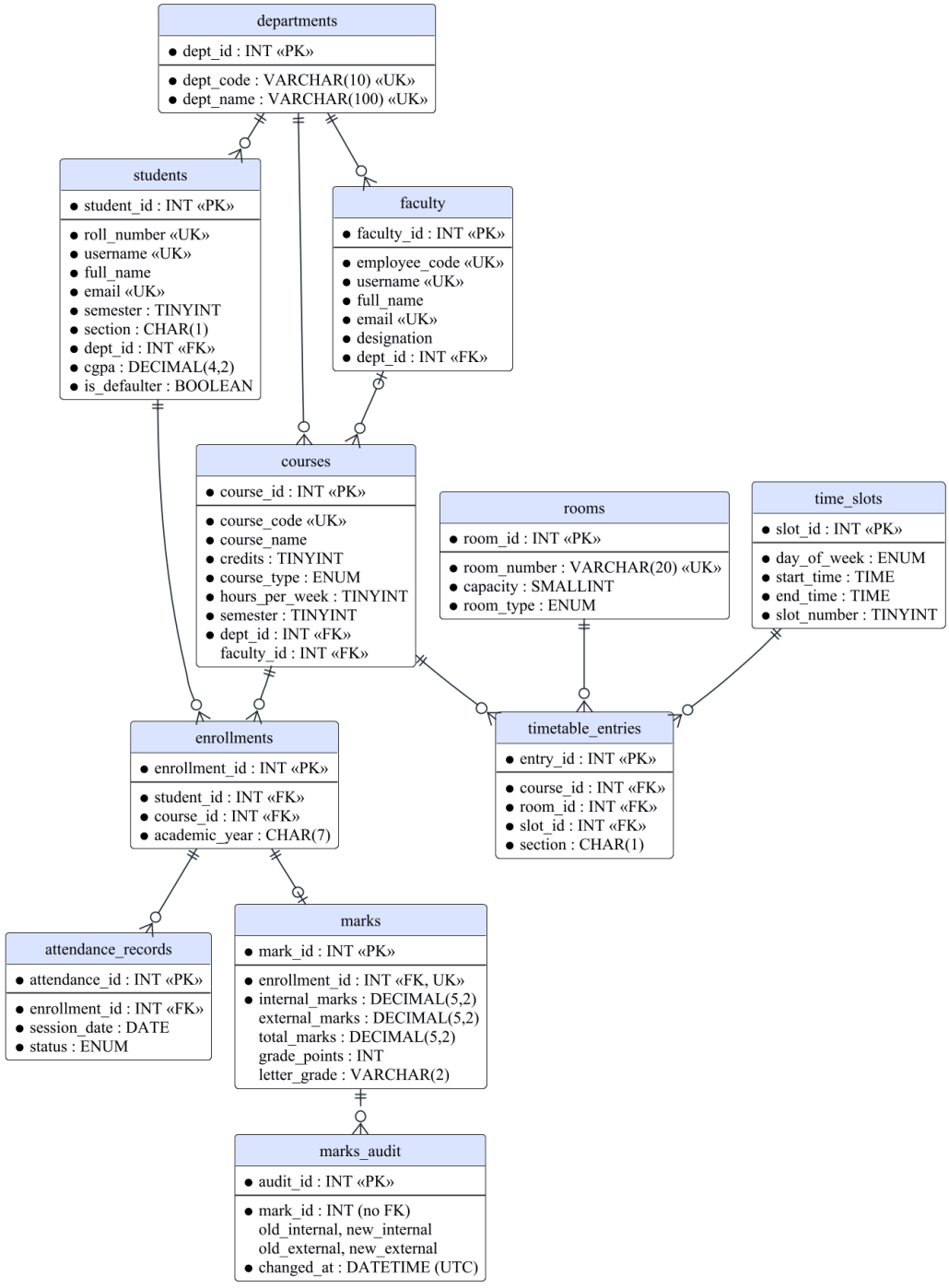


Figure 4.21: Entity relationship diagram (11 tables of smart_campus_db)

4.8 Database design

The database `smart_campus_db` runs on MySQL 8.0.42 with InnoDB and `utf8mb4`. It has 11 domain tables, 4 views, 9 triggers, 2 stored functions and 1 stored procedure, with 11 foreign keys, 17 UNIQUE constraints and 23 CHECK constraints. Every table has a single-column surrogate primary key; natural keys (roll number, course

code, student + course, room + slot, slot + section) are UNIQUE constraints.

Normalisation. Every column is atomic (1NF). Every table has a single-column key, so no attribute can depend on part of a key (2NF). Non-key attributes depend only on the key: for example a student row stores `dept_id`, not the department name, and a course stores `faculty_id`, not the teacher's name (3NF). The full functional dependencies are recorded in `docs/database_design.md`.

Table 4.4: Main functional dependencies

Table	Key functional dependencies
students	student_id -> roll_number, username, email, full_name, semester, section, dept_id, cgpa, is_defaulter; roll_number -> student_id
courses	course_id -> course_code, course_name, credits, course_type, hours_per_week, semester, dept_id, faculty_id
enrollments	enrollment_id -> student_id, course_id, academic_year; (student_id, course_id) -> enrollment_id
attendance_records	attendance_id -> enrollment_id, session_date, status; (enrollment_id, session_date) -> attendance_id
marks	mark_id -> enrollment_id, internal_marks, external_marks; total_marks, grade_points and letter_grade are maintained by triggers (documented exception)
timetable_entries	entry_id -> course_id, room_id, slot_id, section; (room_id, slot_id) -> entry_id; (slot_id, section) -> entry_id

Derived values. Attendance percentage, defaulter status and GPA are computed in views. Three stored copies are documented exceptions: `marks.total_marks / grade_points / letter_grade` (kept by triggers so they can never disagree with the raw marks), `students.cgpa` (the official CGPA, which includes earlier semesters) and `students.is_defaulter` (set only when the defaulter list is declared).

Table 4.5: Triggers in smart_campus_db

Trigger	Timing	Event	Table	Purpose
trg_faculty_before_insert	BEFORE	INSERT	faculty	Username must not exist in students
trg_faculty_before_update	BEFORE	UPDATE	faculty	Same rule on update
trg_marks_after_update	AFTER	UPDATE	marks	Write old/new marks to marks_audit
trg_marks_before_insert	BEFORE	INSERT	marks	Compute total, grade points, letter grade
trg_marks_before_update	BEFORE	UPDATE	marks	Recompute total and grade
trg_prevent_timetable_faculty_clash	BEFORE	INSERT	timetable_entries	Refuse a teacher double-booking
trg_prevent_timetable_faculty_clash_update	BEFORE	UPDATE	timetable_entries	Same rule on update

Trigger	Timing	Event	Table	Purpose
trg_students_before_insert	BEFORE	INSERT	students	Username must not exist in faculty
trg_students_before_update	BEFORE	UPDATE	students	Same rule on update

Table 4.6: Views, functions and procedure

View / routine	Purpose
v_attendance_summary	Classes held, attended and percentage per enrollment
v_defaulter_list	Enrollments below 75% attendance
v_student_cgpa_report	Credit-weighted GPA of graded courses, next to the official CGPA
v_marks_audit	Audit rows joined with roll number, course and old/new grades
fn_grade_points(total)	Grade points 10/9/8/7/6/5/0 from the total
fn_letter_grade(points)	Letter grade O, A+, A, B+, B, P, F
sp_refresh_defaulter_flags()	Sets students.is_defaulter from v_defaulter_list

4.9 Database tables and attributes

The following tables were generated from MySQL's `information_schema` for the running database, so they match `database/schema.sql` exactly.

Table 4.7: Table departments

Attribute	Data type	Null	Key / notes
dept_id	int	No	PK, auto increment
dept_code	varchar(10)	No	UNIQUE
dept_name	varchar(100)	No	UNIQUE

Table 4.8: Table faculty

Attribute	Data type	Null	Key / notes
faculty_id	int	No	PK, auto increment
employee_code	varchar(20)	No	UNIQUE
username	varchar(150)	No	UNIQUE
full_name	varchar(100)	No	
email	varchar(254)	No	UNIQUE
designation	varchar(50)	No	
dept_id	int	No	FK -> departments (on delete restrict)

Table 4.9: Table students

Attribute	Data type	Null	Key / notes
student_id	int	No	PK, auto increment
roll_number	varchar(20)	No	UNIQUE
username	varchar(150)	No	UNIQUE
full_name	varchar(100)	No	
email	varchar(254)	No	UNIQUE
semester	tinyint	No	
section	char(1)	No	

Attribute	Data type	Null	Key / notes
dept_id	int	No	FK -> departments (on delete restrict)
cgpa	decimal(4,2)	No	default 0.00
is_defaulter	tinyint(1)	No	default 0

Table 4.10: Table courses

Attribute	Data type	Null	Key / notes
course_id	int	No	PK, auto increment
course_code	varchar(10)	No	UNIQUE
course_name	varchar(100)	No	
credits	tinyint	No	
course_type	enum('THEORY','LAB')	No	
hours_per_week	tinyint	No	
semester	tinyint	No	
dept_id	int	No	FK -> departments (on delete restrict)
faculty_id	int	Yes	FK -> faculty (on delete set null)

Table 4.11: Table enrollments

Attribute	Data type	Null	Key / notes
enrollment_id	int	No	PK, auto increment
student_id	int	No	FK -> students (on delete cascade)
course_id	int	No	FK -> courses (on delete restrict)
academic_year	char(7)	No	

Table 4.12: Table attendance_records

Attribute	Data type	Null	Key / notes
attendance_id	int	No	PK, auto increment
enrollment_id	int	No	FK -> enrollments (on delete cascade)
session_date	date	No	
status	enum('PRESENT','ABSENT')	No	

Table 4.13: Table marks

Attribute	Data type	Null	Key / notes
mark_id	int	No	PK, auto increment
enrollment_id	int	No	FK -> enrollments (on delete cascade), UNIQUE
internal_marks	decimal(5,2)	No	
external_marks	decimal(5,2)	Yes	
total_marks	decimal(5,2)	Yes	
grade_points	int	Yes	
letter_grade	varchar(2)	Yes	

Table 4.14: Table marks_audit

Attribute	Data type	Null	Key / notes
audit_id	int	No	PK, auto increment

Attribute	Data type	Null	Key / notes
mark_id	int	No	
old_internal	decimal(5,2)	Yes	
new_internal	decimal(5,2)	Yes	
old_external	decimal(5,2)	Yes	
new_external	decimal(5,2)	Yes	
changed_at	datetime	No	

Table 4.15: Table rooms

Attribute	Data type	Null	Key / notes
room_id	int	No	PK, auto increment
room_number	varchar(20)	No	UNIQUE
capacity	smallint	No	
room_type	enum('LECTURE','LAB')	No	

Table 4.16: Table time_slots

Attribute	Data type	Null	Key / notes
slot_id	int	No	PK, auto increment
day_of_week	enum('MON','TUE','WED','THU','FRI')	No	
start_time	time	No	
end_time	time	No	
slot_number	tinyint	No	

Table 4.17: Table timetable_entries

Attribute	Data type	Null	Key / notes
entry_id	int	No	PK, auto increment
course_id	int	No	FK -> courses (on delete cascade)
room_id	int	No	FK -> rooms (on delete restrict)
slot_id	int	No	FK -> time_slots (on delete restrict)
section	char(1)	No	

Table 4.18: CHECK constraints

Table	CHECK constraint	Rule (as stored by MySQL)
attendance_records	chk_attendance_records_weekday	(dayofweek(`session_date`) between 2 and 6)
courses	chk_courses_credits	(`credits` between 1 and 6)
courses	chk_courses_hours	(`hours_per_week` between 1 and 10)
courses	chk_courses_semester	(`semester` between 1 and 8)
departments	chk_departments_dept_code	regexp_like(`dept_code`,_utf8mb4\'^[A-Z]{2,10}$ \',_utf8mb4\'c\')
enrollments	chk_enrollments_academic_year	regexp_like(`academic_year`,_utf8mb4\'^[0-9]{4}-[0-9]{2}$\')
faculty	chk_faculty_designation	(`designation` in (_utf8mb4\'Professor\',_utf8mb4\'Associate Professor\',_utf8mb4\'Assistant Professor\',_utf8mb4\'Lecturer\'))
faculty	chk_faculty_email	(`email` like _utf8mb4\'_%@_%._%\')
faculty	chk_faculty_username	regexp_like(`username`,_utf8mb4\'^[A-Za-z0-9.@+_-]+$\')

Table	CHECK constraint	Rule (as stored by MySQL)
marks	chk_marks_external	(external_marks between 0 and 60)
marks	chk_marks_grade_points	(grade_points in (0,5,6,7,8,9,10))
marks	chk_marks_internal	(internal_marks between 0 and 40)
marks	chk_marks_letter_grade	(letter_grade in (_utf8mb4\'O\',_utf8mb4\'A+ \',_utf8mb4\'A\',_utf8mb4\'B+ \',_utf8mb4\'B\',_utf8mb4\'P\',_utf8mb4\'F\'))
marks	chk_marks_total	(total_marks between 0 and 100)
rooms	chk_rooms_capacity	(capacity between 1 and 500)
students	chk_students_cgpa	(cgpa between 0.00 and 10.00)
students	chk_students_email	(email like _utf8mb4\'_%@_%._%\')
students	chk_students_section	regexp_like(section,_utf8mb4\'^[A-Z]$ \',_utf8mb4\'c\')
students	chk_students_semester	(semester between 1 and 8)
students	chk_students_username	regexp_like(username,_utf8mb4\'^[A-Za-z0-9.@+_-] +$\')
time_slots	chk_time_slots_order	(end_time > start_time)
time_slots	chk_time_slots_slot_number	(slot_number between 1 and 10)
timetable_entries	chk_timetable_entries_section	regexp_like(section,_utf8mb4\'^[A-Z]$ \',_utf8mb4\'c\')

4.10 Input design

All input goes through Django forms, which validate types and ranges before the database checks them again. Inputs are kept short and use choices where possible.

Table 4.19: Input forms

Form	Fields	Validation
Login	Username, password	Django authentication; error on wrong credentials
Attendance sheet	Date; Present/Absent per student	Date picker; weekday CHECK in MySQL
Marks sheet	Internal (0-40), external (0-60) per student	Decimal ranges; external needs internal; CHECK in MySQL
Student / faculty / course	All master-data fields	ModelForm; UNIQUE, CHECK and username triggers
Enrollment	Student, course, academic year	Semester match (app rule); UNIQUE key
Timetable entry	Course, slot, room, section	Clash trigger; UNIQUE (slot, room) and (slot, section)
Engine command	Command, roll number, course code	Allow-list and regex in form and bridge
Search	Free text q	ORM icontains (parameterised)

4.11 Output design

Table 4.20: Outputs of the system

Output	Shown to	Source
Attendance and marks cards, warning banner	Student	v_attendance_summary, marks, v_defaulter_list
Course roster, marks sheet with grades	Faculty	enrollments, marks (trigger-computed)
Dashboard counts and averages	Admin	tables and views

Output	Shown to	Source
Defaulter list, GPA report, marks audit	Admin	v_defaulter_list, v_student_cgpa_report, v_marks_audit
Engine result tables and error cards	Admin	JSON from the Java engine
Text tables, JSON, CSV files	Terminal user	Java engine CLI

4.12 User interface design

The interface is written in semantic HTML5 (`header`, `nav`, `main`, `section`, `table`, `form`, `label`) with one hand-written stylesheet, `static/css/campus.css`. Colours and spacing are CSS custom properties defined once; layouts use Flexbox and Grid, and media queries stack the cards and make tables scroll at phone width. No CSS framework or JavaScript framework is used. Every role has the same header with the user's name and role, a navigation bar and a log-out button. Messages appear as coloured banners; tables show a clear "no records" row when empty. The login page is shown below; the other screens appear in Sections 5.10 and 9.3.

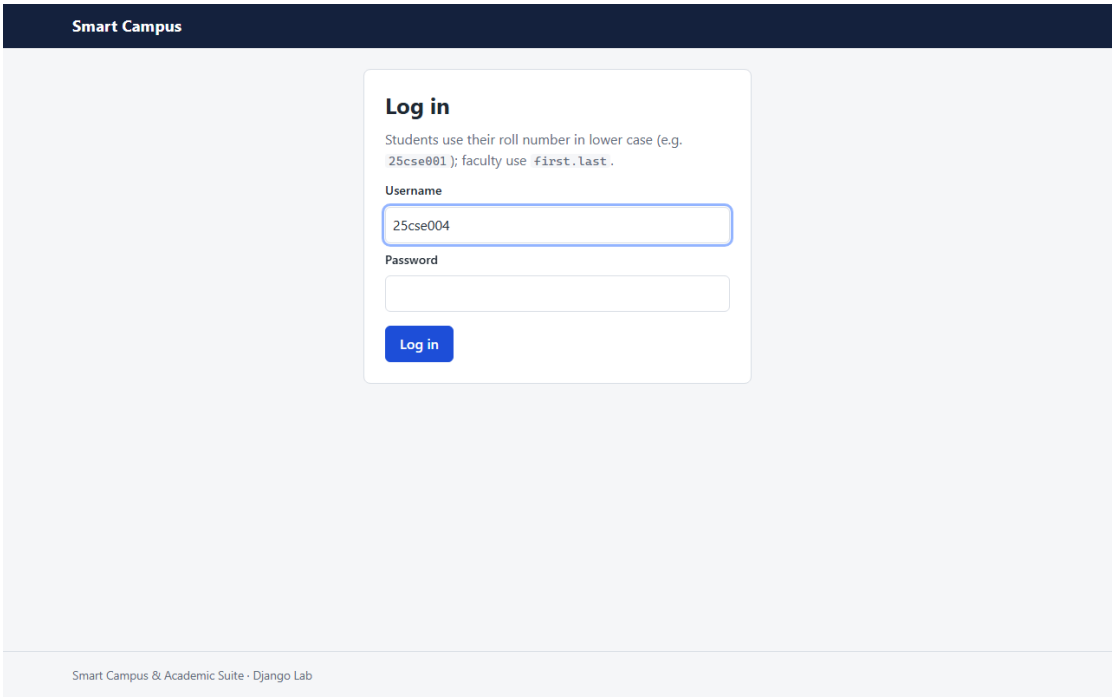


Figure 4.22: Login page

4.13 Module design

Table 4.21: Module design

Module	Inputs	Processing	Outputs
Authentication & roles	Username, password	Django auth; role derived from tables	Session, redirect
Student portal	Session	Read own rows only	Record page

Module	Inputs	Processing	Outputs
Attendance	Date, statuses	update_or_create in one transaction	attendance_records
Marks & grading	Internal, external	Triggers compute grade; AFTER trigger audits	marks, marks_audit
Master data (CRUD)	Form fields	services.save_form / delete_object	Rows or friendly error
Enrollment	Student, course, year	Semester check, insert	enrollments
Timetable	Course, slot, room, section	Clash trigger + UNIQUE	timetable_entries
Reports & defaulters	None / declare button	Views; CALL procedure	Report pages, flags
Java engine	Command, args	DAO -> views/tables; concurrent dashboard	Text, JSON, CSV
Bridge	Command form	Validate, run JAR, parse JSON	Result or typed error

4.14 Security and access control design

Access control follows role-based access control [2]: permissions belong to the three roles, and a user's role is derived on every request, never stored or taken from the URL.

Table 4.22: Role and access matrix (HTTP status returned)

URL prefix	Student	Faculty	Admin	Anonymous
/student/	200	403	403	302 to login
/faculty/ (own courses only)	403	200	403	302 to login
/admin-portal/	403	403	200	302 to login
/engine/	403	403	200	302 to login
/accounts/password/	200	200	200	302 to login
/django-admin/	-	-	200 (superuser)	login

Table 4.23: Security threats and measures

Threat	Design measure
Unauthorised page access	`@role_required` / `RoleRequiredMixin` on every view; a test walks the whole URL configuration
Reading another person's data	Person found from the login (`get_person`); no ids from the URL; faculty course pages check the teacher
SQL injection	Django ORM and Java `PreparedStatement` only
Cross-site scripting	Django template auto-escaping
Cross-site request forgery	CSRF token on every POST form; logout is POST only
Command injection via the bridge	Argument list, `shell=False`, allow-list of 6 commands, regex per argument, 30 s timeout
Credential leaks	Credentials only in git-ignored `.env`; hashed passwords; demo password refused when DEBUG is off
Session theft	HttpOnly cookies, SameSite=Lax; secure cookies and HSTS when DJANGO_HTTPS=True
Silent marks tampering	AFTER UPDATE trigger records every change in marks_audit

CHAPTER 5: SYSTEM DEVELOPMENT AND IMPLEMENTATION

5.1 Introduction

This chapter describes how the design of Chapter 4 was implemented: the development environment, the technologies, the database implementation, each module, the main algorithms and real code from the repository, followed by screenshots of the running system. The source code is version-controlled with Git; configuration management is described in Appendix A.

5.2 Development environment

Table 5.1: Development environment

Item	Details
Processor	AMD Ryzen 5 3550H (4 cores, 8 threads)
Memory	13.9 GB usable RAM
Operating system	Windows 11 Home Single Language, 64-bit, build 26200
Editor / IDE	Visual Studio Code 1.143.1
Version control	Git 2.53.0 with a GitHub remote
Database server	MySQL Community Server 8.0.42
Java	OpenJDK 17.0.20.1 (LTS); build with 3.9.16 (Maven Wrapper)
Python	3.13.3 in a virtual environment (.venv)
Shell	Windows PowerShell 5.1 and Git Bash

5.3 Technologies used

Table 5.2: Technologies

Layer	Technology	Why it was chosen
Database	MySQL 8.0 (InnoDB, utf8mb4)	CHECK constraints, triggers, views, stored routines and transactions; required by the DBMS lab
Engine	Java 17 + JDBC	Core OOP and raw JDBC are what the Java lab assesses; LTS release
Web	Django 5.2 LTS (MVT)	Built-in authentication, forms, CSRF, ORM and admin; LTS until 2028
Front end	HTML5 + CSS3 (no framework)	Semantic markup and hand-written responsive CSS
Integration	subprocess + JSON	Language-neutral contract between Django and the JAR
Documentation	Markdown, Mermaid, PlantUML	Text sources versioned with the code

5.4 Programming languages

Table 5.3: Source size by language (counted from the repository)

Language / file type	Files	Lines
SQL (database/*.sql)	4	833
Python: database/init_db.py	1	243
Java (main)	60	2846
Java (tests)	5	282
Python: Django (excluding tests)	44	1564
Python: Django tests	6	572
HTML templates	26	830
CSS	1	155

5.5 Frameworks and libraries

Table 5.4: Frameworks and libraries

Name	Version	Purpose
Django	5.2.18	Web framework (portal)
PyMySQL	1.2.3	Pure-Python MySQL driver, installed as MySQLdb
python-dotenv	1.2.4	Loads the shared .env file
cryptography	50.0.2	MySQL 8 caching_sha2_password authentication
MySQL Connector/J	9.7.0	JDBC driver for the Java engine
JUnit Jupiter	6.1.3	Java unit tests
maven-shade-plugin	3.6.2	Builds the runnable fat JAR

5.6 Database implementation

The database is created by three re-runnable scripts executed in order by `database/init_db.py`, which reads the credentials from `.env`, splits the scripts (it understands `DELIMITER`, quotes and comments) and prints the row counts:

- `schema.sql`: `CREATE DATABASE`, then `DROP TABLE IF EXISTS` and `CREATE TABLE` for the 11 tables with named PK, FK, UNIQUE and CHECK constraints.
- `views_triggers.sql`: 2 functions, 4 views, 9 triggers and 1 procedure (each dropped and re-created).
- `seed_data.sql`: demo data: 2 departments, 5 faculty, 10 students, 6 courses, 5 rooms, 20 time slots, 20 timetable entries, 60 enrollments, 720 attendance records and 60 marks rows.
- `queries.sql`: lab demonstration queries (joins, subqueries, window functions, transactions); not run by init_db.py.

Django's own tables (`auth_*`, `django_*`) are created by `manage.py migrate` in the

same database; `init_db.py` never touches them, so logins survive a rebuild.

5.7 Module-wise implementation

5.7.1 Authentication and roles

`accounts` uses Django's login view. `core/roles.py` derives the role (superuser, else a `students.username` match, else a `faculty.username` match) and caches the person per request. `role_required` and `RoleRequiredMixin` guard every view.

5.7.2 Student portal

`student_portal/views.py` loads the student from the login, then reads `v_attendance_summary`, the marks, `v_student_cgpa_report` and the timetable for that student only. The warning banner appears when the student has rows in `v_defaulter_list`.

5.7.3 Faculty portal

Course pages check that `course.faculty` is the logged-in teacher. The attendance sheet and marks sheet are formsets; saving calls `services.save_attendance` or `services.save_marks`, which write only the raw columns inside one transaction.

5.7.4 Admin portal

Class-based list, create, update and delete views for students, faculty and courses with search and pagination; enrollment and timetable forms; reports that read the views; the "declare" button calls the stored procedure; the marks audit page reads `v_marks_audit`.

5.7.5 Java engine

`Main` delegates to `cli.Cli`, which parses the arguments, looks the command up in the registry and calls a service. Services depend on DAO interfaces; `cli.Engine` wires the JDBC implementations. Output goes through a text renderer or `JsonWriter`.

5.7.6 Django to Java bridge

`core/java_bridge.py` validates the command against an allow-list, builds the argument list, checks that the JAR exists, runs it with a timeout and parses the single JSON line. The `/engine/` view renders the result or a typed error card.

5.7.7 Team contributions

The system was designed, implemented, tested and documented jointly by the four members of the project team; the work is held in one repository (Appendix A).

Table 5.5: Project team

S.No	Name	Roll number
1	V. Sai Rishi	25911A67B7
2	Mummadisetty Advyth	25911A67A3
3	P.V.S Sebareesh	25911A67C4
4	K. Bharath Reddy	25911A6784

5.8 Algorithm or pseudocode

5.8.1 Grading (database trigger)

```
BEFORE INSERT/UPDATE ON marks:
  IF internal IS NULL OR external IS NULL THEN total, points, letter := NULL
  ELSE total  := internal + external
       points := fn_grade_points(total)   -- >=90:10, >=80:9, >=70:8, >=60:7,
>=50:6, >=40:5, else 0
       letter := fn_letter_grade(points)  -- O, A+, A, B+, B, P, F
AFTER UPDATE ON marks:
  IF NOT (old.internal <=> new.internal AND old.external <=> new.external) THEN
     INSERT INTO marks_audit(old/new internal, old/new external, changed_at =
UTC_TIMESTAMP())
```

5.8.2 Defaulter detection and declaration

```
v_attendance_summary: for each enrollment
    held := COUNT(records); attended := COUNT(status = 'PRESENT')
    percentage := ROUND(100 * attended / held, 2)   (NULL when no classes held)
v_defaulter_list := rows of v_attendance_summary with percentage < 75
sp_refresh_defaulter_flags():
    UPDATE students SET is_defaulter = (student_id IN v_defaulter_list)
```

5.8.3 Role check on every request

```
role_required(roles)(view):
    if user not authenticated: redirect to login with ?next=
    role := superuser ? ADMIN : (username in students ? STUDENT : (username in
faculty ? FACULTY : NONE))
    if role not in roles: raise PermissionDenied  -> 403 page
    return view(request)
```

5.8.4 Bridge call

```
run_engine(command, args):
    if command not in ALLOWED or any arg fails its regex: raise
EngineError(NotAllowed/BadArgument)
    if JAR file missing: raise EngineError(JarMissing)
    proc := subprocess.run([java, -Dfile.encoding=UTF-8, -jar, JAR, command,
args..., --json],
                           shell=False, timeout=30)
    if exit code 0: return data from the JSON line on stdout
    else: raise EngineError(error.type from the JSON line on stderr)
```

5.9 Important code snippets

The snippets below are copied from the repository without changes.

5.9.1 Grade points function (database/views_triggers.sql)

```
CREATE FUNCTION fn_grade_points(p_total DECIMAL(5, 2))
    RETURNS INT
    DETERMINISTIC
    NO SQL
BEGIN
    RETURN CASE
        WHEN p_total IS NULL THEN NULL
        WHEN p_total >= 90 THEN 10
        WHEN p_total >= 80 THEN 9
        WHEN p_total >= 70 THEN 8
        WHEN p_total >= 60 THEN 7
        WHEN p_total >= 50 THEN 6
        WHEN p_total >= 40 THEN 5
        ELSE 0
    END;
END$$
```

5.9.2 Timetable clash trigger (database/views_triggers.sql)

```
CREATE TRIGGER trg_prevent_timetable_faculty_clash
    BEFORE INSERT ON timetable_entries
    FOR EACH ROW
BEGIN
    DECLARE v_faculty_id INT;
    DECLARE v_clash_course VARCHAR(10);
    DECLARE v_msg VARCHAR(255);

    SET v_faculty_id = (SELECT faculty_id FROM courses WHERE course_id =
NEW.course_id);

    IF v_faculty_id IS NOT NULL THEN
        SET v_clash_course = (
            SELECT c.course_code
            FROM timetable_entries te
                JOIN courses c ON c.course_id = te.course_id
            WHERE te.slot_id = NEW.slot_id
              AND c.faculty_id = v_faculty_id
            LIMIT 1);

        IF v_clash_course IS NOT NULL THEN
            SET v_msg = CONCAT('Timetable clash: faculty_id ', v_faculty_id,
                               ' already teaches ', v_clash_course,
                               ' in slot_id ', NEW.slot_id);
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_msg;
        END IF;
    END IF;
END$$
```

5.9.3 Declaring defaulters (database/views_triggers.sql)

```
CREATE PROCEDURE sp_refresh_defaulter_flags()
BEGIN
    UPDATE students s
        LEFT JOIN (SELECT DISTINCT student_id FROM v_defaulter_list) d
            ON d.student_id = s.student_id
    SET s.is_defaulter = (d.student_id IS NOT NULL)
    WHERE s.student_id > 0;
END$$
```

5.9.4 Role check decorator (campus_portal/core/roles.py)

```
def role_required(*roles):
    """
    View decorator: anonymous users go to the login page, logged-in users with
another role get
    a 403 page (templates/403.html), never a crash.

        @role_required(STUDENT)
        def dashboard(request): ...
    """
    def decorator(view):
        @wraps(view)
        def wrapper(request, *args, **kwargs):
            if not request.user.is_authenticated:
                return redirect_to_login(request.get_full_path())
            if get_role(request.user) not in roles:
                raise PermissionDenied("This page is for: " + ",
".join(ROLE_LABELS[r] for r in roles))
            return view(request, *args, **kwargs)
        return wrapper
    return decorator
```

5.9.5 Saving marks (campus_portal/core/services.py)

```
def save_marks(course, entries):
    """
    Saves internal/external marks. ``entries`` maps enrollment_id -> (internal,
external or None).
    Only the two raw columns are written: trg_marks_before_insert/update compute
total_marks,
    grade_points and letter_grade, so Python never does.
    """
    valid_ids = set(course.enrollments.values_list("enrollment_id", flat=True))
    saved = 0
    with db_write():
        for enrollment_id, (internal, external) in entries.items():
            if enrollment_id in valid_ids:
                Marks.objects.update_or_create(
                    enrollment_id=enrollment_id,
                    defaults={"internal_marks": internal, "external_marks":
external})
                saved += 1
    return saved
```

5.9.6 Running the engine (campus_portal/core/java_bridge.py)

```
def run_engine(command, *args, year=None):
    """Runs one allowed command and returns its parsed ``data``; raises EngineError on
failure."""
    argv = build_command(command, args, year)
    jar = Path(settings.JAVA_ENGINE_JAR)
    if not jar.is_file():
        raise EngineError("JarMissing", f"The engine JAR was not found at {jar}.",
hint=BUILD_HINT)

    started = time.monotonic()
    try:
        proc = subprocess.run(
            argv,
            capture_output=True,
            text=True,
            encoding="utf-8",
            errors="replace",
            timeout=settings.JAVA_ENGINE_TIMEOUT,
            cwd=settings.REPO_ROOT,
            shell=False,
            check=False,
        )
    except FileNotFoundError:
        raise EngineError("JavaMissing", f"Java was not found (tried
'{settings.JAVA_BIN}').",
                          hint="Install JDK 17+ or set DJANGO_JAVA_BIN in .env.") from None
    except subprocess.TimeoutExpired:
        raise EngineError("Timeout", f"The engine did not answer within
{settings.JAVA_ENGINE_TIMEOUT} s.",
                          hint="Check that MySQL is running.") from None
    elapsed_ms = int((time.monotonic() - started) * 1000)

    if proc.returncode == 0:
        document = _parse(proc.stdout)
        if not document or document.get("ok") is not True or "data" not in document:
            raise EngineError("BadOutput", "The engine exited successfully but did not print
a JSON result.",
                              exit_code=0)
        return EngineResult(command=document.get("command", command), data=document["data"],
elapsed_ms=elapsed_ms)

    document = _parse(proc.stderr)
    error = document.get("error") if isinstance(document, dict) else None
    if isinstance(error, dict) and "type" in error:
        raise EngineError(error["type"], error.get("message", ""),
exit_code=proc.returncode)
    raw = (proc.stderr or proc.stdout or "").strip()
    raise EngineError("Internal", raw[:500] or f"The engine failed with exit code
{proc.returncode}.",
                      exit_code=proc.returncode)
```

5.9.7 Concurrent dashboard (java_engine/.../services/ReportService.java)

```
    public Dashboard dashboard() throws DatabaseException {
        AtomicInteger threadNumber = new AtomicInteger(1);
        ExecutorService pool = Executors.newFixedThreadPool(3, runnable -> {
            Thread thread = new Thread(runnable, "report-worker-" +
threadNumber.getAndIncrement());
            thread.setDaemon(true);
            return thread;
        });
        long start = System.nanoTime();
        try {
            CompletableFuture<TaskResult<List<AttendanceSummary>>> attendance =
                    CompletableFuture.supplyAsync(() -> timed(reports::findAllAttendance),
pool);
            CompletableFuture<TaskResult<List<AttendanceSummary>>> defaulters =
                    CompletableFuture.supplyAsync(() -> timed(reports::findDefaulters),
pool);
            CompletableFuture<TaskResult<List<GpaReportRow>>> gpa =
                    CompletableFuture.supplyAsync(() -> timed(reports::findGpaReport),
pool);

            CompletableFuture.allOf(attendance, defaulters, gpa).join();
            long wallClock = (System.nanoTime() - start) / 1_000_000;
            return summarise(attendance.join(), defaulters.join(), gpa.join(), wallClock);
        } catch (CompletionException e) {
            if (e.getCause() instanceof DatabaseException db) {
                throw db;
            }
            throw e;
        } finally {
            pool.shutdown();
        }
    }
```

5.9.8 Transaction in JDBC (java_engine/.../dao/JdbcReportDao.java)

```
    public int refreshDefaulterFlags() throws DatabaseException {
        try (Connection conn = openConnection()) {
            conn.setAutoCommit(false);
            try (CallableStatement call = conn.prepareCall(REFRESH_DEFAULTERS);
                 PreparedStatement count = conn.prepareStatement(COUNT_DEFAULTERS)) {
                call.execute();
                int flagged;
                try (ResultSet rs = count.executeQuery()) {
                    rs.next();
                    flagged = rs.getInt(1);
                }
                conn.commit();
                return flagged;
            } catch (SQLException e) {
                conn.rollback();
                throw e;
            }
        } catch (SQLException e) {
            throw new DatabaseException("Could not refresh defaulter flags", e);
        }
    }
```

5.10 Implementation screenshots

The screenshots were captured with Playwright (Chromium, 1280 px wide) from the running development server, logged in as each demo role. No password or `.env` content is visible.

Smart Campus | My dashboard | Sneha Reddy | Student | Password | Log out

Sneha Reddy

25CSE004 · CSE · Semester 3, section A

Attendance warning: below 75% in 6 courses.
CS301 (66.67%), CS351 (66.67%), CS352 (58.33%), CS353 (58.33%), CS354 (58.33%), MA301 (66.67%). Students below 75% may be detained from the end-semester exam. You are currently on the declared defaulter list.

6.85	7.14	14/14	Below 75%
Official CGPA (semesters 1-2)	GPA this semester (credit-weighted)	Credits earned / graded	Attendance status

Courses

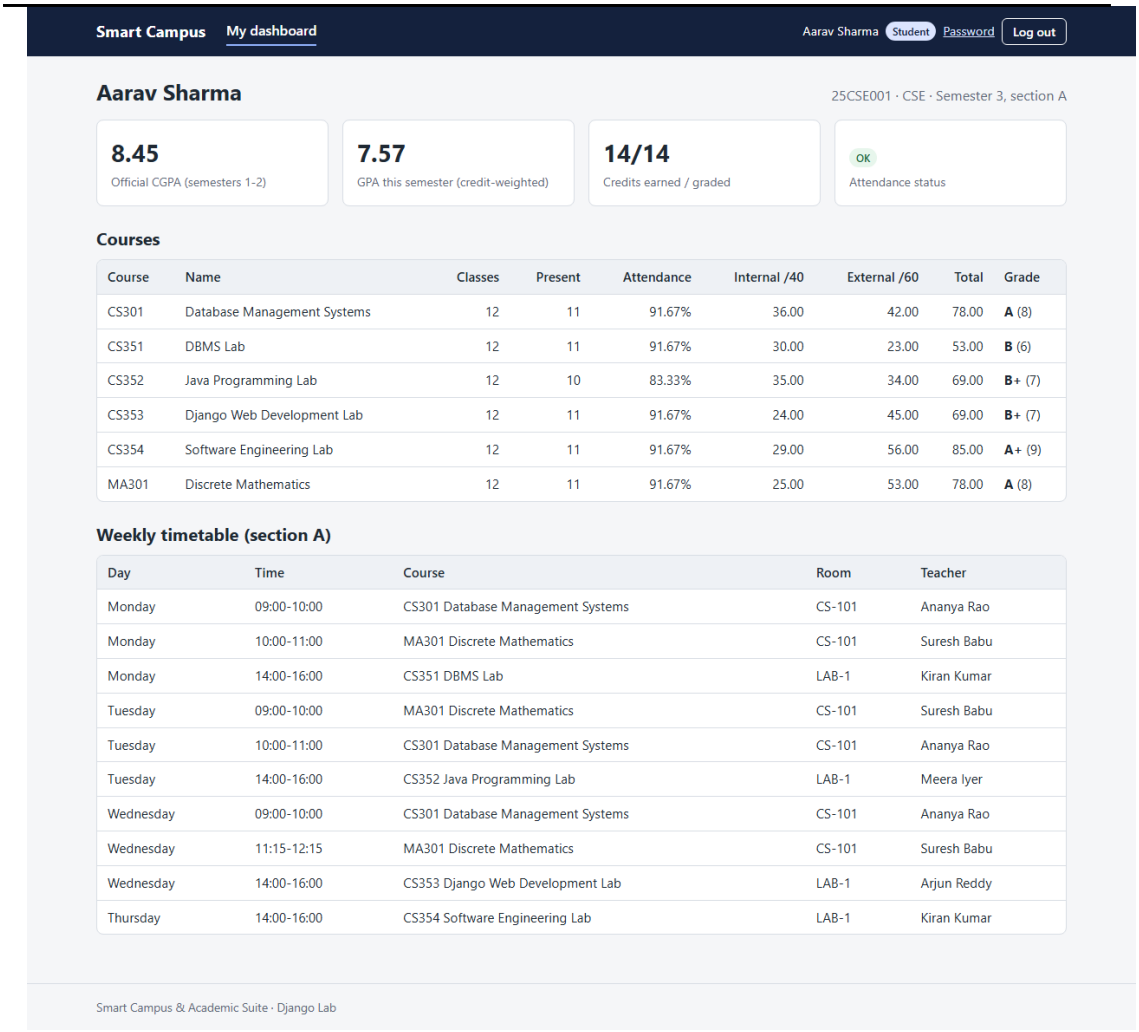
Course	Name	Classes	Present	Attendance	Internal /40	External /60	Total	Grade
CS301	Database Management Systems	12	8	66.67%	18.00	24.50	42.50	P (5)
CS351	DBMS Lab	12	8	66.67%	35.00	21.00	56.00	B (6)
CS352	Java Programming Lab	12	7	58.33%	24.00	32.00	56.00	B (6)
CS353	Django Web Development Lab	12	7	58.33%	29.00	43.00	72.00	A (8)
CS354	Software Engineering Lab	12	7	58.33%	34.00	54.00	88.00	A+ (9)
MA301	Discrete Mathematics	12	8	66.67%	30.00	51.00	81.00	A+ (9)

Weekly timetable (section A)

Day	Time	Course	Room	Teacher
Monday	09:00-10:00	CS301 Database Management Systems	CS-101	Ananya Rao
Monday	10:00-11:00	MA301 Discrete Mathematics	CS-101	Suresh Babu
Monday	14:00-16:00	CS351 DBMS Lab	LAB-1	Kiran Kumar
Tuesday	09:00-10:00	MA301 Discrete Mathematics	CS-101	Suresh Babu
Tuesday	10:00-11:00	CS301 Database Management Systems	CS-101	Ananya Rao
Tuesday	14:00-16:00	CS352 Java Programming Lab	LAB-1	Meera Iyer
Wednesday	09:00-10:00	CS301 Database Management Systems	CS-101	Ananya Rao
Wednesday	11:15-12:15	MA301 Discrete Mathematics	CS-101	Suresh Babu
Wednesday	14:00-16:00	CS353 Django Web Development Lab	LAB-1	Arjun Reddy
Thursday	14:00-16:00	CS354 Software Engineering Lab	LAB-1	Kiran Kumar

Smart Campus & Academic Suite · Django Lab

Figure 5.1: Student dashboard of a defaulter (25cse004) with the 75% warning



Smart Campus | My dashboard | Aarav Sharma | Student | Password | Log out

Aarav Sharma

25CSE001 · CSE · Semester 3, section A

8.45	7.57	14/14	OK
Official CGPA (semesters 1-2)	GPA this semester (credit-weighted)	Credits earned / graded	Attendance status

Courses

Course	Name	Classes	Present	Attendance	Internal /40	External /60	Total	Grade
CS301	Database Management Systems	12	11	91.67%	36.00	42.00	78.00	**A** (8)
CS351	DBMS Lab	12	11	91.67%	30.00	23.00	53.00	**B** (6)
CS352	Java Programming Lab	12	10	83.33%	35.00	34.00	69.00	**B+** (7)
CS353	Django Web Development Lab	12	11	91.67%	24.00	45.00	69.00	**B+** (7)
CS354	Software Engineering Lab	12	11	91.67%	29.00	56.00	85.00	**A+** (9)
MA301	Discrete Mathematics	12	11	91.67%	25.00	53.00	78.00	**A** (8)

Weekly timetable (section A)

Day	Time	Course	Room	Teacher
Monday	09:00-10:00	CS301 Database Management Systems	CS-101	Ananya Rao
Monday	10:00-11:00	MA301 Discrete Mathematics	CS-101	Suresh Babu
Monday	14:00-16:00	CS351 DBMS Lab	LAB-1	Kiran Kumar
Tuesday	09:00-10:00	MA301 Discrete Mathematics	CS-101	Suresh Babu
Tuesday	10:00-11:00	CS301 Database Management Systems	CS-101	Ananya Rao
Tuesday	14:00-16:00	CS352 Java Programming Lab	LAB-1	Meera Iyer
Wednesday	09:00-10:00	CS301 Database Management Systems	CS-101	Ananya Rao
Wednesday	11:15-12:15	MA301 Discrete Mathematics	CS-101	Suresh Babu
Wednesday	14:00-16:00	CS353 Django Web Development Lab	LAB-1	Arjun Reddy
Thursday	14:00-16:00	CS354 Software Engineering Lab	LAB-1	Kiran Kumar

Smart Campus & Academic Suite · Django Lab

Figure 5.2: Student dashboard of a student in good standing (25cse001)

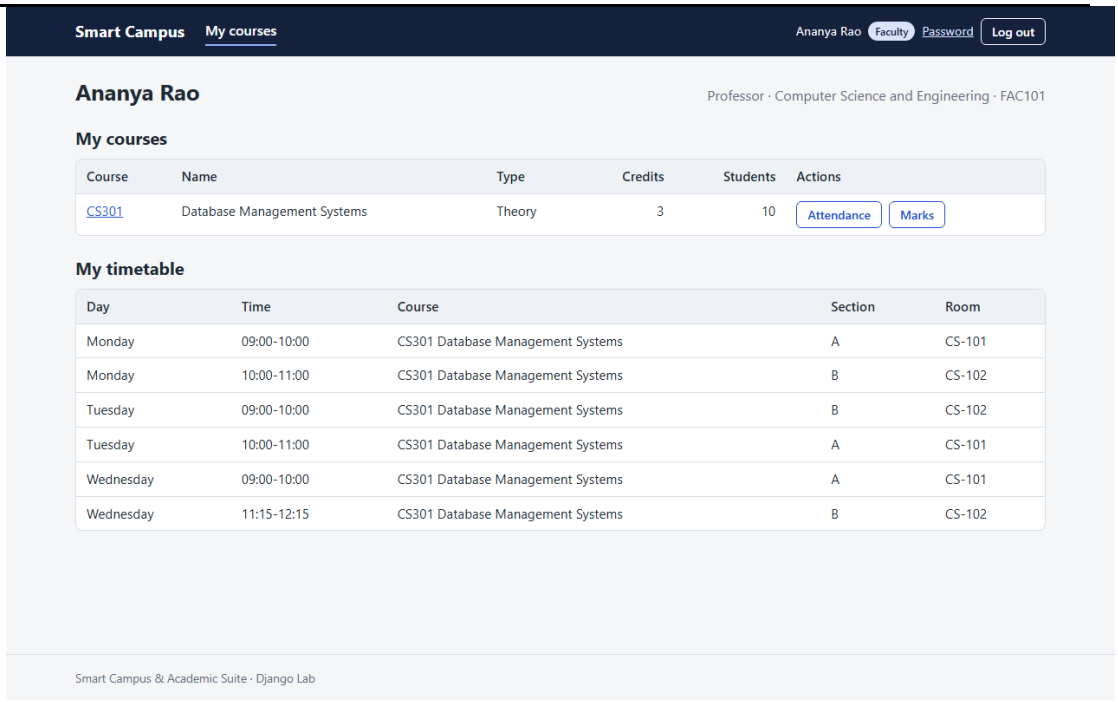


Figure 5.3: Faculty dashboard: own courses and weekly timetable (ananya.rao)

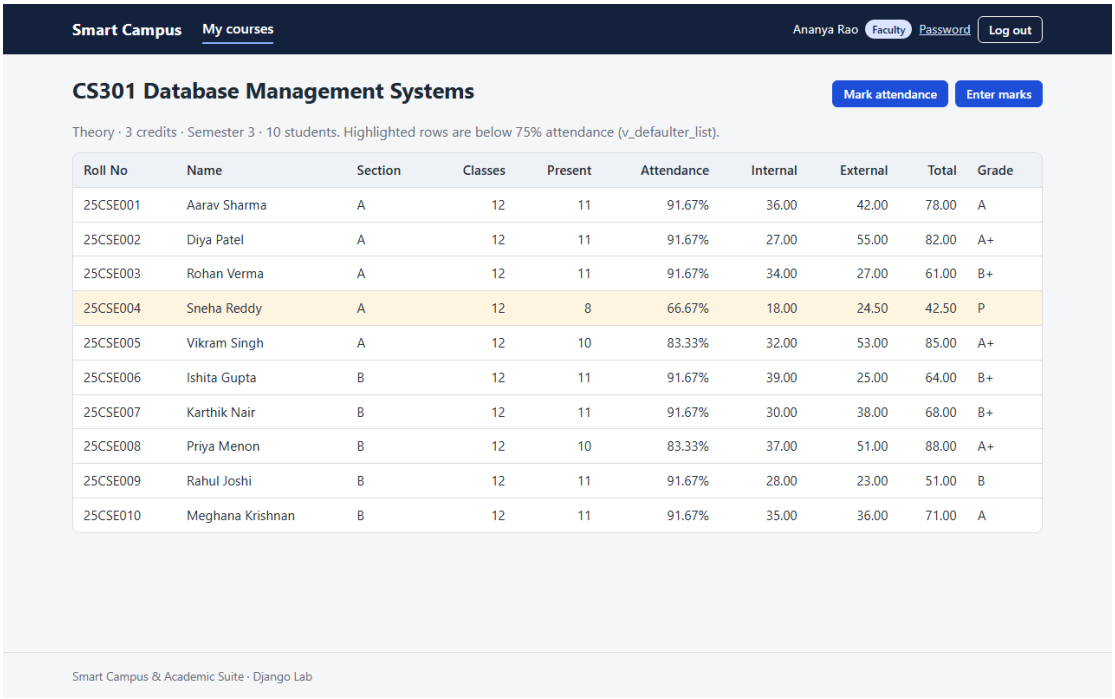


Figure 5.4: Faculty course roster for CS301

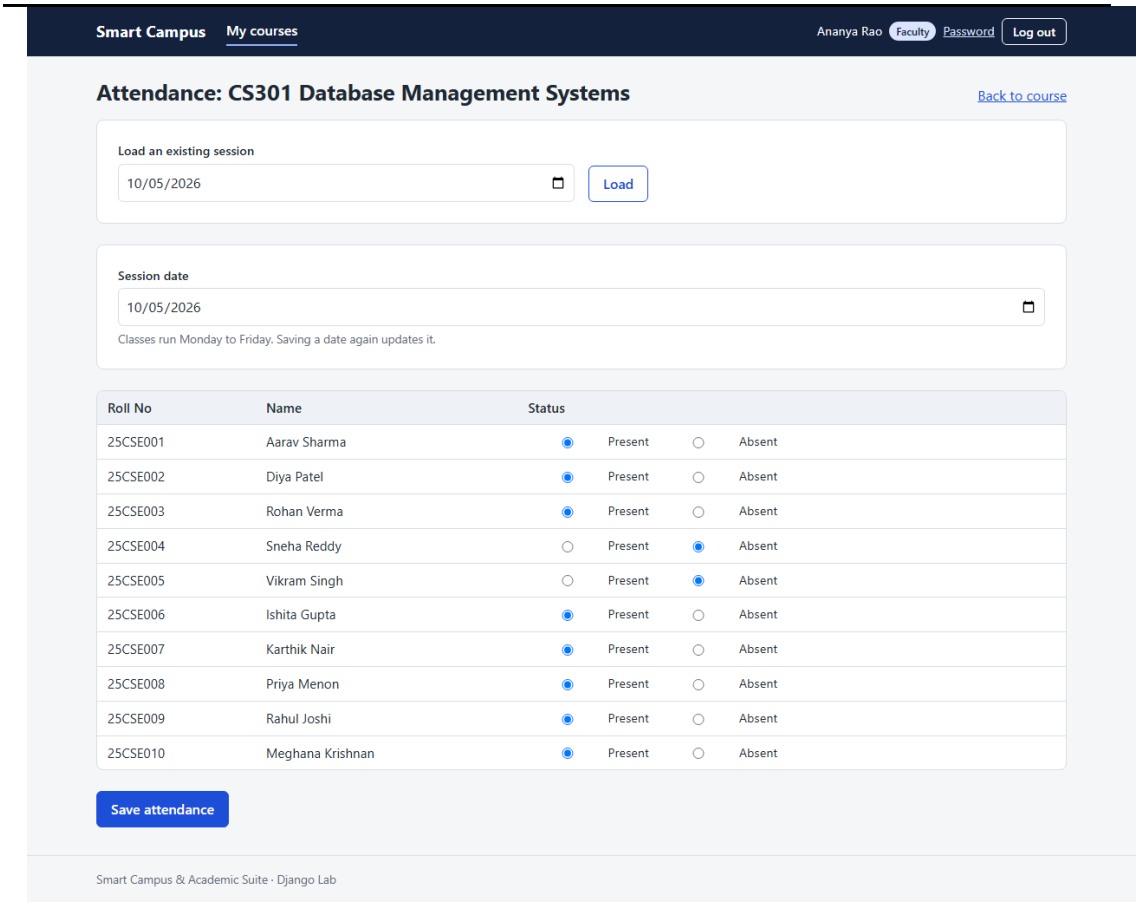


Figure 5.5: Faculty attendance sheet for a date

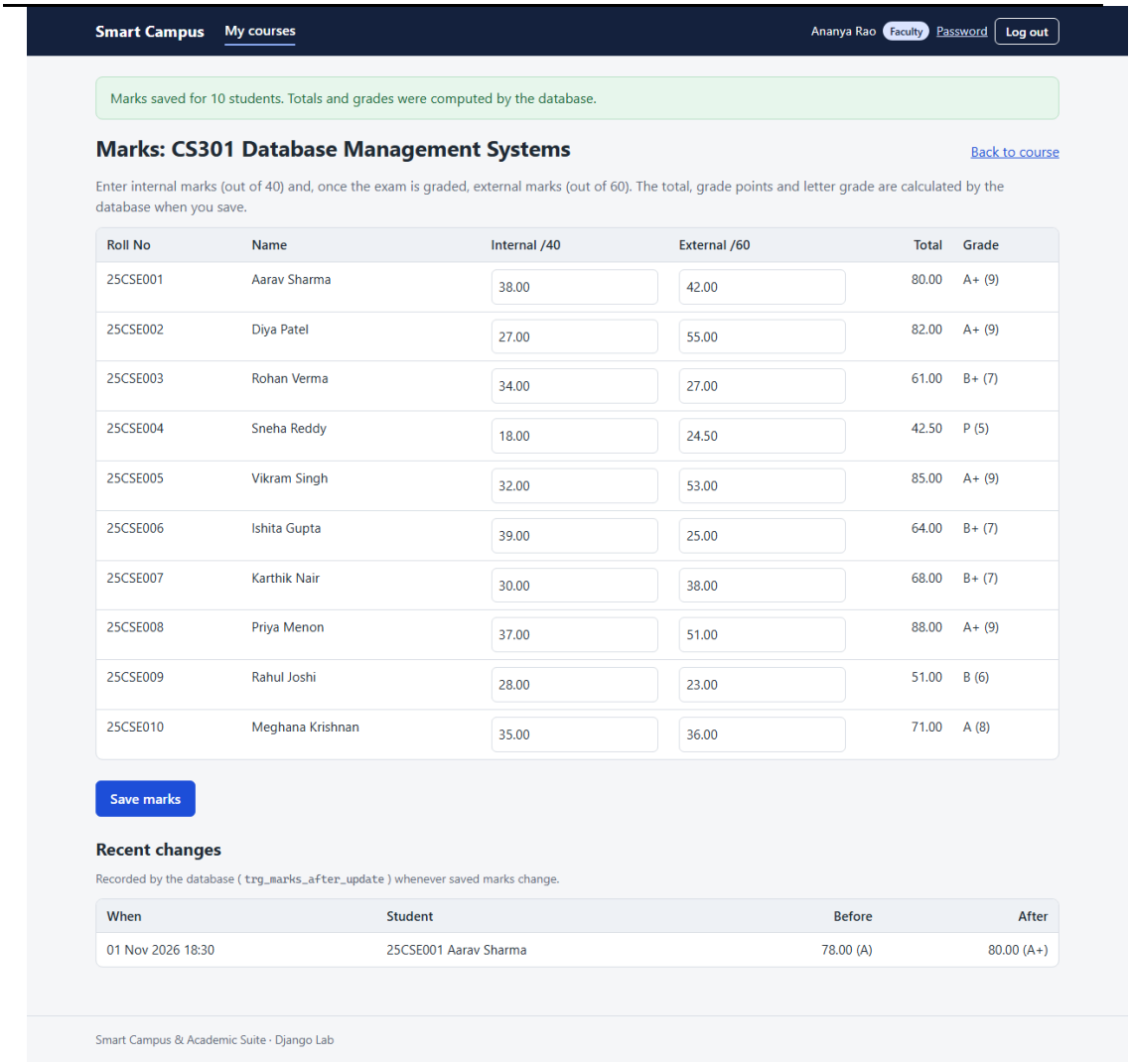


Figure 5.6: Faculty marks sheet; totals and grades come from the database

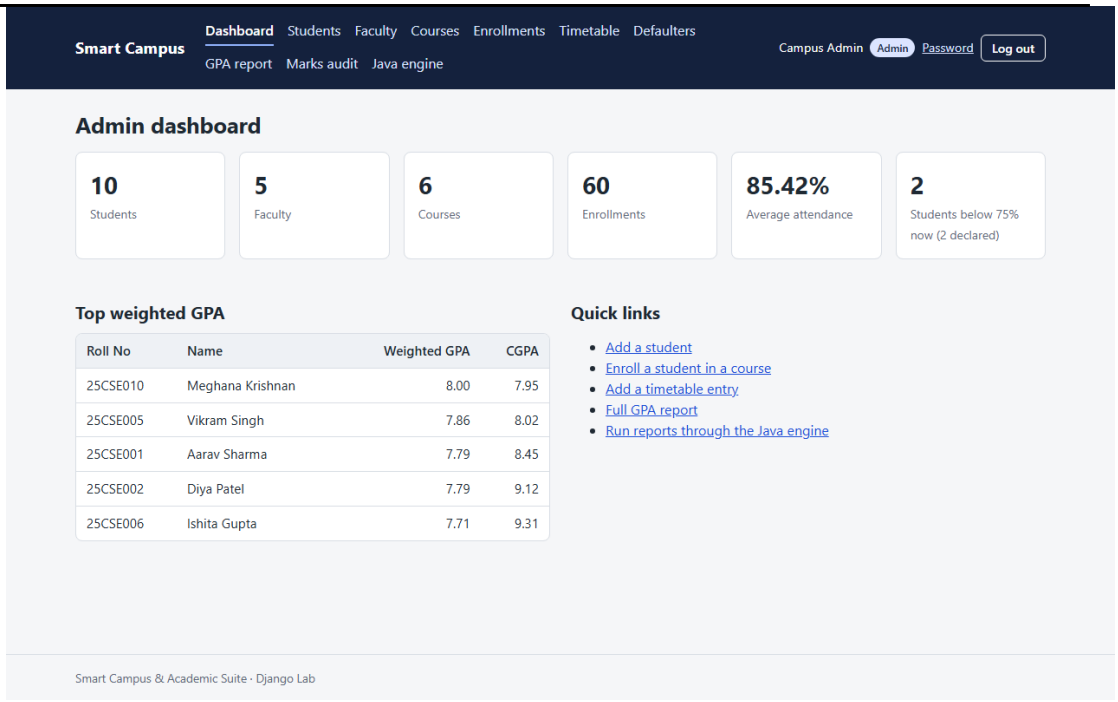


Figure 5.7: Admin dashboard

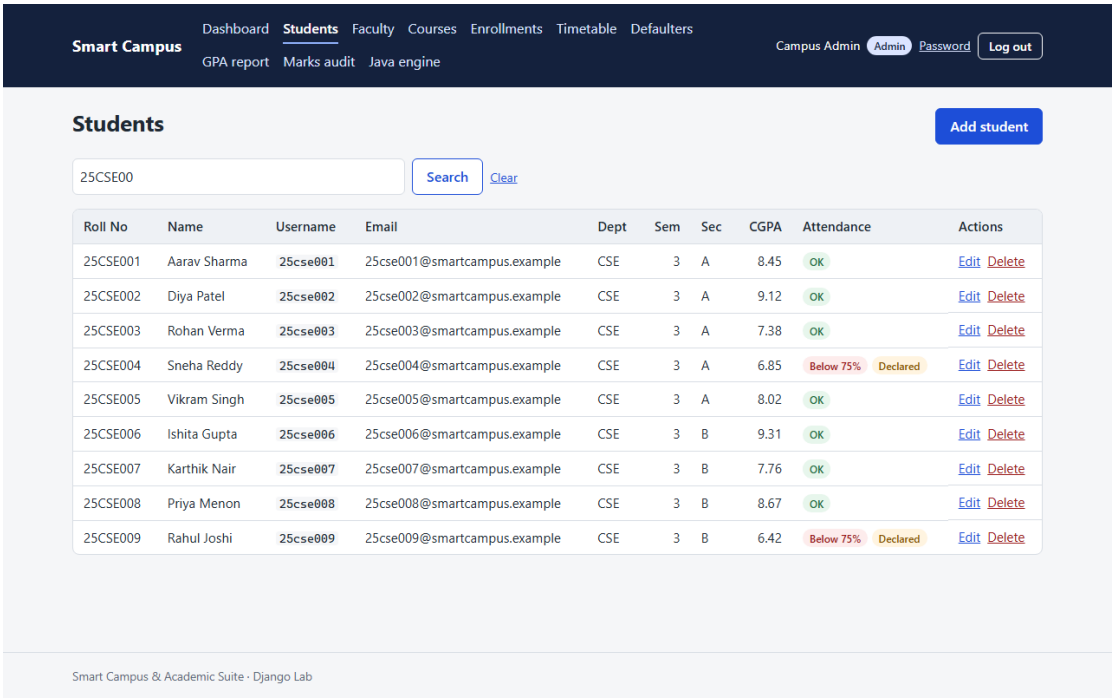


Figure 5.8: Admin student list with search and pagination

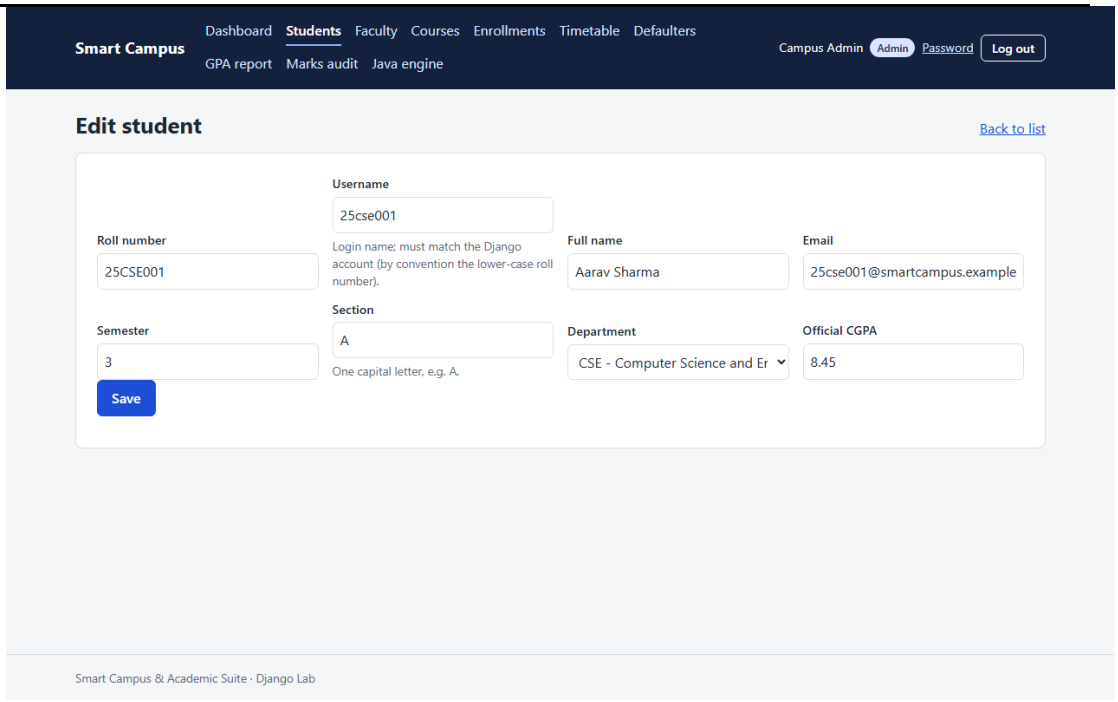


Figure 5.9: Admin student form

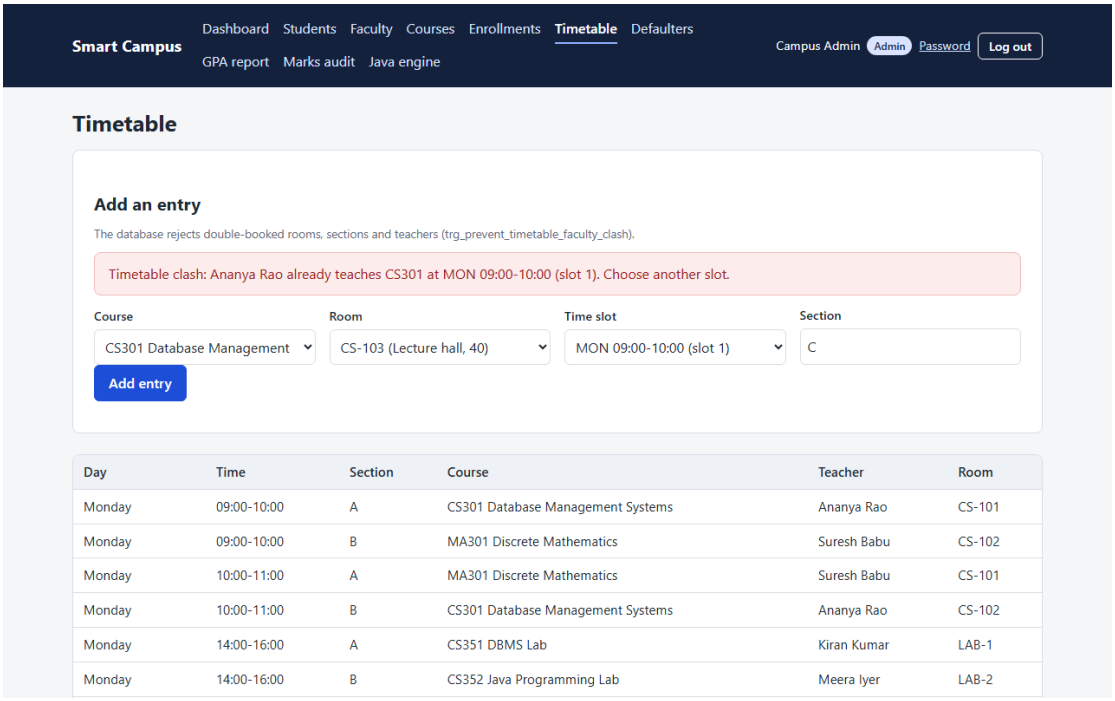


Figure 5.10: Timetable clash refused by the trigger, shown as a readable message

5.11 Integration of modules

The modules are integrated through the shared database and one process boundary:

- **Portal and database:** unmanaged Django models map 10 tables and the 4 views; all writes go through `core/services.py`, which wraps them in

`transaction.atomic()` and converts database rejections into friendly messages.

- **Engine and database:** DAOs use `PreparedStatement` on the same tables and views; the configuration is read from the same `.env`.
- **Portal and engine:** `core/java_bridge.py` starts the JAR with an argument list and parses one line of JSON; the child process inherits the `DB_*` variables, so no extra configuration is needed.
- **Rules in one place:** grades, the 75% rule, clash checks and audit live in MySQL, so the two applications cannot disagree. Integration cases IT01-IT10 (Section 6.5) verify each path.

5.12 Challenges faced during development

Table 5.6: Development challenges and solutions

Challenge	How it was solved
`REGEXP` CHECK constraints accepted lower-case sections because the column collation is case-insensitive	Rewrote them as `REGEXP_LIKE(col, pattern, 'c')` and added a regression test (commit c96e1f6)
Java 17 on Windows wrote Cp1252 to a pipe, corrupting JSON text	The bridge passes `-Dfile.encoding=UTF-8`
Building mysqlclient needs a C compiler on Windows	Used PyMySQL installed as MySQLdb
Connector/J refused to connect: "Public Key Retrieval is not allowed"	Added `allowPublicKeyRetrieval=true` to the local JDBC URL
PowerShell 5.1 adds a UTF-8 BOM to piped stdin	The interactive menu strips the BOM
AUTO_INCREMENT leaves gaps, so ids differ from expectations	Tests look rows up by natural key
Django 6.1 needs MySQL 8.4	Stayed on Django 5.2 LTS
JVM start-up plus a new connection costs about 0.7 s per call	At most one engine call per page

5.13 Summary

The system was implemented in about 7325 lines of SQL, Java, Python, HTML and CSS. The database holds the rules, the Java engine and the Django portal share it, and a guarded bridge connects them. The next chapter shows how this implementation was tested.

CHAPTER 6: SOFTWARE TESTING

6.1 Introduction

Testing was done at four levels (unit, integration, system and acceptance) and with both white-box and black-box techniques. Two automated suites run on every build: 29 JUnit tests for the Java engine and 71 Django tests for the portal. In addition, 78 documented test cases were executed by a script (`docs/src/evidence/collect_cases.py`) against the real MySQL database, the real JAR and the Django test client. **Every "Actual result" and "Status" in this chapter was produced by actually running the tests on 2026-11-01**; nothing was typed in by hand. Test writes were made inside transactions that were rolled back, and the database was rebuilt from the seed afterwards.

6.2 Testing objectives

- Verify every functional requirement FR1-FR20 at least once.
- Show that the database rejects invalid data even when the application is bypassed.
- Show that every page enforces its role and that users see only their own records.
- Exercise every branch of the grading function and the main decision points in the code.
- Measure response times of pages and engine commands.
- Find and fix defects before the final submission.

6.3 Testing methodology

Table 6.1: Test levels and tools

Level / technique	Tool	What was run
Unit (Java)	JUnit 6 via Maven Wrapper; JaCoCo	29 tests, no database
Unit + integration (Django)	Django test runner; coverage.py	71 tests on a throwaway test_smart_campus_db built from the real SQL scripts
Documented cases (UT, IT, WB, BB)	collect_cases.py	78 cases on the real database, JAR and Django client
System and performance	Playwright + dev server; CLI timing	17 live pages, 6 CLI commands
Acceptance	Manual walk-through per role	Section 6.10

6.4 Unit testing

Unit tests check one function or class in isolation. The documented unit cases are listed first, followed by the automated suites grouped by test class.

Table 6.2: Unit test cases

ID	Module	Objective	Input	Expected result	Actual result	Status
UT01	core.services	Academic year rolls over in July	current_academic_year(2026-10-31) and (2026-06-30)	2026-27 and 2025-26	2026-27 and 2025-26	Pass
UT02	core.db_errors	Duplicate-enrollment error becomes a sentence	MySQL error 1062 on uq_enrollments_student_course	This student is already enrolled in that course.	This student is already enrolled in that course.	Pass
UT03	core.java_bridge	Allowed command builds a safe argument list	build_command('student', ['25CSE001'])	List ending student, 25CSE001, --json with -Dfile.encoding	['student', '25CSE001', '--json'] (list)	Pass
UT04	core.java_bridge	Command outside the allow-list is refused	build_command('rm')	EngineError NotAllowed	EngineError NotAllowed: 'rm' is not an allowed engine command.	Pass
UT05	core.roles	Role is derived from the login	admin, ananya.rao, 25cse001	admin, faculty, student	admin, faculty, student	Pass
UT06	fn_grade_points / fn_letter_grade	Grading function maps 85 marks	SELECT fn_grade_points(85), fn_letter_grade(...)	9, A+	9, A+	Pass

Table 6.3: JUnit results (mvnw clean package, build SUCCESS)

JUnit test class	Tests	Passed	Failed	Time (s)
cli.CliTest	7	7	0	0.172
cli.InteractiveMenuTest	6	6	0	0.090
cli.JsonWriterTest	9	9	0	0.042
config.DatabaseConfigTest	4	4	0	0.271
services.ReportExporterTest	3	3	0	0.067

Table 6.4: Django test results (71 tests, result: OK)

Django test module	Tests	Passed	Failed
core.tests.test_access	11	11	0
core.tests.test_java_bridge	23	23	0
core.tests.test_polish	18	18	0
core.tests.test_portals	19	19	0

6.5 Integration testing

Integration tests follow data across module boundaries: portal to database to trigger to

another portal, and portal to bridge to JAR to view.

Table 6.5: Integration test cases

ID	Module	Objective	Input	Expected result	Actual result	Status
IT01	accounts + core.roles	Login lands each role on its own portal	POST /accounts/login/ for admin, ananya.rao, 25cse001	/admin-portal/, /faculty/, /student/	admin -> /admin-portal/; ananya.rao -> /faculty/; 25cse001 -> /student/	Pass
IT02	faculty_portal -> trg_marks_before_update -> student_portal	Marks entered by faculty are graded by the trigger and shown to the student	Faculty saves 38 + 57 for 25CSE001 in CS301	95.00, 10, O stored and visible on /student/	DB: 95.00, 10, O; student page shows grade: True	Pass
IT03	faculty_portal -> attendance_records -> v_attendance_summary	A saved session updates the attendance view	Mark all CS301 students PRESENT on Mon 2026-10-12	total_classes increases by 1	total_classes 12 -> 13; attendance 92.31%	Pass
IT04	admin_portal -> core.services -> enrollments	Admin enroll form creates an enrollment	New semester-3 student 25CSE061 + CS301, 2026-27	Success message and new enrollments row	message shown: True; row exists: True	Pass
IT05	admin_portal -> trg_prevent_timetable_faculty_clash -> core.db_errors	A teacher double-booking is refused with a readable message	CS301 (Ananya Rao) in MON 09:00 slot for section C	Clash message naming the teacher; no new row	Timetable clash: Ananya Rao already teaches CS301 at MON 09:00-10:00 (slot 1). Choose another slot.; entries 20 -> 20	Pass
IT06	faculty_portal -> trg_marks_after_update -> v_marks_audit -> admin_portal	A marks change is audited and visible to the admin	Change 25CSE002 CS301 marks to 40 + 60	One audit row with old and new grade, listed on /admin-portal/marks-audit/	1 audit row: 82.00 (A+) -> 100.00 (O); on admin page: True	Pass
IT07	admin_portal -> sp_refresh_defaulter_flags()	Declaring defaulters re-computes every flag	Stale flag on 25CSE001, then 'Declare defaulter list now'	Only 25CSE004 and 25CSE009 flagged	flagged after CALL: 25CSE004, 25CSE009; message: True	Pass
IT08	core.java_bridge -> Java JAR -> v_defaulter_list	Django runs the real JAR and parses its JSON	run_engine('defaulters')	25CSE004 and 25CSE009 in the parsed data	2 students (25CSE004, 25CSE009), 8 course rows, 844 ms	Pass
IT09	Java CLI -> JDBC -> v_student_cgpa_report	Engine reads the GPA view and sorts it	java -jar smart-campus-engine.jar gpa-report --json	Exit 0, 10 rows, 25CSE010 first	exit 0, 10 rows, first 25CSE010 (8.0), 741 ms	Pass
IT10	Java CLI -> enrollments UNIQUE constraint	Duplicate enrollment is reported by the database through the engine	enroll 25CSE001 CS301 --json	Exit 1, error type DuplicateEnrollment	exit 1, type DuplicateEnrollment: 25CSE001 is already enrolled in CS301	Pass

6.6 System testing

The complete system was started (MySQL, the Django development server and the built JAR) and every main page was requested as the role that owns it, using Playwright. All returned HTTP 200 with real data; the screenshots in Sections 5.10 and 9.3 were taken in the same run.

Table 6.6: System test: live pages

Logged in as	Page	HTTP status
25cse004	/student/	200
ananya.rao	/faculty/	200
ananya.rao	/faculty/courses/CS301/	200
ananya.rao	/faculty/courses/CS301/attendance/	200
ananya.rao	/faculty/courses/CS301/marks/	200
admin	/admin-portal/	200
admin	/admin-portal/students/	200
admin	/admin-portal/faculty/	200
admin	/admin-portal/courses/	200
admin	/admin-portal/enrollments/	200
admin	/admin-portal/timetable/	200
admin	/admin-portal/defaulters/	200
admin	/admin-portal/gpa-report/	200
admin	/admin-portal/marks-audit/	200
admin	/engine/	200
admin	/engine/?command=dashboard	200
admin	/engine/?command=defaulters	200

6.7 Functional testing

Each functional requirement is traced to the tests that exercise it:

Table 6.7: Requirement traceability

Requirement	Test cases	Result
FR1, FR2 (login, role)	IT01, UT05, WB09-WB11	Pass
FR3 (password, logout)	Django tests in test_access / test_polish	Pass
FR4-FR7 (student views, warning)	IT02, Django test_portals, live page /student/	Pass
FR8-FR9 (faculty courses, attendance)	IT03, BB13-BB15, BB35	Pass
FR10 (marks, grading)	IT02, BB01-BB12, BB23-BB27, WB01-WB08, WB15-WB18	Pass
FR11 (CRUD)	BB16-BB18, BB31, Django test_polish	Pass
FR12 (enroll)	IT04, WB12-WB14, BB19-BB22, IT10	Pass
FR13 (timetable clash)	IT05	Pass
FR14 (defaulters)	IT07, BB28	Pass
FR15, FR17 (GPA report, audit)	IT06, IT09, WB19-WB21	Pass
FR16 (engine page)	IT08, BB30, UT03, UT04	Pass

Requirement	Test cases	Result
FR18-FR20 (engine commands, CSV, dashboard)	JUnit CliTest, ReportExporterTest; WB22-WB27	Pass

6.8 Performance testing

Each page was requested five times per role on the development server (Section 5.2 hardware) and the median, minimum and maximum response times were recorded. Each engine command was run five times from the command line.

Table 6.8: Page response times (5 requests each)

Page	User	Median (ms)	Min (ms)	Max (ms)
/student/	25cse004	69.2	66.3	86.6
/faculty/	ananya.rao	59.0	56.0	87.0
/faculty/courses/CS301/	ananya.rao	67.3	62.8	86.1
/faculty/courses/CS301/attendance/	ananya.rao	62.8	58.7	88.8
/faculty/courses/CS301/marks/	ananya.rao	65.9	63.6	94.4
/admin-portal/	admin	68.5	64.9	68.7
/admin-portal/students/	admin	59.5	59.1	60.1
/admin-portal/faculty/	admin	53.0	49.7	77.0
/admin-portal/courses/	admin	54.8	52.3	60.0
/admin-portal/enrollments/	admin	63.8	58.8	76.5
/admin-portal/timetable/	admin	66.4	64.3	71.6
/admin-portal/defaulters/	admin	62.7	54.7	83.1
/admin-portal/gpa-report/	admin	52.6	48.2	60.8
/admin-portal/marks-audit/	admin	50.3	48.1	52.8
/engine/	admin	47.4	46.6	70.3
/engine/?command=dashboard	admin	750.4	737.3	785.2
/engine/?command=defaulters	admin	668.7	632.5	708.5

Table 6.9: Java engine command times (5 runs each, including JVM start-up)

Command	Median (ms)	Min (ms)	Max (ms)
java -jar ... help	168	154	201
java -jar ... courses	697	608	816
java -jar ... student 25CSE001	813	701	1139
java -jar ... defaulters	636	589	858
java -jar ... gpa-report	707	665	816
java -jar ... dashboard	773	703	873

All ordinary pages answered in under 70 ms (median). Pages that call the engine took about 0.67-0.75 s, almost all of it JVM start-up and a new MySQL connection, which is why the design allows at most one engine call per page (NFR2). NFR1 (under 1 s) is met.

6.9 Security testing

Security tests tried the attacks a student or outsider is most likely to attempt (error guessing). In addition, `core/tests/test_polish.py` walks the whole URL configuration and checks that every route redirects anonymous users to the login page and returns 403 to every role outside its own prefix.

Table 6.10: Security test cases (error guessing)

ID	Module	Objective	Input	Expected result	Actual result	Status
BB29	Security / robustness	Guess: sqli	Search q = 25CSE001' OR '1'='1	HTTP 200, 0 rows (treated as text)	HTTP 200, 0 rows	Pass
BB30	Security / robustness	Guess: bridge	Engine roll number '25CSE001; dir'	Form error, engine not called	HTTP 200, form error shown: True	Pass
BB31	Security / robustness	Guess: username	New student with faculty username ananya.rao	Trigger message shown	message shown: True	Pass
BB32	Security / robustness	Guess: xss	Student name <script>alert(1)< /script>	Name rendered escaped	rendered escaped: True	Pass
BB33	Security / robustness	Guess: csrf	POST /admin-portal/defaulters/ without CSRF token	HTTP 403	HTTP 403	Pass
BB34	Security / robustness	Guess: role	Student opens /admin-portal/students/	HTTP 403	HTTP 403	Pass
BB35	Security / robustness	Guess: other_teacher	kiran.kumar opens CS301 marks (not his course)	HTTP 403	HTTP 403	Pass

6.10 User Acceptance Testing (UAT)

UAT was carried out as a team walk-through: each scenario below was performed on the running system with the demo accounts, following the use cases of Section 3.9. No external users took part, so this is not a formal UAT with real staff and students; that remains future work (Section 8.7).

Table 6.11: User acceptance walk-through

ID	Role	Scenario	Acceptance criterion	Evidence	Result
UAT1	Student	Open the dashboard as 25cse004	Warning for courses below 75%	Figure 5.1	Accepted
UAT2	Student	Open the dashboard as 25cse001	No warning; grades and CGPA shown	Figure 5.2	Accepted
UAT3	Faculty	Record CS301 attendance on a weekday	Saved; percentage changes	IT03	Accepted
UAT4	Faculty	Enter CS301 marks	Total and grade computed	IT02, Figure 5.6	Accepted

ID	Role	Scenario	Acceptance criterion	Evidence	Result
UAT5	Faculty	Open another teacher's course	403 page	BB35	Accepted
UAT6	Admin	Double-book a teacher in the timetable	Readable clash message	IT05, Figure 5.10	Accepted
UAT7	Admin	Declare defaulters	25CSE004 and 25CSE009 flagged	IT07	Accepted
UAT8	Admin	Run the engine dashboard	Counts, averages and thread timings	Figure 9.6	Accepted

6.11 Test cases and test results

This section groups the documented cases by technique. White-box cases were designed from the code and SQL; black-box cases from the requirements only.

6.11.1 White-box testing: statement and branch coverage

`fn_grade_points` has eight branches (NULL, seven grade bands). One input per branch executes every statement and every branch.

Table 6.12: Statement and branch coverage of fn_grade_points

ID	Module	Objective	Input	Expected result	Actual result	Status
WB01	fn_grade_points (CASE)	Execute every CASE branch once	total = 95	10, O	10, O	Pass
WB02	fn_grade_points (CASE)	Execute every CASE branch once	total = 85	9, A+	9, A+	Pass
WB03	fn_grade_points (CASE)	Execute every CASE branch once	total = 75	8, A	8, A	Pass
WB04	fn_grade_points (CASE)	Execute every CASE branch once	total = 65	7, B+	7, B+	Pass
WB05	fn_grade_points (CASE)	Execute every CASE branch once	total = 55	6, B	6, B	Pass
WB06	fn_grade_points (CASE)	Execute every CASE branch once	total = 45	5, P	5, P	Pass
WB07	fn_grade_points (CASE)	Execute every CASE branch once	total = 30	0, F	0, F	Pass
WB08	fn_grade_points (CASE)	Execute every CASE branch once	total = None	None, None	None, None	Pass

Table 6.13: Branch coverage of role_required

ID	Module	Objective	Input	Expected result	Actual result	Status
WB09	core.roles.role_required	Anonymous branch	anonymous GET /student/	HTTP 302	HTTP 302	Pass
WB10	core.roles.role_required	Wrong-role branch	ananya.rao GET /student/	HTTP 403	HTTP 403	Pass

ID	Module	Objective	Input	Expected result	Actual result	Status
WB11	core.roles.role_required	Allowed branch	25cse001 GET /student/	HTTP 200	HTTP 200	Pass

6.11.2 White-box testing: path coverage

`core.services.enroll_student` has three independent paths: semester mismatch, database rejection (duplicate) and success.

Table 6.14: Path coverage of enroll_student

ID	Module	Objective	Input	Expected result	Actual result	Status
WB12	core.services.enroll_student	Execute path: mismatch	Semester-3 student, semester-5 course	ServiceError (semester rule)	ServiceError: CS501 is a semester 5 course, but 25CSE001 is in semester 3.	Pass
WB13	core.services.enroll_student	Execute path: duplicate	25CSE001 + CS301 (already enrolled)	ServiceError (duplicate, from MySQL 1062)	ServiceError: This student is already enrolled in that course.	Pass
WB14	core.services.enroll_student	Execute path: ok	New semester-3 student + CS301	Enrollment created	created enrollment for 25CSE062 in CS301	Pass

6.11.3 White-box testing: condition coverage

Each atomic condition of the compound decisions was made true and false at least once.

Table 6.15: Condition coverage of MarksSheetForm.clean and trg_marks_after_update

ID	Module	Objective	Input	Expected result	Actual result	Status
WB15	MarksSheetForm.clean	internal is None AND external is not None	internal='', external=''	valid	valid	Pass
WB16	MarksSheetForm.clean	internal is None AND external is not None	internal='', external='50'	error: Enter internal marks before external marks.	error: Enter internal marks before external marks.	Pass
WB17	MarksSheetForm.clean	internal is None AND external is not None	internal='30', external=''	valid	valid	Pass
WB18	MarksSheetForm.clean	internal is None AND external is not None	internal='30', external='50'	valid	valid	Pass
WB19	trg_marks_after_update	NOT(old <=> new internal AND old <=> new external)	internal same, external same	0 audit row(s)	0 audit row(s)	Pass
WB20	trg_marks_after_update	NOT(old <=> new internal AND old <=> new external)	internal 20, external same	1 audit row(s)	1 audit row(s)	Pass

ID	Module	Objective	Input	Expected result	Actual result	Status
WB21	trg_marks_after_update	NOT(old <=> new internal AND old <=> new external)	internal same, external NULL	1 audit row(s)	1 audit row(s)	Pass

6.11.4 White-box testing: loop testing

The argument-parsing loop was run with zero, one and many arguments, and the `.env` search loop with zero, one and two parent-directory steps.

Table 6.16: Loop testing

ID	Module	Objective	Input	Expected result	Actual result	Status
WB22	cli.CommandLine.parse (argument loop)	Loop over 0 arguments (shows help)	java -jar ...	exit 0	exit 0: Smart Campus Java engine	Pass
WB23	cli.CommandLine.parse (argument loop)	Loop over 1 argument	java -jar ... courses	exit 0	exit 0: Code Name Type Credits Sem Faculty	Pass
WB24	cli.CommandLine.parse (argument loop)	Loop over 6 arguments incl. options	java -jar ... enroll 25CSE001 CS301 --year 2026-27 --json	exit 1	exit 1: {"ok":false,"command":"enroll","error":{"type":"DuplicateEnrollment","	Pass
WB25	config.DatabaseConfig.findEnvFile (walk-up loop)	Find .env after 0 parent steps (repo root)	java -jar ... courses (cwd=3rdSemLabProject)	exit 0 (connected)	exit 0	Pass
WB26	config.DatabaseConfig.findEnvFile (walk-up loop)	Find .env after 1 parent step	java -jar ... courses (cwd=3rdSemLabProject\java_engine)	exit 0 (connected)	exit 0	Pass
WB27	config.DatabaseConfig.findEnvFile (walk-up loop)	Find .env after 2 parent steps	java -jar ... courses (cwd=3rdSemLabProject\java_engine\target)	exit 0 (connected)	exit 0	Pass

6.11.5 Code coverage

Table 6.17: JaCoCo coverage of the Java engine by the JUnit suite

Java package	Lines	Line coverage	Branch coverage
(root)	2	0.0%	n/a
cli	599	41.1%	66.7%
config	57	61.4%	70.0%
dao	170	0.0%	0.0%
exceptions	29	27.6%	0.0%
models	167	0.0%	0.0%
services	161	20.5%	38.7%
Total	1185	27.2%	42.6%

The JUnit suite deliberately needs no database, so the JDBC DAO classes are exercised by the integration cases (IT08-IT10, WB22-WB27) and live runs rather than by unit tests. This is why the `dao` and `models` packages show 0% unit-test coverage and the total is low; the CLI, configuration and export code that the unit tests target is covered much better.

Table 6.18: coverage.py results for the Django suite

Django app	Statements	Line coverage	Branch coverage
accounts	66	95.5%	75.0%
admin_portal	185	98.9%	92.9%
campus_portal	67	77.6%	50.0%
core	625	99.2%	89.6%
faculty_portal	125	96.0%	91.7%
student_portal	20	100.0%	n/a
Total	1088	97.2%	87.5%

6.11.6 Black-box testing: equivalence partitioning and boundary value analysis

Internal marks have one valid class (0-40) and invalid classes below 0, above 40 and non-numeric. Boundaries were tested just outside, on and just inside each limit, and the grade scale at its boundaries.

Table 6.19: Equivalence partitioning and boundary value test cases

ID	Module	Objective	Input	Expected result	Actual result	Status
BB01	Faculty marks entry (internal, 0-40)	Validate internal marks	internal = 25	accepted	accepted, stored 25.00	Pass
BB02	Faculty marks entry (internal, 0-40)	Validate internal marks	internal = -5	rejected	rejected (form error), stored value unchanged 36.00	Pass
BB03	Faculty marks entry (internal, 0-40)	Validate internal marks	internal = 45	rejected	rejected (form error), stored value unchanged 36.00	Pass
BB04	Faculty marks entry (internal, 0-40)	Validate internal marks	internal = abc	rejected	rejected (form error), stored value unchanged 36.00	Pass
BB05	Faculty marks entry (internal, 0-40)	Validate internal marks	internal = -0.01	rejected	rejected (form error), stored value unchanged 36.00	Pass
BB06	Faculty marks entry (internal, 0-40)	Validate internal marks	internal = 0	accepted	accepted, stored 0.00	Pass
BB07	Faculty marks entry (internal, 0-40)	Validate internal marks	internal = 40	accepted	accepted, stored 40.00	Pass
BB08	Faculty marks entry (internal, 0-40)	Validate internal marks	internal = 40.01	rejected	rejected (form error), stored value unchanged 36.00	Pass

ID	Module	Objective	Input	Expected result	Actual result	Status
BB09	Grade scale via trg_marks_before_update	Grade at a scale boundary	39.99 + 0	0, F	39.99 -> 0, F	Pass
BB10	Grade scale via trg_marks_before_update	Grade at a scale boundary	40 + 0	5, P	40.00 -> 5, P	Pass
BB11	Grade scale via trg_marks_before_update	Grade at a scale boundary	30 + 59.99	9, A+	89.99 -> 9, A+	Pass
BB12	Grade scale via trg_marks_before_update	Grade at a scale boundary	30 + 60	10, O	90.00 -> 10, O	Pass
BB13	Attendance date (weekday rule, CHECK)	Friday session	2026-10-16	saved	saved	Pass
BB14	Attendance date (weekday rule, CHECK)	Saturday session	2026-10-17	rejected (weekend)	rejected: Attendance can only be recorded for Monday to Friday.	Pass
BB15	Attendance date (weekday rule, CHECK)	Monday session	2026-10-19	saved	saved	Pass
BB16	Student section (REGEXP_LIKE CHECK)	Validate section	section = 'A'	created	created	Pass
BB17	Student section (REGEXP_LIKE CHECK)	Validate section	section = 'a'	rejected	rejected: Section must be a single capital letter	Pass
BB18	Student section (REGEXP_LIKE CHECK)	Validate section	section = 'AB'	rejected	rejected: at most 1 character	Pass

6.11.7 Black-box testing: decision tables

Table 6.20: Decision table: enrollment

Rule	Semester matches	Already enrolled	Expected action
R1	Y	N	Enroll
R2	Y	Y	Refuse: already enrolled
R3	N	N	Refuse: wrong semester
R4	N	Y	Refuse: wrong semester

Table 6.21: Decision table: marks entry

Rule	Internal given	External given	Expected action
R1	N	N	Nothing written
R2	Y	N	Store; no total or grade yet
R3	Y	Y	Store; total and grade computed
R4	N	Y	Form error

Table 6.22: Decision table test cases

ID	Module	Objective	Input	Expected result	Actual result	Status
BB19	Enroll (semester match x already enrolled)	Rule: semester match=Y, enrolled=N	match=Y, enrolled=N	enrolled	enrolled	Pass
BB20	Enroll (semester match x already enrolled)	Rule: semester match=Y, enrolled=Y	match=Y, enrolled=Y	refused: This student is already enrolled	refused: This student is already enrolled in that course.	Pass
BB21	Enroll (semester match x already enrolled)	Rule: semester match=N, enrolled=N	match=N, enrolled=N	refused: CS502 is a semester 5 course	refused: CS502 is a semester 5 course, but 25CSE064 is in semester 3.	Pass
BB22	Enroll (semester match x already enrolled)	Rule: semester match=N, enrolled=Y	match=N, enrolled=Y	refused: CS502 is a semester 5 course	refused: CS502 is a semester 5 course, but 25CSE064 is in semester 3.	Pass
BB23	Marks entry (internal given x external given)	Rule from the marks decision table	internal='', external=''	no marks row written	no marks row written	Pass
BB24	Marks entry (internal given x external given)	Rule from the marks decision table	internal='30', external=''	stored; total=None, grade=None	stored; total=None, grade=None	Pass
BB25	Marks entry (internal given x external given)	Rule from the marks decision table	internal='30', external='50'	stored; total=80.00, grade=A+	stored; total=80.00, grade=A+	Pass
BB26	Marks entry (internal given x external given)	Rule from the marks decision table	internal='', external='50'	form error	form error: Enter internal marks before external marks.	Pass

6.11.8 Black-box testing: state transition

The marks record moves through the states of Figure 4.14 and a student's defaulter flag through those of Figure 4.15.

Table 6.23: State transition test cases

ID	Module	Objective	Input	Expected result	Actual result	Status
BB27	Marks life cycle	No marks -> internal -> graded -> re-graded	Save internal 30; then 30+50; then 35+50	No marks -> Internal only -> Graded (A+) -> Re-graded (A+, 2 audit rows)	No marks -> Internal only -> Graded (A+) -> Re-graded (A+, 2 audit rows)	Pass
BB28	Defaulter declaration (25CSE004, 66.67% / 58.33%)	Not declared -> Declared on 'declare'	Clear the flag, then CALL sp_refresh_defaulter_flags()	False -> True	declared flag False -> True	Pass

6.12 Defect identification and correction

Table 6.24: Defects found and corrected

ID	Defect	Found by	Correction	Status
D1	CHECK constraints for section and department code used `REGEXP`, which is case-insensitive under utf8mb4_unicode_ci, so 'a' was accepted	Django test test_check_constraint_message_for_bad_section	`REGEXP_LIKE(col, pattern, 'c')` in schema.sql (commit c96e1f6); test BB17 now passes	Fixed
D2	Paginated faculty and course lists had no stable order (Django UnorderedObjectListWarning)	Warning seen while collecting test evidence for this report	`.order_by()` added in admin_portal/views.py (commit 6de2bea)	Fixed
D3	Engine JSON written as Cp1252 when Java ran under a pipe on Windows	Integrating the bridge on Windows	`-Dfile.encoding=UTF-8` added to the argv	Fixed
D4	Menu misread the first command when input was piped from PowerShell (BOM)	Piping commands into `menu`	BOM stripped in `InteractiveMenu.readLine()`	Fixed
D5	Test assumed the next enrollment id would be 61; InnoDB left a gap (64)	Testing `enroll` against MySQL	Tests look rows up by roll number and course code	Fixed

No open defects remained when the final test run was made.

6.13 Test summary

Table 6.25: Test summary

Category	Executed	Passed	Failed
Unit	6	6	0
Integration	10	10	0
White-box	27	27	0
Black-box (incl. security)	35	35	0
JUnit suite	29	29	0
Django suite	71	71	0
System test pages	17	17	0
Total	195	195	0

All 195 executed tests and cases passed in the final run (78 of 78 documented cases, 29 JUnit tests, 71 Django tests and 17 live pages). Five defects found earlier were corrected before this run.

CHAPTER 7: DEPLOYMENT AND INSTALLATION

7.1 Introduction

This chapter explains how the system is installed and run. For the lab it is deployed on one Windows laptop; the same steps work on macOS and Linux with the shell commands adjusted.

7.2 Deployment architecture

All components run on one machine (deployment diagram, Figure 4.17): the browser, the Django development server on port 8000, a JVM started for each engine call, and MySQL Server on port 3306. For a real deployment Django would run behind a production web server with TLS and `DJANGO_HTTPS=True`.

7.3 Deployment environment

Table 7.1: Deployment environment

Component	Deployed as
Database	MySQL Community Server 8.0.42, database smart_campus_db, localhost:3306
Engine	java_engine/target/smart-campus-engine.jar (fat JAR with the JDBC driver)
Portal	Django 5.2.18 development server, http://127.0.0.1:8000/
Configuration	Repository-root .env (git-ignored), copied from .env.example
Client	Any browser on the same machine

7.4 System installation procedure

1. Install MySQL Server 8.0.16+, JDK 17+ and Python 3.10+ (Maven is downloaded by the wrapper).
2. Clone the repository and copy `.env.example` to `.env`; set `DB_PASSWORD`.
3. Create the virtual environment: `python -m venv .venv`, then `.venv\Scripts\python.exe -m pip install -r requirements.txt`.
4. Build the database: `.venv\Scripts\python.exe database\init_db.py --yes`.
5. Build the engine: `cd java_engine; .\mvnw.cmd clean package` (runs the 29 JUnit tests).
6. Prepare Django: `cd campus_portal`, then `manage.py migrate` and `manage.py seed_demo_users`.

7. Start the portal: `manage.py runserver` and open http://127.0.0.1:8000/.

7.5 Configuration details

All configuration is read from the repository-root `.env` file, which is git-ignored; only the template `.env.example` is committed. The values are not reproduced here.

Table 7.2: Configuration variables

Variable	Used by	Meaning
DB_HOST, DB_PORT, DB_NAME	Django, Java, init_db.py	MySQL server and database name
DB_USER, DB_PASSWORD	Django, Java, init_db.py	MySQL account (never committed)
DJANGO_SECRET_KEY	Django	Signing key (required when DEBUG is off)
DJANGO_DEBUG, DJANGO_ALLOWED_HOSTS	Django	Debug mode and host names
DJANGO_HTTPS	Django	Turns on secure cookies, SSL redirect and HSTS
DJANGO_DEMO_PASSWORD	seed_demo_users	Shared demo password (demo only)
DJANGO_JAVA_BIN, DJANGO_JAVA_ENGINE_JAR, DJANGO_JAVA_ENGINE_TIMEOUT	Bridge	Java executable, JAR path and timeout (default 30 s)

7.6 Database setup

`init_db.py` creates `smart_campus_db`, runs the three scripts and prints the row counts. On the development machine it completed in 1.3 s. It asks for confirmation unless `--yes` is given, because it drops and recreates every domain table (Django's tables are not touched).

```
PS C:\dev\3rdSemLabProject> .venv\Scripts\python.exe database\init_db.py --yes
Connected to localhost:3306 as root; database `smart_campus_db` is ready.
  ran schema.sql            22 statements
  ran views_triggers.sql    28 statements
  ran seed_data.sql         21 statements

Verification
  object                 type     rows
  ------------------------------------
  departments            table       2
  rooms                  table       5
  time_slots             table      20
  faculty                table       5
  students               table      10
  courses                table       6
  enrollments            table      60
  attendance_records     table     720
  marks                  table      60
  timetable_entries      table      20
  marks_audit            table       0
  v_attendance_summary   view       60
  v_defaulter_list       view        8
  v_student_cgpa_report  view       10
  v_marks_audit          view        0

  9 trigger(s), 2 function(s), 1 procedure(s); 2 student(s) flagged as defaulters

Done.
```

Figure 7.1: Database build with init_db.py

7.7 Application deployment steps

`mvnw.cmd clean package` compiled the engine, ran the JUnit suite and built the fat JAR in 13.7 s. `manage.py migrate` and `manage.py check` then completed with exit code 0, and the development server was started.

```
PS C:\dev\3rdSemLabProject\java_engine> .\mvnw.cmd clean package
[INFO] Building Smart Campus Java Engine 1.0.0-SNAPSHOT
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.195 s -- in com.smartcampus.cli.CliTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.069 s -- in com.smartcampus.cli.InteractiveMenuTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.053 s -- in com.smartcampus.cli.JsonWriterTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.174 s -- in com.smartcampus.config.DatabaseConfigTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.031 s -- in com.smartcampus.services.ReportExporterTest
[INFO] Tests run: 29, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building jar: C:\dev\3rdSemLabProject\java_engine\target\smart-campus-engine.jar
[INFO] BUILD SUCCESS
[INFO] Total time:  10.157 s
```

Figure 7.2: Maven build of the engine JAR

7.8 User access and credentials management

- Logins are Django users; passwords are stored only as salted hashes.
- `manage.py seed_demo_users` creates one login per student and faculty row with the same username, plus an `admin` superuser. The shared demo password is documented in the README for evaluators and is not repeated here. The command refuses to run when DEBUG is off unless `--force` is given.
- The role is never stored: it is derived from the username on each request (Section 4.14).
- Every user can change their own password at /accounts/password/.
- Database credentials live only in the git-ignored `.env`; a secret scan is run before every push.

7.9 User manual

7.9.1 Student

1. Log in with your roll number (for example 25cse001).
2. The dashboard shows each course with attendance %, marks, grade and your GPA and CGPA.
3. A red banner means at least one course is below 75% attendance.
4. Use "Change password" in the header; log out with the header button.

7.9.2 Faculty

1. Log in with your username (first.last).
2. Choose a course to see the roster.

3. Attendance: pick a weekday date, mark each student, save.
4. Marks: enter internal (0-40) and external (0-60) marks, save; totals and grades appear.

7.9.3 Admin

1. Log in as admin; the dashboard shows counts and averages.
2. Students / Faculty / Courses: search, add, edit or delete (a confirmation page is shown).
3. Enrollments and Timetable: add entries; clashes and duplicates are explained.
4. Defaulters: review the live list, then "Declare defaulter list now".
5. GPA report and Marks audit: read-only reports.
6. Java engine: choose a command, run it, read the tables or the error card.

7.9.4 Command line

```
java -jar java_engine/target/smart-campus-engine.jar help
java -jar java_engine/target/smart-campus-engine.jar student 25CSE001
java -jar java_engine/target/smart-campus-engine.jar dashboard
java -jar java_engine/target/smart-campus-engine.jar export gpa-report gpa.csv
java -jar java_engine/target/smart-campus-engine.jar menu
```

7.10 Deployment screenshots



Figure 7.3: Engine help output after deployment

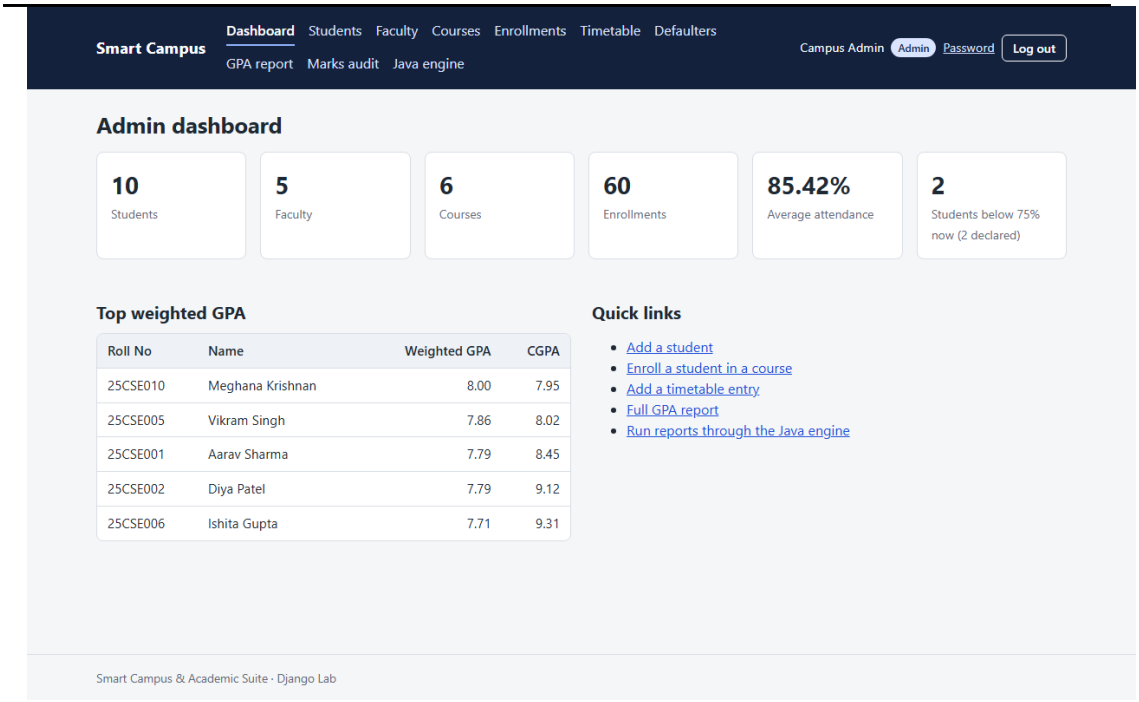


Figure 7.4: Portal running after deployment (admin dashboard)

7.11 Deployment challenges

- PowerShell's execution policy can block `Activate.ps1`; the steps call `.venv\Scripts\python.exe` directly so no system setting has to change.
- PowerShell has no `<` redirection, so the SQL scripts are run through `init_db.py`.
- A stale `DB_*` variable in the shell overrides `.env` for the JAR; the variables must be cleared.
- Django runs the last built JAR, so the engine must be rebuilt after any Java change.
- A fresh clone was tested end-to-end in Phase 4.5 to confirm that the six setup commands are enough.

CHAPTER 8: MAINTENANCE AND FUTURE ENHANCEMENTS

8.1 Introduction

Maintenance keeps the system correct and useful after delivery. This chapter classifies the maintenance work already done, describes how future changes will be handled, and lists the current limitations and planned enhancements. Change control and release practice are given in Appendix A, and the project risks in Appendix B.

8.2 Types of maintenance

Corrective maintenance

Fixing defects after they are found. Examples from this project: case-sensitive CHECK constraints (commit c96e1f6) and stable ordering of paginated lists (commit 6de2bea). Each fix comes with a test.

Adaptive maintenance

Adapting to a changed environment. The project stays on Django 5.2 LTS because Django 6.1 needs MySQL 8.4; moving to MySQL 8.4 will allow the upgrade. HTTPS settings are switched on by `DJANGO_HTTPS=True` when the portal moves behind TLS.

Perfective maintenance

Adding features or improving usability. Phase 4.5 added the marks audit trail, CSV export, the interactive menu, delete pages, search and pagination, password change and the Django admin.

Preventive maintenance

Preventing future faults: the URL-wide access test fails if a new page forgets its role check; rules live in the database so a new client cannot bypass them; scripts are re-runnable; dependency versions are pinned.

8.3 Bug fixing and updates

A bug is first reproduced with a failing test, then fixed in the layer that owns the rule (SQL, Java or Django), and committed together with the test. Schema changes are made in one commit that updates the SQL, the unmanaged Django model, the Java model/DAO and the ER diagram (Appendix A.4).

8.4 Performance improvement

- Reports read the database views instead of re-implementing joins in Python or Java.
- The engine dashboard loads its three views concurrently on a 3-thread pool.
- At most one engine call per page; most pages need none (about 50-70 ms).
- Future: a connection pool or a long-running engine service would remove the 0.6-0.8 s cost of each engine call.

8.5 Security updates

- Follow Django 5.2 LTS security releases (supported until April 2028) and MySQL 8.0 updates.
- Keep `cryptography` and the JDBC driver current.
- Before production: set DEBUG off, a real secret key, allowed hosts and HTTPS; replace the demo logins.
- Run `manage.py check --deploy` after every settings change.

8.6 User feedback and enhancements

Feedback from the guide and lab evaluators will be recorded as GitHub issues, prioritised, and handled through the change-control steps in Appendix A. Usability feedback during development already led to readable error messages with names instead of ids (for example in timetable clashes).

8.7 Limitations of the current system

- Single department and a single active academic year in the demo data.
- No e-mail or SMS notifications to students or parents.
- Runs on the Django development server on localhost; not hardened for the internet.
- Demo accounts share one password.
- Each engine call starts a new JVM (about 0.7 s).
- No bulk import of students or marks from spreadsheets.
- Acceptance testing was done by the project team, not by real staff and students.

8.8 Future enhancements

- Bulk CSV import of students, enrollments and marks.
- E-mail alerts to students who fall below 75% attendance.
- Multiple departments and academic years with history.
- Charts of attendance trends on the dashboards.
- Production deployment with Gunicorn/Nginx, TLS and a managed MySQL.
- A REST API for a mobile app.

8.9 Maintenance plan

Table 8.1: Maintenance plan

Activity	Frequency	Responsibility
Back up smart_campus_db (mysqldump)	Daily during the semester	Admin
Run both test suites	Before every commit	Project team
Apply Django / MySQL security releases	Monthly	Project team
Review marks_audit	End of each assessment	Admin
Declare defaulters	At each attendance cut-off	Admin
Review risks (Appendix B)	Each semester	Project team and guide

CHAPTER 9: RESULTS AND DISCUSSION

9.1 Introduction

This chapter presents what was built and how it performed. All numbers were measured on the development machine described in Section 5.2.

9.2 Results obtained

Table 9.1: Results obtained

Result	Value
Domain tables / views / triggers / functions / procedures	11 / 4 / 9 / 2 / 1
Seed data	10 students, 5 faculty, 6 courses, 60 enrollments, 720 attendance records
Live defaulter rows (v_defaulter_list)	8 course rows for 2 students (25CSE004, 25CSE009)
Java engine commands	13 (incl. help, menu, export)
Portal roles / main pages	3 roles; 17 pages measured
JUnit tests	29 run, 0 failures
Django tests	71 run, result OK
Django line coverage	96.0%
Documented test cases	78 executed, all passed

9.3 Output screenshots

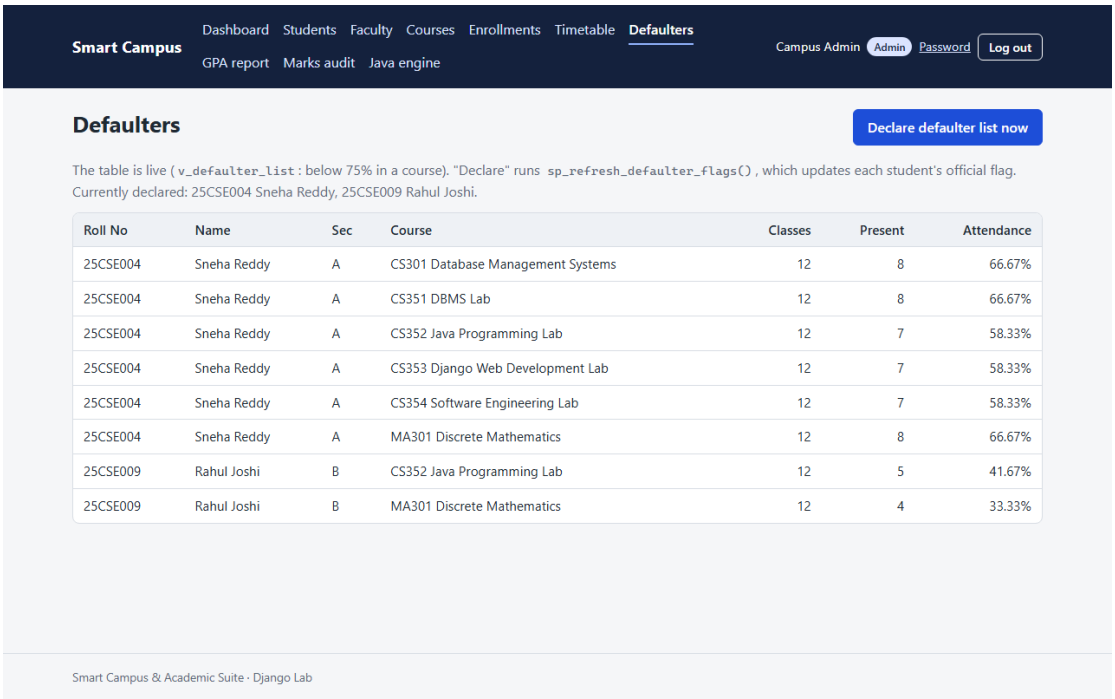


Figure 9.1: Admin defaulter list with the declare button

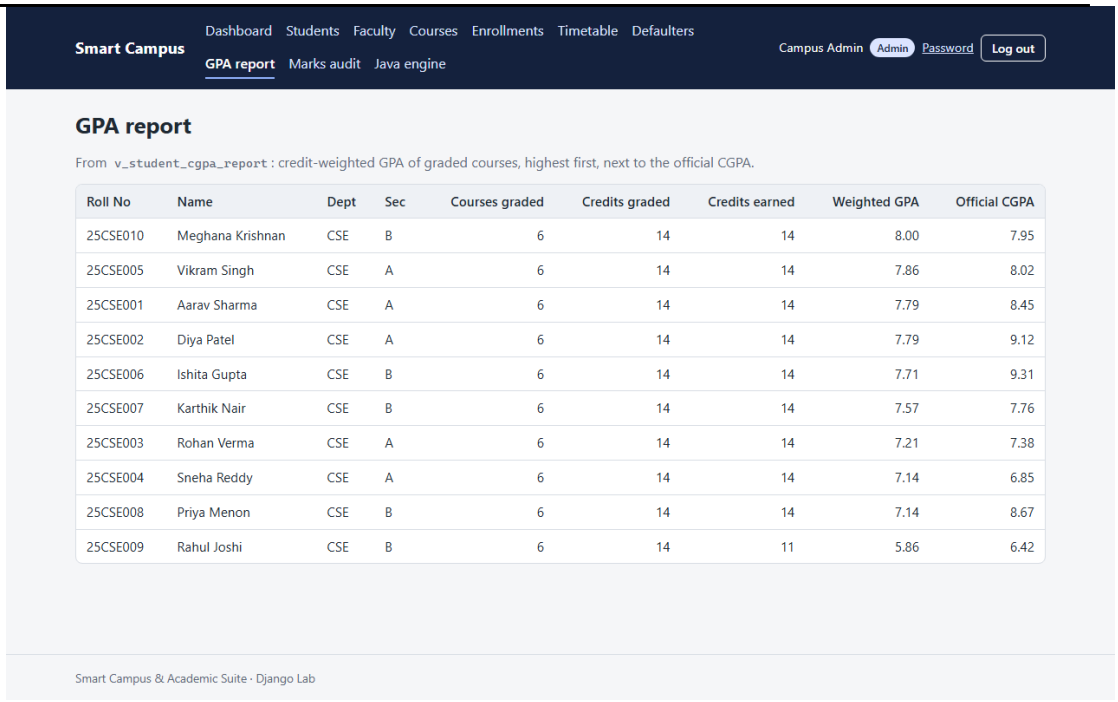


Figure 9.2: GPA report: weighted GPA next to the official CGPA

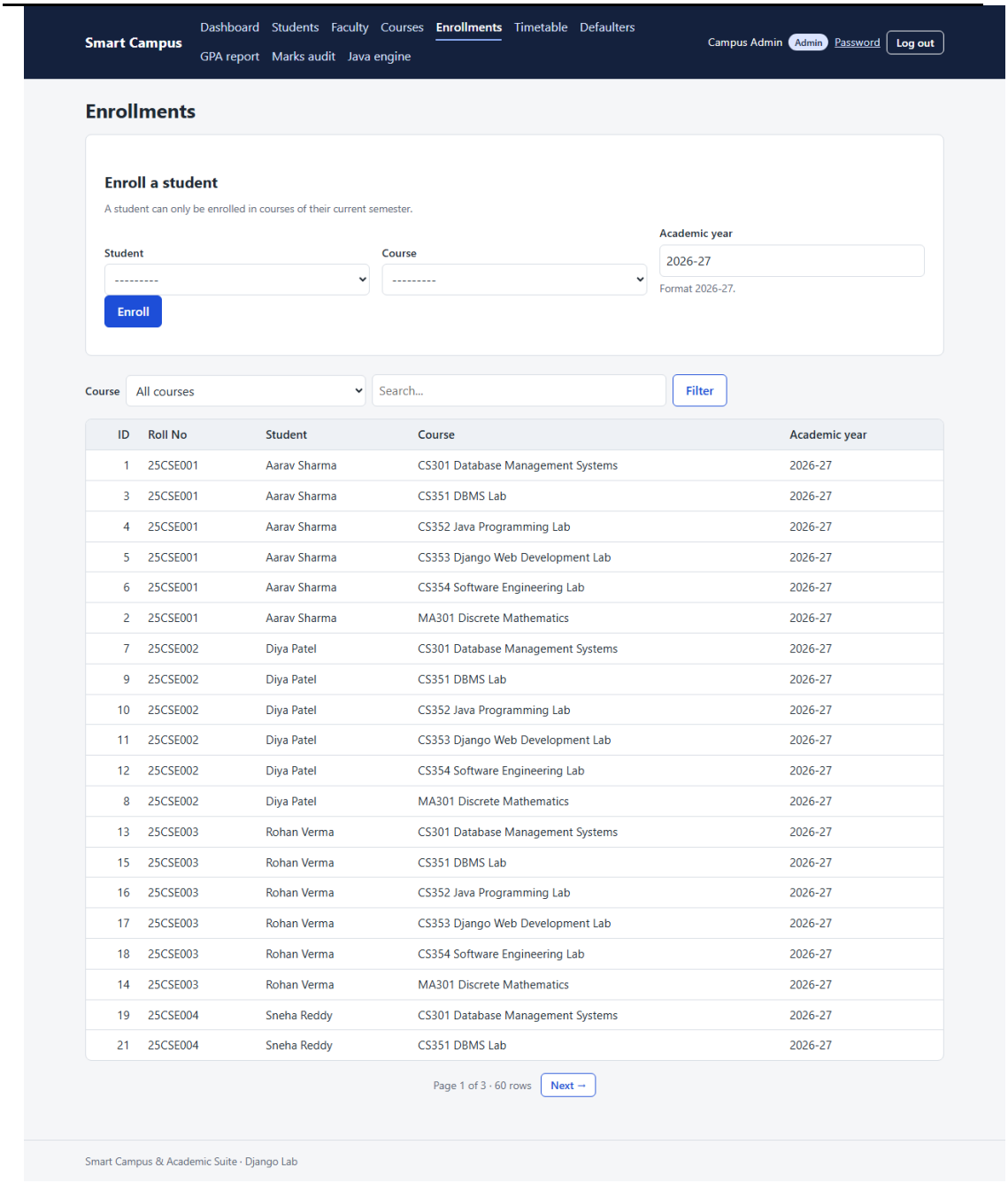


Figure 9.3: Enrollments list and enroll form

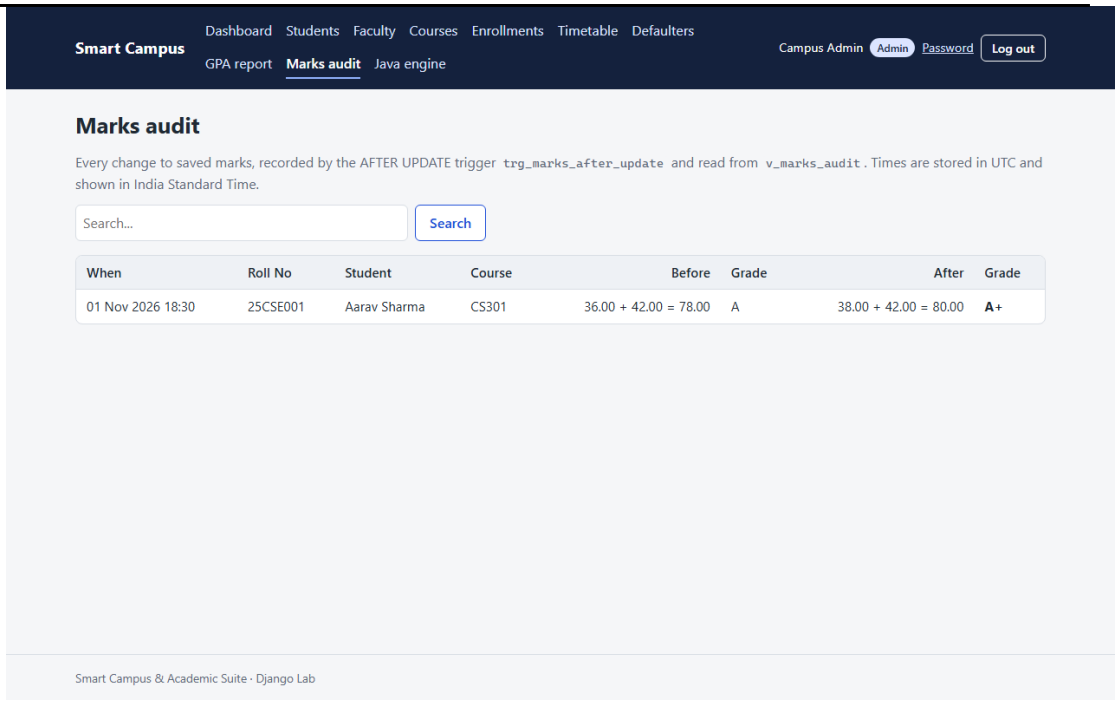


Figure 9.4: Marks audit history written by the AFTER UPDATE trigger

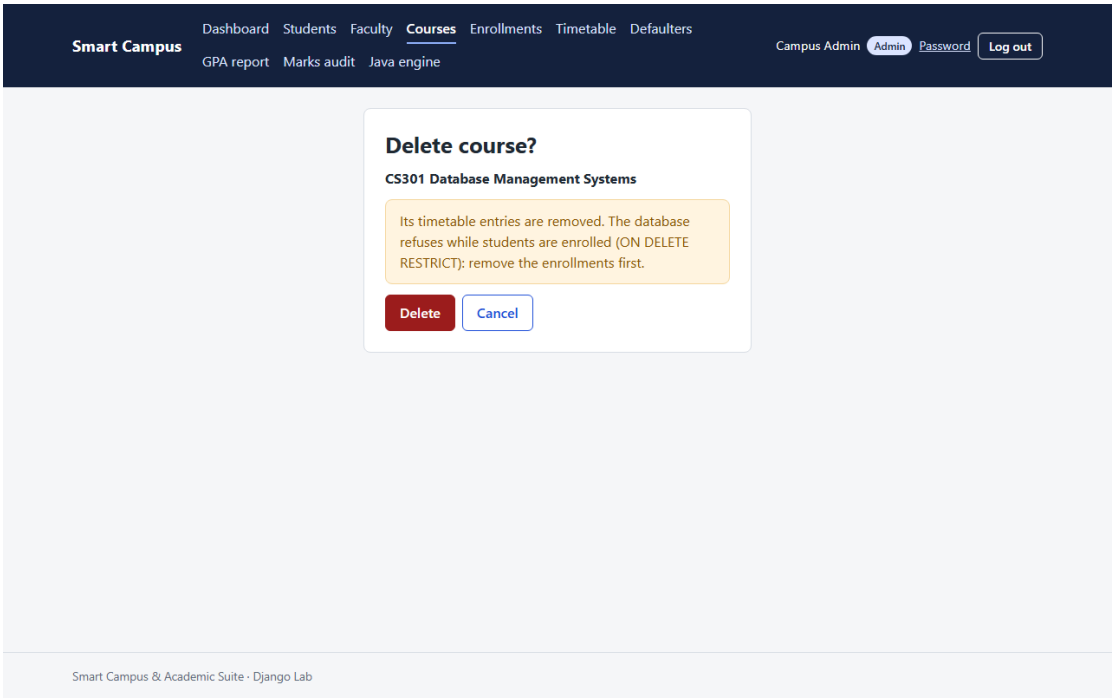


Figure 9.5: Delete confirmation page

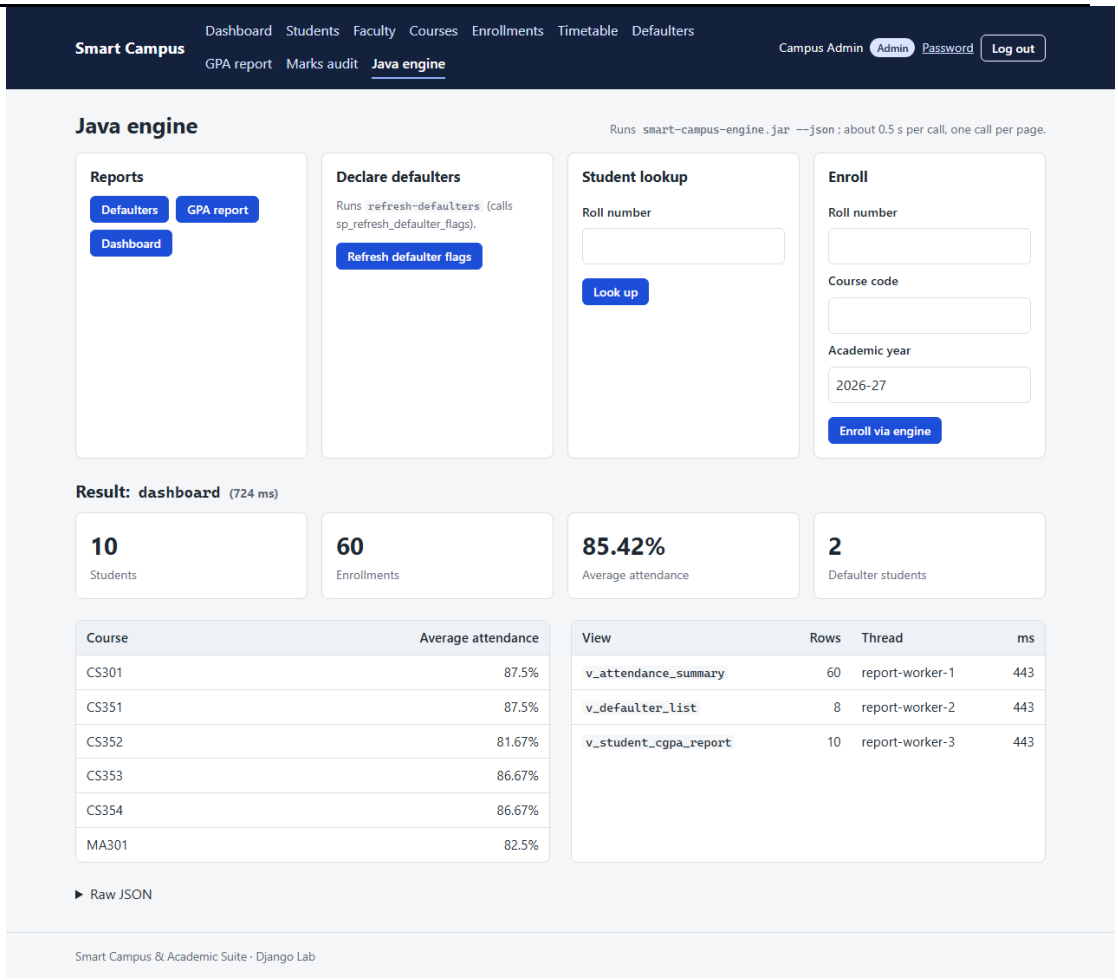


Figure 9.6: Java engine dashboard run from the portal, with thread names and timings

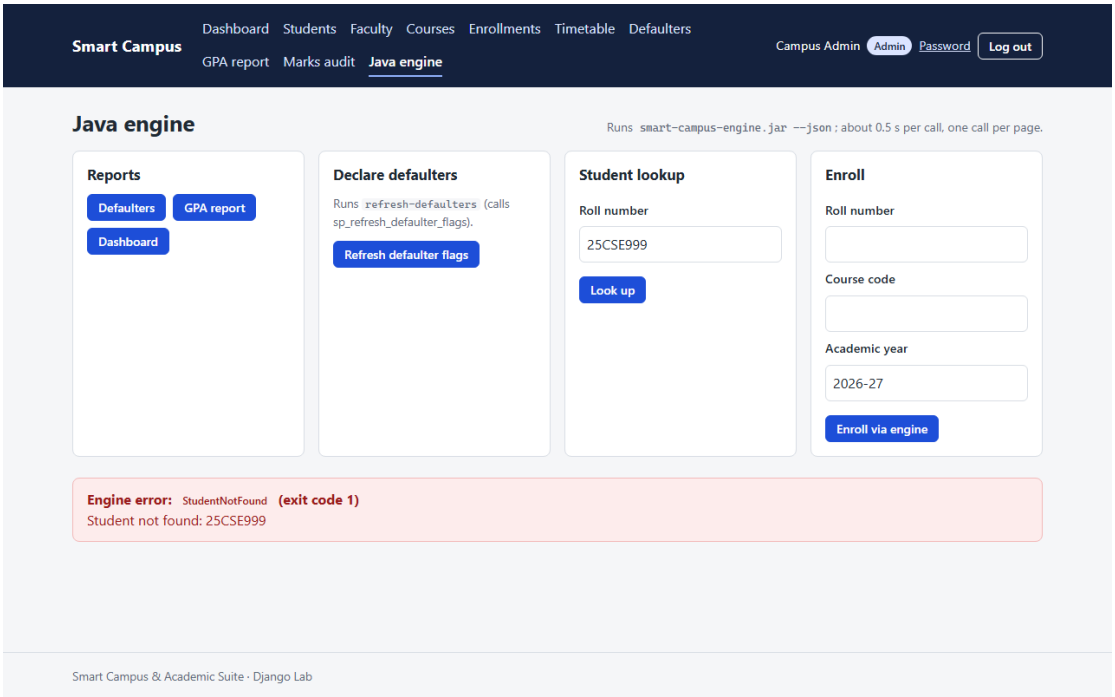


Figure 9.7: Typed error card from the engine page

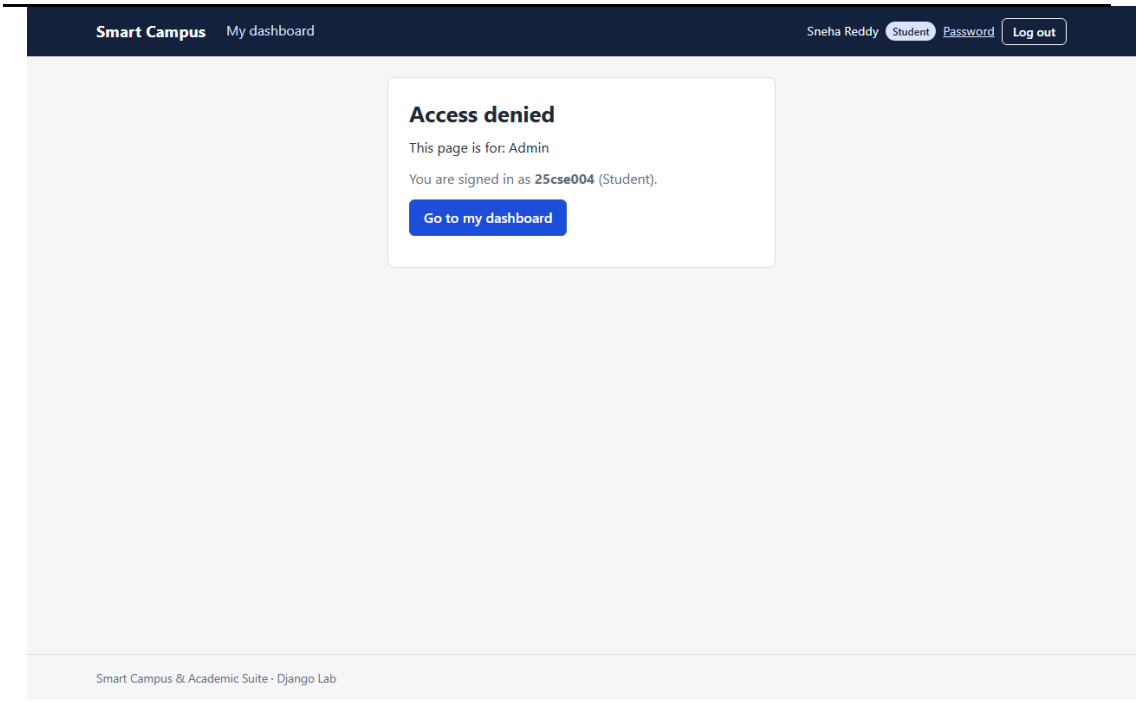


Figure 9.8: Access denied (403) page for a wrong role

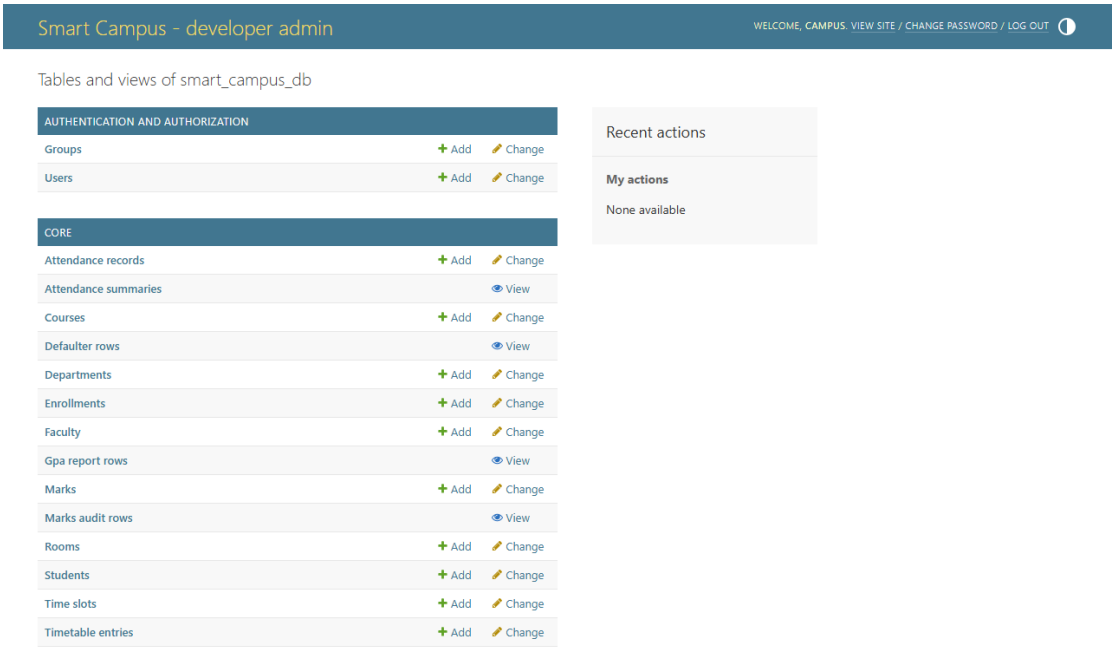


Figure 9.9: Django admin over the domain tables and the four views

```
PS C:\dev\3rdSemLabProject> java -jar java_engine\target\smart-campus-engine.jar dashboard
Smart Campus dashboard

Students:            10
Enrollments:         60
Average attendance:  85.42%
Defaulters:          2 student(s), 8 course entries below 75%

Average attendance by course
Course  Attendance
------  ----------
CS301       87.50%
CS351       87.50%
CS352       81.67%
CS353       86.67%
CS354       86.67%
MA301       82.50%

Top weighted GPA
Roll No   Name              Weighted GPA
--------  ----------------  ------------
25CSE010  Meghana Krishnan          8.00
25CSE005  Vikram Singh              7.86
25CSE001  Aarav Sharma              7.79

Loaded concurrently in 469 ms
View                    Rows  Thread            ms
--------------------  ----  ---------------  ---
v_attendance_summary     60  report-worker-1  456
v_defaulter_list          8  report-worker-2  455
v_student_cgpa_report    10  report-worker-3  455
```

Figure 9.10: Engine dashboard in the terminal

```
PS C:\dev\3rdSemLabProject> java -jar java_engine\target\smart-campus-engine.jar defaulters
2 student(s) below 75% attendance in 8 course(s)

25CSE004  Sneha Reddy (section A)
  Course  Name                       Classes  Present  Attendance
  ------  -------------------------  -------  -------  ----------
  CS301   Database Management Systems       12        8      66.67%
  CS351   DBMS Lab                          12        8      66.67%
  CS352   Java Programming Lab              12        7      58.33%
  CS353   Django Web Development Lab        12        7      58.33%
  CS354   Software Engineering Lab          12        7      58.33%
  MA301   Discrete Mathematics              12        8      66.67%

25CSE009  Rahul Joshi (section B)
  Course  Name                  Classes  Present  Attendance
  ------  --------------------  -------  -------  ----------
  CS352   Java Programming Lab       12        5      41.67%
  MA301   Discrete Mathematics       12        4      33.33%
```

Figure 9.11: Engine defaulter report in the terminal

```
PS C:\dev\3rdSemLabProject> java -jar java_engine\target\smart-campus-engine.jar student 25CSE004
Student 25CSE004 - Sneha Reddy <25cse004@smartcampus.example>, CSE, semester 3 section A, CGPA 6.85, DEFAULTER
Username: 25cse004

Course  Name                       Classes  Present  Attendance  Internal  External  Total  Grade
------  -------------------------  -------  -------  ----------  --------  --------  -----  -----
CS301   Database Management Systems       12        8      66.67%     18.00     24.50  42.50  P
CS351   DBMS Lab                          12        8      66.67%     35.00     21.00  56.00  B
CS352   Java Programming Lab              12        7      58.33%     24.00     32.00  56.00  B
CS353   Django Web Development Lab        12        7      58.33%     29.00     43.00  72.00  A
CS354   Software Engineering Lab          12        7      58.33%     34.00     54.00  88.00  A+
MA301   Discrete Mathematics              12        8      66.67%     30.00     51.00  81.00  A+
```

Figure 9.12: Engine student lookup in the terminal

9.4 Performance evaluation

Table 9.2: Performance evaluation

Measurement	Result
Ordinary portal pages (median of medians, 15 pages)	62.7 ms (range 47.4-69.2 ms)

Measurement	Result
Engine page: dashboard (median)	750.4 ms
Engine page: defaulters (median)	668.7 ms
CLI help (JVM start-up only)	168 ms
CLI commands that query MySQL	636-813 ms
Database rebuild (init_db.py)	1.3 s
Engine build with tests (mvnw clean package)	13.7 s
Django test suite (71 tests)	9.4 s test time, 16.7 s total

The difference between `help` (no database) and the other commands, about 0.5 s, is the cost of loading the JDBC driver and opening a MySQL connection; the JVM itself starts in about 0.17 s.

9.5 Comparison with the existing system

Table 9.3: Existing system vs proposed system

Aspect	Existing (manual)	Proposed system
Data storage	Registers and spreadsheets, duplicated	One 3NF database
Grade calculation	By hand or ad-hoc formulas	Trigger, identical for everyone
Attendance %	Totalled near the cut-off	Live from a view
Defaulter list	Prepared manually, late	Live list; declared with one click
Timetable clashes	Found when they happen	Refused when entered
Access control	Whoever holds the file	Role checked on every page
Change history	None	Automatic marks audit
Report time	Days	Under 1 s (Table 9.2)

9.6 Discussion of results

Placing the rules in the database proved to be the key design decision. Tests IT02, IT05 and IT06 show that a change made through one application is graded, checked and audited without that application doing any of the work, and IT09-IT10 show the Java engine receiving the same answers and errors. Security tests showed no way for a student or teacher to reach another person's data, and the URL-wide test guarantees this for pages added later. The main cost is the engine's start-up time, which is acceptable for one call per page but would need a resident service for heavy use. Unit-test coverage of the Java DAO layer is low by design (no database in unit tests); that layer is covered by the integration and loop cases instead.

9.7 Achievement of project objectives

Table 9.4: Achievement of objectives

Objective	Achieved	Evidence
O1 Normalised database with rules	Yes	Section 4.8-4.9; CHECK/UNIQUE/FK tests
O2 Derived values in the database	Yes	WB01-WB08, BB09-BB12, IT02, IT03
O3 Role-based portal	Yes	IT01, WB09-WB11, BB34-BB35, test_polish
O4 OOP Java engine with JDBC	Yes	Class diagrams; 29 JUnit tests; IT09-IT10
O5 Safe portal-engine bridge	Yes	UT03-UT04, IT08, BB30
O6 Tests and design documents	Yes	Chapters 4 and 6; Appendix C

9.8 Summary

The system meets all six objectives and every non-functional requirement that was measured. Pages respond in tens of milliseconds, engine reports in under a second, and all automated tests and documented cases pass.

CHAPTER 10: CONCLUSION

10.1 Conclusion

Smart Campus & Academic Suite replaces registers and spreadsheets with one consistent database and role-based access for students, faculty and the administrator. By enforcing grading, attendance, enrollment and timetable rules inside MySQL, it guarantees that the Django portal, the Java engine and any future client always agree. The system was designed, implemented, tested and documented following the SDLC, and all planned objectives were achieved.

10.2 Summary of the project

- A 3NF MySQL schema with 11 tables, 4 views, 9 triggers, 2 functions and 1 procedure.
- A Java 17 engine with an OOP domain model, JDBC DAOs, services, a concurrent dashboard, CSV export and a CLI with JSON output.
- A Django 5.2 portal with Student, Faculty and Admin roles, CRUD, reports, an audit view and a bridge to the engine.
- Requirements, UML, DFD and ER design; 78 documented test cases and 100 automated tests, all passing.

10.3 Contribution of the project

The project is the joint work of a four-member team (Section 5.7.7), which together built the database, the Java engine, the Django portal and the Software Engineering documentation. Its main contributions are:

- Shows how one database can serve two applications in different languages with identical rules.
- Provides an automatic, tamper-evident history of marks changes.
- Gives students early warning of attendance shortfalls.
- Serves as a single worked example for the DBMS, Java, Django and Software Engineering labs.

10.4 Learning outcomes

- Normalising a real schema and expressing business rules as constraints, triggers

and views.

- Applying OOP (abstraction, inheritance, interfaces, generics, exceptions) and JDBC transactions in Java.
- Building a secure MVT web application with role-based access in Django.
- Integrating programs across languages through a process boundary and a JSON contract.
- Writing an SRS, drawing UML/DFD/ER diagrams as code, designing white- and black-box tests, and managing the work with Git.

10.5 Scope for future work

Future work includes bulk import, notifications, multi-department support, charts, a REST API for a mobile app and a production deployment with TLS (Section 8.8), as well as a formal acceptance test with real faculty and students.

REFERENCES

[1] E. F. Codd, "A relational model of data for large shared data banks," Commun. ACM, vol. 13, no. 6, pp. 377-387, Jun. 1970, doi: 10.1145/362384.362685.

[2] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," Computer, vol. 29, no. 2, pp. 38-47, Feb. 1996, doi: 10.1109/2.485845.

[3] D. F. Ferraiolo, J. F. Barkley, and D. R. Kuhn, "A role-based access control model and reference implementation within a corporate intranet," ACM Trans. Inf. Syst. Secur., vol. 2, no. 1, pp. 34-64, Feb. 1999, doi: 10.1145/300830.300834.

[4] M. N. F. Hidayah and H. Sofyan, "Design and implementation of an integrated web-based academic management and student admission monitoring system using Django framework," CODEX: J. Softw. Eng., vol. 1, no. 1, 2026, doi: 10.33650/codex.v1i01.14454.

[5] J. Thilagavathi, G. P. Narasimha, S. Yasaswini, and E. Aiswarya, "College management system using Python & Django," Shanlax Int. J. Arts, Sci. Humanities, vol. 11, no. S3, Jul. 2024, doi: 10.34293/sijash.v11iS3-July.7911.

[6] M. J. Davis and Z. Huang, "ERP in higher education: A case study of SAP and Campus Management," Issues Inf. Syst., vol. VIII, no. 1, pp. 120-126, 2007. [Online]. Available: https://iacis.org/iis/2007/Davis_Huang.pdf

[7] OS4ED, "openSIS Classic: Open-source student information system," GitHub repository. [Online]. Available: https://github.com/OS4ED/openSIS-Classic (accessed Nov. 1, 2026).

[8] Foradian Technologies, "Fedena: School management software," [Online]. Available: https://fedena.com/ (accessed Nov. 1, 2026).

[9] D. Marsden et al., "Attendance (mod_attendance)," Moodle Plugins Directory. [Online]. Available: https://moodle.org/plugins/mod_attendance (accessed Nov. 1, 2026).

[10] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill, 2019.

[11] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2015.

[12] G. Booch, J. Rumbaugh, and I. Jacobson, The Unified Modeling Language User Guide, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.

[13] Django Software Foundation, "Django 5.2 documentation," [Online]. Available: https://docs.djangoproject.com/en/5.2/ (accessed Nov. 1, 2026).

[14] Oracle Corporation, "MySQL 8.0 reference manual," [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/ (accessed Nov. 1, 2026).

[15] PlantUML, "PlantUML language reference guide," [Online]. Available: https://plantuml.com/ (accessed Nov. 1, 2026).

[16] The JUnit Team, "JUnit user guide," [Online]. Available: https://docs.junit.org/ (accessed Nov. 1, 2026).

[17] Microsoft, "Playwright for Python," [Online]. Available: https://playwright.dev/python/ (accessed Nov. 1, 2026).

[18] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998, doi: 10.1109/IEEESTD.1998.88286.

APPENDIX A: SOFTWARE CONFIGURATION MANAGEMENT

Software Configuration Management (SCM) identifies the items that make up the system, controls changes to them, and records how each version was built and released. This appendix describes the SCM actually used in the project and is referred to from Chapters 5 and 8.

A.1 Configuration identification

Table A.1: Configuration items

Configuration item	Location	Identified by
Database scripts	database/*.sql, init_db.py	Git path + commit
Java engine source and build	java_engine/ (pom.xml, src/)	Maven version 1.0.0-SNAPSHOT + commit
Django portal	campus_portal/	Git path + commit
Dependencies	requirements.txt, pom.xml	Pinned versions
Configuration template	.env.example (real .env is git-ignored)	Git path
Design documents	docs/*.md, docs/src/diagrams/*.puml	Git path + commit
Test evidence	docs/src/evidence/*.json	Generated by scripts; committed
Project guide	CLAUDE.md, README.md	Updated when each phase lands

A.2 Version control

The repository is managed with Git and hosted on GitHub; all work is on the `main` branch. Commit messages follow the Conventional Commits style (`feat`, `fix`, `docs`, `test`, `chore` with a scope such as `database`, `java` or `django`). Table A.2 lists all 28 commits up to the conversion of this report into a group report (249 tracked files at that point); only the commit that stores the regenerated document itself comes after it.

The repository (`rishi-ventrapragada/3rdSemLabProject`) is hosted on V. Sai Rishi's GitHub account, and every commit in the history was made from that account; the team's work was integrated through this single repository.

Table A.2: Git history of the project

Commit	Date	Message
4ae9dc0	2026-10-15	chore: scaffold 4-lab project structure, Maven fat-JAR setup, and CLAUDE.md
c62e999	2026-10-16	feat(database): add 3NF schema, views, triggers, seed data and init_db.py
30445ca	2026-10-31	feat(java): add DB config loader and checked exception hierarchy
c9efb16	2026-10-31	feat(java): add domain models with Person -> Student/Faculty hierarchy
0b1d670	2026-10-31	feat(java): add DAO interfaces and JDBC implementations

Commit	Date	Message
ecf5b18	2026-10-31	feat(java): add student and report services
f7aa6bb	2026-10-31	feat(java): add CLI commands with text and --json output
5937f1f	2026-10-31	docs: document Phase 3 Java engine commands and pitfalls in CLAUDE.md
2177c9d	2026-10-31	feat(django): create campus_portal project with unmanaged models
af5e1b5	2026-10-31	feat(django): add login, role routing, demo users and base UI
9d320e0	2026-10-31	feat(django): add student and faculty portals with DB-backed writes
fea3450	2026-10-31	feat(django): add admin portal with reports and CRUD
d523464	2026-10-31	feat(django): add admin-only Java engine bridge page
8442463	2026-10-31	test(django): add role, bridge and real-database tests
6f74e2b	2026-10-31	docs: document the Django portal, demo logins and Phase 4 pitfalls
c96e1f6	2026-11-01	fix(database): make section and dept_code CHECKs case-sensitive
e586990	2026-11-01	feat(database): add marks audit trail (AFTER trigger) and lab query examples
d0d4413	2026-11-01	feat(java): add CSV export (file I/O) and an interactive Scanner menu
eee70ad	2026-11-01	feat(django): complete CRUD, Django admin, audit pages and UX polish
659f320	2026-11-01	feat(django): harden session/cookie settings and add URL-wide access tests
dd549b1	2026-11-01	docs: add README and document the Phase 4.5 changes
6de2bea	2026-11-01	fix(django): give paginated faculty and course lists a stable order
7f90322	2026-11-01	docs(report): add evidence collectors and their real-run results
e7eec24	2026-11-01	docs(report): add PlantUML sources and rendered diagrams
ebc6299	2026-11-01	docs(report): add portal and CLI screenshots from the live run
cac55d6	2026-11-01	docs(report): add the python-docx report builder
896a2bd	2026-11-01	docs: add the SE Lab project report (Phase 5)
be5c1de	2026-11-04	docs(report): convert the report sources to a group report

A.3 Baselines

Table A.3: Baselines

Baseline	Commit	Contents
Phase 1: scaffolding	4ae9dc0	Folder structure, Maven fat-JAR setup, project guide
Phase 2: database	c62e999	Schema, views, triggers, seed data, init_db.py
Phase 3: Java engine	5937f1f	Config, models, DAOs, services, CLI, tests, docs
Phase 4: Django portal	6f74e2b	Three portals, bridge, tests, docs
Phase 4.5: polish	6de2bea	Audit trail, CSV export, menu, CRUD, security, fixes
Phase 5: SE report	896a2bd	Evidence scripts, diagrams, screenshots, report builder, report

A.4 Change control

1. A change starts from a requirement, a defect (Chapter 6.12) or review feedback.
2. Its impact is checked against the rule "the SQL scripts are the source of truth": a schema change must update, in one commit, the SQL, the unmanaged Django model, the Java model and DAO, and the ER diagram.
3. The change is made with a test that fails before and passes after.
4. Both suites are run; then a secret scan checks that no `.env` content or password is staged.

5. The commit is pushed only after the scan is clean; database fixes were approved by the team before `database/` was touched.

A.5 Build and release management

Table A.4: Build and release

Artifact	Built by	Release check
smart_campus_db	python database/init_db.py --yes	Row counts printed; Django tests load the same scripts
smart-campus-engine.jar	mvnw clean package (Maven 3.9.16 pinned by the wrapper)	29 JUnit tests must pass
Django portal	pip install -r requirements.txt; migrate	manage.py check; 71 tests must pass
Project report	docs/src/report/build_report.py	Rendered with LibreOffice and checked page by page

A.6 SCM tools

Table A.5: SCM tools

Tool	Use
Git 2.53.0	Version control, history, baselines
GitHub	Remote repository and backup
.gitignore	Keeps .env, build output, virtual environments and downloaded tools out of Git
.gitattributes	Forces LF line endings for mvnw
Maven Wrapper	Reproducible Maven version for every build
requirements.txt / pom.xml	Pinned dependency versions

APPENDIX B: RISK MANAGEMENT

Risks were identified at the start of each phase, rated for probability and impact (Low, Medium, High), and tracked until closed. This appendix is referred to from Chapter 8.

Table B.1: Risk register

ID	Risk	Prob.	Impact	Mitigation	Status
R1	Database password or .env committed to GitHub	Medium	High	.env git-ignored; secret scan in a separate step before every push	Controlled
R2	Rules implemented differently in Django and Java	High	High	Rules placed in MySQL triggers, views and constraints	Closed
R3	init_db.py run against real data and erases it	Low	High	Confirmation prompt unless --yes; Django tables never touched	Controlled
R4	Engine JAR missing or Java not installed on the server	Medium	Medium	Bridge checks the JAR first; typed errors with a build hint; never a 500 page	Closed
R5	Command injection through the bridge	Low	High	Argument list, shell=False, allow-list, regex, timeout	Closed
R6	Framework incompatibility (Django 6.1 needs MySQL 8.4)	Medium	Medium	Stay on Django 5.2 LTS	Closed
R7	Platform differences (Windows encodings, BOM, line endings)	High	Medium	-Dfile.encoding=UTF-8, BOM stripping, .gitattributes for mvnw	Closed
R8	A new page added without a role check	Medium	High	URL-wide access test fails the suite	Controlled
R9	Schedule slip near the semester end	Medium	Medium	Phased plan; each phase committed and usable on its own	Closed
R10	Slow engine calls if used per table row	Medium	Medium	Design rule: one engine call per page	Controlled
R11	Loss of the laptop or disk	Low	High	Code pushed to GitHub after each phase; database rebuilt from scripts	Controlled

Risk exposure was highest in Phase 3 and 4 (R2, R5, R7), when the two applications were connected; all three were closed by design decisions that are verified by tests in Chapter 6.

APPENDIX C: AUTOMATED TEST LOG

Every automated test from the final run on 2026-11-01, as reported by Maven Surefire and the Django test runner.

Table C.1: JUnit tests

#	JUnit class	Test	Time (s)	Status
1	cli.CliTest	academicYearRollsOverInJuly	0.123	Pass
2	cli.CliTest	wrongArgumentCountIsAUsageErrorBeforeAnyDatabaseCall	0.026	Pass
3	cli.CliTest	optionsAreCheckedPerCommand	0.004	Pass
4	cli.CliTest	usageErrorsAreJsonOnStderrWhenRequested	0.006	Pass
5	cli.CliTest	unknownCommandIsAUsageError	0.002	Pass
6	cli.CliTest	noArgumentsShowsHelp	0.008	Pass
7	cli.CliTest	helpListsEveryCommandAndExitsZero	0.003	Pass
8	cli.InteractiveMenuTest	listsChoicesAndExitsOnZero	0.046	Pass
9	cli.InteractiveMenuTest	blankAnswerCancelsWithoutRunningTheCommand	0.004	Pass
10	cli.InteractiveMenuTest	endOfInputExitsCleanly	0.002	Pass
11	cli.InteractiveMenuTest	leadingByteOrderMarkFromPowerShellPipesIsIgnored	0.003	Pass
12	cli.InteractiveMenuTest	rejectsUnknownChoices	0.004	Pass
13	cli.InteractiveMenuTest	menuRefusesJson	0.031	Pass
14	cli.JsonWriterTest	escapesBackslash	0.002	Pass
15	cli.JsonWriterTest	writesAWholeStudentRecordWithAHostileName	0.008	Pass
16	cli.JsonWriterTest	writesNullsNumbersAndNestedCollections	0.006	Pass
17	cli.JsonWriterTest	escapesQuoteInAName	0.003	Pass
18	cli.JsonWriterTest	rejectsValuesJsonCannotRepresent	0.011	Pass
19	cli.JsonWriterTest	leavesOrdinaryUnicodeAndApostrophesAlone	0.003	Pass
20	cli.JsonWriterTest	escapesLineAndParagraphSeparators	0.003	Pass
21	cli.JsonWriterTest	escapesOtherControlCharactersAsUnicode	0.004	Pass
22	cli.JsonWriterTest	escapesNewlinesTabsAndShortEscapes	0.002	Pass
23	config.DatabaseConfigTest	findsEnvFileInAParentDirectory(Path)	0.225	Pass
24	config.DatabaseConfigTest	parsesEnvFileSkippingCommentsAndBlankLines(Path)	0.018	Pass
25	config.DatabaseConfigTest	reportsMissingAndInvalidSettings	0.011	Pass
26	config.DatabaseConfigTest	buildsJdbcUrlWithRequiredOptionsAndHidesPassword	0.017	Pass
27	services.ReportExporterTest	writesHeaderAndRowsAndCreatesParentFolders(Path)	0.056	Pass
28	services.ReportExporterTest	quotesFieldsWithCommasQuotesAndLineBreaks	0.002	Pass
29	services.ReportExporterTest	reportNamesAreCaseInsensitiveAndUnknownOnesAreRejected	0.009	Pass

Table C.2: Django tests

#	Module.Class	Test	Status
1	test_access.LoginRedirectTests	test_each_role_lands_on_its_dashboard	Pass
2	test_access.LoginRedirectTests	test_login_honours_next	Pass
3	test_access.LoginRedirectTests	test_login_not_linked_to_a_person_gets_403_not_a_crash	Pass
4	test_access.LoginRedirectTests	test_logout_requires_post	Pass
5	test_access.LoginRedirectTests	test_wrong_password_stays_on_login	Pass
6	test_access.RoleAccessTests	test_admin_reaches_only_the_admin_portal_and_engine	Pass
7	test_access.RoleAccessTests	test_anonymous_users_are_sent_to_login	Pass
8	test_access.RoleAccessTests	test_faculty_reaches_only_the_faculty_portal	Pass
9	test_access.RoleAccessTests	test_student_reaches_only_the_student_portal	Pass
10	test_access.RoleAccessTests	test_teacher_cannot_open_another_teachers_course	Pass
11	test_access.RoleAccessTests	test_unknown_course_is_404	Pass
12	test_java_bridge.EngineViewTests	test_engine_error_type_is_shown	Pass

#	Module.Class	Test	Status
13	test_java_bridge.EngineViewTests	test_enroll_form_passes_year	Pass
14	test_java_bridge.EngineViewTests	test_gpa_report_renders_a_table_with_one_call	Pass
15	test_java_bridge.EngineViewTests	test_invalid_roll_number_never_reaches_the_engine	Pass
16	test_java_bridge.EngineViewTests	test_missing_jar_shows_build_instructions	Pass
17	test_java_bridge.EngineViewTests	test_non_admins_get_403	Pass
18	test_java_bridge.EngineViewTests	test_page_without_a_command_makes_no_call	Pass
19	test_java_bridge.EngineViewTests	test_student_lookup_passes_the_validated_roll_number	Pass
20	test_java_bridge.EngineViewTests	test_writes_require_post	Pass
21	test_polish.DeleteTests	test_confirm_page_then_cascading_delete_of_a_student	Pass
22	test_polish.DeleteTests	test_course_with_enrollments_is_protected_by_restrict	Pass
23	test_polish.DeleteTests	test_delete_requires_post	Pass
24	test_polish.DeleteTests	test_deleting_a_teacher_unassigns_their_courses	Pass
25	test_polish.DjangoAdminTests	test_deletes_are_routed_to_the_portal	Pass
26	test_polish.DjangoAdminTests	test_every_registered_model_has_a_working_changelist	Pass
27	test_polish.ErrorPageAndHeaderTests	test_custom_404_page	Pass
28	test_polish.ErrorPageAndHeaderTests	test_custom_500_page_needs_no_context	Pass
29	test_polish.ErrorPageAndHeaderTests	test_security_headers_and_session_cookie	Pass
30	test_polish.EveryUrlIsProtectedTests	test_anonymous_requests_redirect_to_login	Pass
31	test_polish.EveryUrlIsProtectedTests	test_each_role_is_refused_outside_its_own_portal	Pass
32	test_polish.MarksAuditTests	test_changed_marks_are_audited_by_the_after_trigger	Pass
33	test_polish.PasswordChangeTests	test_user_can_change_password_and_log_in_with_it	Pass
34	test_polish.PasswordChangeTests	test_weak_or_mismatched_passwords_are_rejected	Pass
35	test_polish.SearchAndPaginationTests	test_active_navigation_link	Pass
36	test_polish.SearchAndPaginationTests	test_empty_search_result_shows_an_empty_state	Pass
37	test_polish.SearchAndPaginationTests	test_enrollments_are_paginated	Pass
38	test_polish.SearchAndPaginationTests	test_search_filters_and_keeps_other_filters	Pass
39	test_portals.AdminPortalTests	test_check_constraint_message_for_bad_section	Pass
40	test_portals.AdminPortalTests	test_declaring_defaulters_calls_the_procedure	Pass
41	test_portals.AdminPortalTests	test_duplicate_enrollment_is_explained	Pass
42	test_portals.AdminPortalTests	test_enroll_enforces_the_semester_rule_and_succeeds_otherwise	Pass
43	test_portals.AdminPortalTests	test_gpa_report_is_ordered_by_weighted_gpa	Pass
44	test_portals.AdminPortalTests	test_room_double_booking_uses_the_unique_constraint_message	Pass
45	test_portals.AdminPortalTests	test_student_can_be_added_and_edited	Pass
46	test_portals.AdminPortalTests	test_student_cannot_take_a_faculty_username	Pass
47	test_portals.AdminPortalTests	test_teacher_double_booking_shows_a_friendly_message	Pass
48	test_portals.AdminPortalTests	test_valid_timetable_entry_is_added	Pass
49	test_portals.FacultyPortalTests	test_attendance_is_saved_and_updated	Pass
50	test_portals.FacultyPortalTests	test_internal_only_leaves_the_grade_empty	Pass
51	test_portals.FacultyPortalTests	test_marks_are_graded_by_the_trigger_not_python	Pass
52	test_portals.FacultyPortalTests	test_out_of_range_marks_are_rejected_per_row	Pass
53	test_portals.FacultyPortalTests	test_weekend_attendance_shows_the_database_rule	Pass
54	test_portals.SeedDemoUsersTests	test_rerunning_updates_instead_of_duplicating	Pass
55	test_portals.StudentPortalTests	test_defaulter_sees_the_warning_from_v_defaulter_list	Pass
56	test_portals.StudentPortalTests	test_student_in_good_standing_sees_no_warning	Pass
57	test_portals.StudentPortalTests	test_student_sees_only_their_own_courses	Pass
58	test_java_bridge.BuildCommandTests	test_arguments_are_validated	Pass
59	test_java_bridge.BuildCommandTests	test_argv_is_a_list_with_json_flag_and_utf8	Pass
60	test_java_bridge.BuildCommandTests	test_commands_off_the_allow_list_are_refused	Pass
61	test_java_bridge.BuildCommandTests	test_enroll_with_year	Pass
62	test_java_bridge.BuildCommandTests	test_year_is_validated_and_only_for_enroll	Pass
63	test_java_bridge.RunEngineTests	test_engine_error_type_comes_from_stderr_json	Pass
64	test_java_bridge.RunEngineTests	test_jar_missing_never_starts_java	Pass
65	test_java_bridge.RunEngineTests	test_java_missing	Pass
66	test_java_bridge.RunEngineTests	test_json_after_jvm_warnings_is_still_parsed	Pass

#	Module.Class	Test	Status
67	test_java_bridge.RunEngineTests	test_non_json_failure_is_internal	Pass
68	test_java_bridge.RunEngineTests	test_success_parses_stdout_and_runs_without_a_shell	Pass
69	test_java_bridge.RunEngineTests	test_success_without_json_is_bad_output	Pass
70	test_java_bridge.RunEngineTests	test_timeout	Pass
71	test_java_bridge.RunEngineTests	test_usage_error_keeps_exit_code_2	Pass